

System Software Design for Undiluted Attestation

Presented on 3rd July 2026

School of Computer and Communication Sciences
Data Center Systems Laboratory
Doctoral program in Computer and Communication Sciences

for the award of the degree of Docteur ès Sciences (PhD)

by

Charly Nicolas Lucien CASTES

Accepted on the jury's recommendation

Prof. C. Pit-Claudel, jury president
Prof. E. Bugnion, thesis director
Prof. E. Witchel, examiner
Prof. J. Nieh, examiner
Prof. G. Candea, examiner

Là où il y a une volonté, il y a un chemin.
— *Bernard GAILLOT, Commandant de promotion X16*

To my family...

Acknowledgments

This thesis is the final stop of a journey that started five years ago. Although common wisdom has it that a Ph.D. is a lonely endeavor, I have personally never felt that way thanks to all the people that supported me, shared part of the road, or with whom I could simply share a laugh and a good time.

First and foremost, I would like to thank my thesis advisor, Edouard Bugnion. Ed has been a fantastic thesis advisor, both in the context of research and beyond. Ed gave me the freedom to explore and pursue my own ideas, while occasionally nudging me back toward the right track when I went astray. Throughout my Ph.D. I have always felt empowered and supported, knowing Ed would be there should I need his guidance. Thank you, Ed!

Although not formally my co-advisors, I had two other mentors who played very important roles throughout my journey. The first is Adrien Ghosn. Adrien is an alumnus from the lab, and the one who taught me the most about hands-on systems research, first during a master project, and later through our long-lasting research collaboration. It has been a real pleasure and a lot of fun working with Adrien, and despite the ups and downs he always held the course. The second is Thomas Bourgeat. Thomas joined the EPFL faculty during the second half of my Ph.D., and had a great influence on extending the scope of my research toward formal methods. It has been incredibly insightful and enjoyable to work with Thomas, and a breath of fresh air for the last few years of my Ph.D. Adrien and Thomas, I would not have come this far without you, it has been a pleasure, and I hope we will get the opportunity to work together again in the future.

Then I would like to thank my thesis committee: Clément Pit-Claudel as president, George Candea as internal examiner, and Emmett Witchel and Jason Nieh as external examiners. Thank you Clément for running a smooth process, the interesting follow-up discussions, and the kind words. Thank you George for getting me interested in systems research in the first place through POCS. Thank you Emmett and Jason, your research has been a source of inspiration since the start of my Ph.D., and it feels like a real milestone for me as a researcher to have you in my thesis committee. Further, I would like to thank Jason for hosting me as a visiting Ph.D. in his lab for a semester, it has been a truly enriching experience.

In addition to some of the people mentioned above, the work presented in this thesis would not have been possible without my co-authors and collaborators, of whom I would like to give some special mentions. Andrew Baumann, for offering me the opportunity to join the Systems Research team at Google, and with whom I worked on core gapping and learned about ARM's internals. Nate Foster, for his insights and feedback on the verification of MIRALIS, as

well as for his support and the opportunities he offered me. And finally Marios Kogias and Jim Larus, for their help and feedback throughout the whole duration of the TYCHE project.

During those five years the DCSL lab has been my home. A big thank you to my lab mates, Neelu, Konstantinos, Yuchen, and Rui, my Ph.D. would not have been the same without you. Another big thank you to Maggy, who has always been there for us and makes DCSL feel like a family. Thank you also to all the great students I had the chance to work with: Michael, David, Filippo, Maëlys, Abel, Sofia, Frederic, François, Noé, and Lucie.

Finally, I would like to thank all the friends I made along the way, as well as those outside and before EPFL. My flatmate Gaétan, my friends from EPFL, from undergrad (or l'X, as we call it), as well as from high school, for all the chats, laughs, BBQs, and games that helped me stay sane during the last five years. The members of my family, my mother, father, sister, and brother, to whom I am forever grateful and without whom I would not be where I am today. And last but not least Marie, thank you sincerely for riding along.

★★★★★

I would also like to thank the people and organizations that financially supported this thesis, and allowed me to work on the problems that I thought important. My thesis was supported by EPFL, the MICROSOFT-EPFL Joint Research Center, and the SwissChips initiative.

Lausanne, May 2026

Charly Castes

Abstract

In trusted execution environments, the authenticity and security properties of a computer system are determined by examining its attestation. Reviewing the attestation should be sufficient to evaluate the system’s correctness; however, this is not the case today. Proprietary system software cannot be audited for security, while existing isolation abstractions fail to capture important nuances, such as the existence of transient-execution attacks.

In this thesis, we demonstrate that, through software-only changes, it is possible to design systems for which correctness can be derived solely from examination of their attestation, assuming only the correctness of hardware and of software that can be audited.

We address this problem in two steps. First, we ensure that system software can be relied upon for isolation and for producing the attestation. Then, that trusted system software can enforce strong isolation policies whose security properties can be verified from the attestation.

In [Part I](#) we tackle the challenge of proprietary software running at the highest privilege level — *i.e.*, in firmware mode. We introduce the concept of virtual firmware monitors, a new kind of system that can virtualize unmodified vendor firmware, and thus remove proprietary code from the highest privilege mode. We discuss the design and implementation of MIRALIS, a RISC-V virtual firmware monitor, and demonstrate the virtualization of unmodified firmware on three off-the-shelf RISC-V computers from different vendors. To ensure the isolation guarantees of virtual firmware monitors cannot be bypassed by the virtualized firmware, we propose a framework for automated verification of firmware mode virtualization, and discuss the verification of MIRALIS. Our framework leverages existing ISA specifications and lightweight formal verification tools, such as model checkers. As a result, we enable safe and efficient sandboxing of untrusted vendor firmware.

In [Part II](#) we focus on the isolation mechanisms and their integration with the system’s attestation. We propose core gapping, a new technique that provably prevents an attacker and a victim from being co-scheduled on the same core, eliminating all same-core attacks — and thus the majority of transient-execution attacks. We demonstrate its application to confidential VMs on ARM CCA platforms, and discuss how to integrate core gapping with minimal changes and performance overhead on existing systems. We then introduce the idea of capability-based security monitors, inspired by the design of micro-kernels, but with support for confidential memory and integration with the system’s attestation. We present the design and implementation of TYCHE, a capability-based security monitor for x86-64 and RISC-V. We show that TYCHE can reconstruct existing abstractions such as enclaves and confidential VMs, while also enabling new use cases including multi-domain attestation and shared confidential memory.

Keywords: trusted execution environments, attestation, zero trust, firmware, transient-execution attacks, security monitors, capabilities

Résumé

Dans les environnements d'exécution sécurisés, l'authenticité et les propriétés de sécurité d'un système informatique sont déterminées par l'examen de son attestation. L'examen de l'attestation devrait suffire à évaluer la conformité du système ; or, ce n'est pas le cas aujourd'hui. Les logiciels système propriétaires ne peuvent pas être audités pour des raisons de sécurité, tandis que les abstractions d'isolation existantes ne parviennent pas à capturer certaines nuances importantes, comme l'existence d'attaques par exécution transitoire.

Dans cette thèse, nous démontrons que, par des modifications logicielles uniquement, il est possible de concevoir des systèmes dont la correction peut être déduite uniquement de l'examen de leur attestation, en supposant uniquement la correction du matériel et des logiciels pouvant être audités.

Nous abordons ce problème en deux étapes. Premièrement, nous garantissons que les logiciels système peuvent être utilisés de manière fiable pour l'isolation et pour produire l'attestation. Ensuite, que ces logiciels système de confiance peuvent appliquer des politiques d'isolation strictes dont les propriétés de sécurité peuvent être vérifiées à partir de l'attestation.

Dans la [Partie I](#), nous abordons le défi des logiciels propriétaires s'exécutant au niveau de privilège le plus élevé — *i.e.*, en mode firmware. Nous introduisons le concept de moniteurs de firmware virtuels, un nouveau type de système capable de virtualiser un firmware vendeur non modifié, supprimant ainsi le code propriétaire du niveau de privilège le plus élevé. Nous discutons de la conception et de l'implémentation de MIRALIS, un moniteur de firmware virtuel RISC-V, et démontrons la virtualisation de firmware non modifié sur trois ordinateurs RISC-V du commerce provenant de différents vendeurs. Pour garantir que les propriétés d'isolation des moniteurs de firmware virtuels ne peuvent pas être contournées par le firmware virtualisé, nous proposons un cadre de vérification automatisée de la virtualisation en mode firmware, et discutons de la vérification de MIRALIS. Notre cadre exploite des spécifications ISA existantes et des outils de vérification formelle légers, tels que des model checkers. En résultat, nous permettons un sandboxing sûr et efficace de firmware vendeur non fiable.

Dans la [Partie II](#), nous nous concentrons sur les mécanismes d'isolation et leur intégration avec l'attestation du système. Nous proposons le « core gapping », une nouvelle technique qui empêche prouvablement qu'un attaquant et une victime soient co-ordonnés sur le même cœur, éliminant ainsi toutes les attaques sur le même cœur — et donc la majorité des attaques par exécution transitoire. Nous démontrons son application aux VM confidentielles sur les plateformes ARM CCA, et discutons de la façon d'intégrer le core gapping avec des modifi-

cations minimales et une surcharge de performance faible sur les systèmes existants. Nous introduisons ensuite l'idée de moniteurs de sécurité à base de capacités, inspirés de la conception des micro-noyaux, mais avec le support de la mémoire confidentielle et l'intégration avec l'attestation du système. Nous présentons la conception et l'implémentation de TYCHE, un moniteur de sécurité à base de capacités pour x86-64 et RISC-V. Nous montrons que TYCHE peut reconstruire des abstractions existantes telles que les enclaves et les VM confidentielles, tout en permettant de nouveaux cas d'utilisation incluant l'attestation multi-domaine et la mémoire confidentielle partagée.

Mots clés: environnements d'exécution sécurisés, attestation, zero trust, firmware, attaques par exécution transitoire, moniteur de sécurité, capacités

Contents

Acknowledgments	i
Abstract	iii
Résumé	v
Contents	vii
List of Figures	xiii
List of Tables	xvii
1 Introduction	1
1.1 Attestation Dilution	2
1.2 Thesis Goal	3
1.3 Approach	3
1.3.1 Removing Low-Level Firmware from the TCB	3
1.3.2 Blocking Transient Execution Attacks on Confidential VMs	4
1.3.3 Attestation-Isolation Co-Design	5
1.3.4 Holistic System Design for Undiluted Attestation	5
1.4 Thesis Statement	6
1.5 Thesis Contributions	6
1.6 Thesis Organization	7
1.7 Bibliographic Notes	8
2 Background	9
2.1 A Zero-Trust Computing Infrastructure	9
2.1.1 Definition: Zero-Trust Computing Infrastructure	9
2.1.2 Threat Model for a Zero-Trust Computing Infrastructure	10
2.1.3 Use Cases	11
2.2 Attestation	12
2.2.1 The Attestation Process	12
2.2.2 Attestation Dilution	13
2.2.3 Example: ARM CCA	14
2.3 Security Monitors	15
2.3.1 Principles	16
2.3.2 Primitives	16
2.4 A Short History of Trusted Execution Environments	17
2.4.1 Inception and Historical Context	18

2.4.2	– 1 st Generation: Hypervisor-based TEEs	19
2.4.3	– 2 nd Generation: Hardware-based TEEs	19
2.4.4	– 3 rd Generation: Hardware-Software Co-Design	21
Virtual Firmware Monitors		23
3	The Design and Implementation of a Virtual Firmware Monitors	25
3.1	The Anatomy of Firmware	26
3.2	Vendor Firmware Dilutes the Attestation	27
3.3	Threat Model	27
3.4	Virtual Firmware Monitors	28
3.4.1	Theory of Classical Virtualization	28
3.4.2	Firmware Virtualization	29
3.4.3	Access Control to System Resources	30
3.4.4	Fast Path Offloading	30
3.5	M-mode Virtualization with MIRALIS	31
3.5.1	CPU Virtualization	32
3.5.2	Physical Memory Protection Virtualization	33
3.5.3	Interrupts, Devices, and MMIO Virtualization	34
3.5.4	Comparison with Hypervisors	35
3.6	Policies	35
3.6.1	Policy Modules	36
3.6.2	Firmware Sandbox Policy	36
3.6.3	KEYSTONE Policy	37
3.6.4	ACE Policy	37
3.7	Security Analysis	38
3.8	Evaluation	39
3.8.1	Experiment Setup	39
3.8.2	Virtualizing Unmodified Firmware	39
3.8.3	Impact on OS Performance	40
3.8.3.1	Cost of MIRALIS Operations	40
3.8.3.2	Microbenchmarks	41
3.8.3.3	Application Benchmarks	43
3.8.3.4	Support for Isolation Policies	44
3.9	Discussion and Related Work	45
3.10	Conclusion	47
4	Lightweight Virtual Firmware Monitor Verification	49
4.1	Background	49
4.1.1	Trap & Emulate Hypervisors	49
4.1.2	Lifting Architecture Specifications	50

4.2	A Simulation Problem	51
4.3	Correctness Criteria	52
4.3.1	Modeling the Architecture	52
4.3.2	Faithful Emulation	53
4.3.3	Faithful Execution	54
4.4	Checking the Criteria	55
4.5	Automated Verification of MIRALIS	56
4.5.1	Preparing the Architecture Specification	56
4.5.2	Checking Faithful Emulation	57
4.5.3	Checking Faithful Execution of Loads and Stores	58
4.6	Bugs Found	58
4.7	Discussion	61
4.7.1	Lightweight Formal Methods	61
4.7.2	Formal Guarantees and Limitations	62
4.7.3	Comparison with Other Tools	63
4.8	Related Work	63
4.9	Conclusion	64
	Systems Software and Attestation Co-Design	65
5	Blocking Transient Execution Attacks with Core-Gapped Confidential VMs	67
5.1	Background	68
5.1.1	Confidential VMs	68
5.1.2	Transient-Execution Vulnerabilities	71
5.1.3	Core Gapping as the Basis of Isolation	73
5.1.4	Threat Model	74
5.2	Design	74
5.3	Implementation	76
5.3.1	ARM CCA	76
5.3.2	Dedicating Cores	77
5.3.3	Remote RMM Calls	78
5.3.4	Interrupt Management	80
5.4	Evaluation	82
5.4.1	Experimental Setup	82
5.4.2	Scaling a CPU-intensive Workload	84
5.4.3	I/O Performance	85
5.4.4	Application Benchmarks	85
5.4.5	Summary	87
5.5	Discussion	87
5.5.1	INTEL TDX	87

5.5.2	AMD SEV	88
5.6	Related Work	89
5.7	Conclusion	90
6	Composable Isolation as a Foundation to Manage Trust	91
6.1	Introduction	91
6.2	Background	95
6.3	Overview	98
6.3.1	Architecture	98
6.3.2	Threat Model	99
6.3.3	Running Example	100
6.4	TYCHE Design: API & Capabilities	101
6.4.1	Capability Derivation Trees	102
6.4.2	Memory Region Capabilities	102
6.4.3	Security Domain Capabilities	103
6.4.4	Attestation and Security Guarantees	106
6.5	Implementation	107
6.5.1	TYCHE Attested Boot	107
6.5.2	Capability Engine	108
6.5.3	TYCHE x86-64 Backend in Root Mode	109
6.5.4	TYCHE RISC-V Backend in M-Mode	110
6.6	Integrating Existing Software with TYCHE	110
6.6.1	Interfacing with TYCHE from LINUX Environments	111
6.6.2	Backward Compatibility	111
6.6.3	Preserving Runtime Behavior	112
6.7	Evaluation	113
6.7.1	Microbenchmarks	113
6.7.2	Enclaves, VMs, CVMs, and Composable Isolation	116
6.7.3	Case Studies	118
6.8	Related Work	119
6.9	Conclusion	121
	Future Work and Closing Remarks	123
7	Future Research Directions	125
7.1	Enforcing Isolation Policies at the OS Boundary	125
7.1.1	Dynamic Linkers Are the Narrow Waist of Operating Systems	126
7.2	Attestation Beyond the Application CPU	127
7.2.1	Full SoC Attestation	127
7.2.2	Rack-Scale Attestation	129

8 Conclusion	131
A Appendix	133
AA ARM CCA Attestation	133
Bibliography	135
Curriculum Vitae	161

List of Figures

Figure 2.1	Comparison between ambient authority (left), and zero-trust environments (right). With ambient authority, all systems within a security perimeter can freely interact with each other. In zero-trust environments, systems must prove their authenticity before interacting with others.	9
Figure 2.2	Overview of the attestation process.	13
Figure 2.3	Overview of a confidential VM (CVM) deployment on ARM CCA, from the point of view of the tenant of CVM 1. Software components in blue can typically be audited by the tenant and are measured as part of the attestation. Software components in yellow can not be audited by the tenant, and only the EL3 software (AP_BL31) is part of the attestation.	14
Figure 2.4	A selection of notable academic and industrial TEEs in recent history. . . .	19
Figure 2.5	Overview of ARM CCA	21
Figure 3.1	Comparison of TEE deployments on RISC-V. With KEYSTONE [166] and most TEEs the security monitor is co-located with the vendor firmware. DORAMI [159] is a recent attempt at privilege separation that requires firmware refactoring and binary scanning. Our solution achieves privilege separation with no firmware modification.	26
Figure 3.2	Classical vs. firmware virtualization.	28
Figure 3.3	Distributions of M-mode trap causes over time during LINUX boot on the VISIONFIVE 2. Only five causes account for 99.98% of all traps.	30
Figure 3.4	Overview of MIRALIS: Firmware executes in virtual M-mode and OS traps and associated devices are emulated. Green boxes are verified via symbolic execution (see Chapter 4).	33
Figure 3.5	Physical PMP access rights configuration for memory virtualization in the different execution modes. RWX stands for read-write-execute, --- for no permission.	33
Figure 3.6	Modified boot-flow with the MIRALIS VFM.	39
Figure 3.7	Relative COREMARK-PRO scores.	42
Figure 3.8	IOZONE throughputs on the VISIONFIVE 2.	42
Figure 3.9	MEMCACHED latency distribution.	42
Figure 3.10	Relative performance comparison for various application workloads. . . .	44
Figure 3.11	Relative performance of KEYSTONE enclaves implemented by MIRALIS on RV8.	45
Figure 4.1	The host and virtual hardware interfaces follow the same hardware specification, but with different configurations.	50
Figure 4.2	The reference (left) and host (right) machines.	51

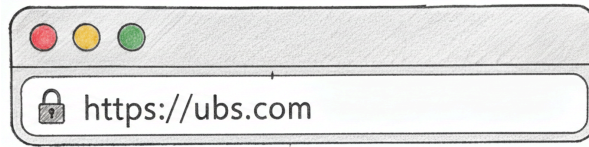
Figure 4.3	Lock-step execution between a VM and the corresponding reference machine. For each instruction, the VM can either execute the step directly or trap to the VFM for emulation.	52
Figure 4.4	The faithful emulation criteria. A VFM implementation properly emulates the virtual hardware interface if for any state and instruction the resulting state is equivalent to what a reference machine would produce.	54
Figure 4.5	The faithful execution criteria. A VFM must configure the host machine such that direct execution produces observable results similar to a reference machine.	55
Figure 4.6	Off-by-one error in the immediate decoding of compressed store instructions.	59
Figure 4.7	Adding support for the C (compressed instructions) extension changes valid bit patterns in some configuration registers.	60
Figure 4.8	The “lock” bit (0x80) is missing from the bitmask used to convert virtual PMP registers into the physical registers.	60
Figure 5.1	Generic architecture support for confidential VMs [20, 138, 253]. A trusted security monitor protects CVMs, which may run on any core. Cores context switch between trusted and untrusted domains in response to VM entry/exit.	68
Figure 5.2	Core-gapped confidential VMs (compare to Figure 5.1). Physical cores are bound to specific CVMs; VM exits are handled by cross-core RPCs rather than context switching.	70
Figure 5.3	Transient-execution vulnerabilities and CPU bugs that break security isolation. Only NetSpectre [263] and CrossTalk [231] demonstrated cross-core leaks in typical cloud VM settings.	71
Figure 5.4	Asynchronous call mechanism. A wake-up thread (new to our design) polls RPC channels to wake runnable vCPU threads in response to an IPI.	78
Figure 5.5	The host observes and manipulates a virtual list containing a subset of the guest interrupts, allowing the RMM to inject interrupts without perturbing host operations.	80
Figure 5.6	CoreMark-PRO [87] scaling for shared-core (baseline) VMs and core-gapped CVMs.	84
Figure 5.7	Scaling to multiple VMs. We plot the aggregate CoreMark-PRO score for an increasing count of 4-core VMs/CVMs.	84
Figure 5.8	NetPIPE [269] TCP results, 1500-byte IPv6 packets.	85
Figure 5.9	IOZONE [212] sync read/write throughput to a virtio block device, using 0_DIRECT to bypass the guest’s cache.	85
Figure 5.10	LINUX kernel build; virtio disk & 48 GiB RAM	85
Figure 6.1	Composing enclaves and sandboxes within a CVM. Left: INTEL TDX intra-isolation, with gVisor [316] sandboxing both an application (sandbox) and	

	a VMM-provided enclave. Right: our approach. Arrows indicate management dependencies.	93
Figure 6.2	Mutual-distrust LLM inference deployment on top of TYCHE. The left side shows the participating SDs—SD0 hypervisor, SD1 CVM, and SD2 nested enclave. The right side shows the machine’s physical memory address space as SDs are created; the colors indicate the regions accessible to each SD on the left, distinguishing exclusive regions (X) from shared ones (S). The encrypted model, prompts, and replies traverse memory along the black path, while interrupts are routed as shown by the red arrows.	98
Figure 6.3	Memory region derivation tree and SD memory views based on Figure 6.2 . New regions are created by carving and aliasing an existing region. Sends between SDs are omitted. (S)hared and e(X)clusive memory is reported on views.	102
Figure 6.4	Capability operations to create and configure SD1.	106
Figure 6.5	The capability engine maintains the system state across all cores.	106
Figure 6.6	Superposed system and SD views of the deployments benchmarked in Figure 6.10 . SDs are blue rounded boxes. System abstractions (processes, kernels, and VMs) are full rectangles, and libraries and drivers dotted ones.	113
Figure 6.7	Create and revoke latencies as a function of size for carve, with and without hash and clean, and alias SD memory.	113
Figure 6.8	RV8 RISC-V CPU microbenchmarks comparison between native, unmodified KEYSTONE, and TYCHE.	114
Figure 6.9	COREMARK-PRO performance relative to bare-metal LINUX (Native) for different SD deployments and varying number of cores on x86-64 (raw native score at the top).	114
Figure 6.10	Performance relative to native execution (green) for real-world applications isolated as SDs on x86-64 (blue).	116
Figure 6.11	REDIS GET latency distribution as measured by <code>memtier</code> (max throughput) during Figure 6.10 ’s experiment.	117
Figure III.1	Example of ARM CCA deployment incorporating the ideas presented in this thesis, from the perspective of the tenant of CVM 1.	123
Figure 7.1	Scalable seccomp sandboxing. A seccomp module in the dynamic linker aggregates the authorized system calls based on symbol resolution and a per-file <code>.seccomp</code> section.	126
Figure 7.2	EIC7700X SoC block diagram. The application CPU is composed of the four RISC-V RV64GC cores.	128
Figure A.1	Content of an ARM CCA attestation [21 , 252].	133

List of Tables

Table 2.1	System software measurements in the ARM CCA attestation.	15
Table 3.1	MIRALIS lines of code decomposition.	35
Table 3.2	Characteristics of our evaluation platforms	39
Table 3.3	Overhead of MIRALIS operations in cycles.	41
Table 3.4	Cost of timer read and IPI	41
Table 4.1	Symbol definitions	52
Table 4.2	Verification time of the emulation pipeline.	57
Table 4.3	List of bugs found during verification. Bugs of high severity enable privilege escalation, bugs of medium severity can crash MIRALIS, bugs of low severity corrupt the state of the VM but do not compromise security.	58
Table 5.1	Cost of timer read and IPI	68
Table 5.2	Comparison of null RMM call latencies.	79
Table 5.3	Virtual interprocessor interrupt latency.	82
Table 5.4	Interrupt delegation effect on CoreMark-PRO.	84
Table 5.5	REDIS benchmark. 50 clients, 512-byte objects.	85
Table 6.1	Nested composition of isolation abstractions in prior work. Rows denote the outer isolation boundary and columns the inner boundary. Prior systems support only specific combinations, each requiring dedicated hardware or software extensions, motivating the need for a unified composable isolation solution.	94
Table 6.2	TYCHE’s API	101
Table 6.3	Average switch latency and standard deviation, including monitor entry+exit (hardware privilege transitions).	113
Table 6.4	Network benchmarks configuration.	115

1 Introduction



Deciding whether or not to trust a remote computer is a problem that we face on a daily basis. Take visiting a bank website as an example. Before typing in their credentials, one should verify the website's URL and look for the small lock icon as a hint that this website can reasonably be trusted. Similarly, computers also need to be able to decide whether or not to trust other computers. Operations such as downloading a software update or transferring sensitive data require high assurance that the remote computer is behaving correctly.

The demand for trustworthy computing has never been higher. The rapid progress of artificial intelligence has created digital assets of extraordinary value: training a frontier model can cost hundreds of millions of dollars [78, 271], and the resulting weights must remain confidential even when deployed on third-party infrastructure. At the same time, growing geopolitical tensions have prompted governments to invest in sovereign computing infrastructure, demanding verifiable security guarantees for both civilian and defense systems [82, 91].

The way to establish trust in a specific computer is by reviewing a set of evidence called the *attestation*. For a web server, the lock icon is a materialization of the server's SSL certificate [238], a form of attestation that provides evidence that the machine is operated by the entity who registered the website's domain name. For use-cases with stricter security requirements, the attestation must provide sufficient evidence to reason about the correctness of the system. Typically, such an attestation would identify a specific machine or hardware manufacturer, and include hashes of both the system and application software [93, 189].

Applications requiring the strictest security requirements are executed in *trusted execution environments* (TEEs), *i.e.*, environments whose integrity and confidentiality are provably protected from the rest of the system. Examples of TEEs include the enclaves used by smartphones and laptops to store sensitive biometric identifiers [19, 111], or the confidential VMs offered by cloud providers [151]. TEEs have gained widespread adoption in the last two decades, to the point where all major architectures have introduced dedicated hardware extensions [5, 22, 136, 138, 175].

Despite their names, it is not sufficient to run an application in a TEE for it to be secure. Rather, TEE hardware extensions provide a new set of tools to systems designers: a hardware root of trust, attestation protocols, and new isolation primitives. These new tools do not automatically remove the threats that existed before, such as an operator abusing its privileges through

firmware-level attacks [65, 288], or side-channels and transient execution attacks [47, 50, 54, 203, 311]. Rather, it is up to the system designer to leverage those new primitives to build a secure system, according to a given threat model.

In this thesis we study how to leverage TEE primitives to achieve *undiluted attestation*, that is, to design computer systems such that security properties can be *logically deduced* from their attestation rather than relying on *unverifiable assumptions*. We show that through software-only changes and minimal assumptions, trust in computer systems can be derived solely from studying their attestation.

1.1 Attestation Dilution

To be trustworthy, a computer system should be able to prove its correctness and security properties. For a system to be correct, the totality of hardware, firmware, and software that are critical to the security of the system must themselves be correct and behave as intended. Such a set of components is called the *trusted computing base* [210], or TCB for short. The bigger the TCB the more chances there are to be bugs or vulnerabilities [68, 265], and the harder it is to reason about or formally verify the correctness of the system [98, 154, 175]. Beyond the size, it also matters who controls the individual components of the TCB. For instance, if a cloud provider is considered the adversary, it is important that none of the TCB of a confidential VM is under direct control of the cloud provider.

Therefore, a system should be able to prove — through its attestation — the correctness of its entire TCB. The purpose of TEEs is specifically to reduce the TCB and to make it explicit, such that, in theory, the correctness of a system can be mathematically derived from the attestation, with the only assumption being hardware correctness. In practice, however, the attestation is often not sufficient to assess the system correctness by logical deduction alone — a phenomenon that we refer to as *attestation dilution*.

A system's attestation is diluted when the attestation's evidences cannot stand on their own to assess the system's correctness. Consider a system whose firmware, security monitor, or paravisor [151] can not be publicly audited: the software hashes in the attestation only identify specific binary versions, but say nothing about their correctness. Attestation dilution can arise for other reasons, such as a mismatch between the threat model and isolation guarantees, *e.g.*, lacking protection for a class of attacks, or because the TCB includes software that is not deemed trustworthy, such as a full OS in a confidential VM to protect a single high-value key.

In the absence of a tight co-design between a computer system and its attestation mechanism, the user of the system is left with no choice but to take on additional risks, in the form of increased TCB and set of assumptions.

1.2 Thesis Goal

The goal of this thesis is to enable *undiluted attestation*, that is, to design systems in such a way that their correctness can be logically deduced from their attestation. To that end, we focus on reducing attestation dilution through systems software design.

Throughout this thesis we make two fundamental assumptions: that *hardware is correct*, and that *software that can be audited is correct* too. That hardware is correct does not mean it is perfect — and we will discuss the case of micro-architectural leaks — but at least hardware is assumed to implement its specification and can be relied upon for hardware root of trust. That some software is assumed to be correct is a simplification, as full formal verification is outside the scope of this thesis. Yet, formal methods have seen great success in the area of systems software [118, 154, 165, 175, 313], and recent successes of AI agents in the field of mathematics [32] hint toward a future where formalization efforts can be scaled-up.

From those assumptions, we investigate how to design systems software in such a way that no additional assumption is required to establish a system’s correctness from its attestation.

1.3 Approach

Specifically, we identify three problems that need to be solved to achieve undiluted attestation: (1) remove proprietary firmware from the TCB, (2) provide isolation guarantees against side-channels and transient-execution attacks, and (3) reduce the size of the system’s TCB through attestation-isolation co-design.

1.3.1 Removing Low-Level Firmware from the TCB

An important cause of attestation dilution is the presence of proprietary software in the TCB. Although TEEs enable removing the host OS or hypervisor from the TCB, they usually rely on low-level proprietary firmware to enforce isolation [286]. For instance, ARM CCA relies on proprietary EL3 firmware for correctness (§ 2.2.3). Low-level firmware conflates security and non-security-related functions [159], and because the non-security critical functions include proprietary logic, the whole firmware needs to be proprietary, therefore diluting the attestation. We argue that this is not necessary, and that in fact low-level firmware can be safely and efficiently de-privileged.

We introduce the concept of *virtual firmware monitors*, a new kind of system that enables running unmodified vendor firmware in user-space. Virtual firmware monitors rely on classical “trap & emulate” virtualization, but virtualize firmware mode instead of traditional kernel mode. We design and implement MIRALIS, a RISC-V virtual firmware monitor, and demonstrate the virtualization of unmodified vendor firmware on three different RISC-V boards.

Further, we propose a new framework leveraging lightweight formal methods to verify functional correctness of firmware mode virtualization. Our framework leverages existing high-quality executable hardware specifications and symbolic execution tools to automate the verification process. We apply our framework to verify functional correctness of the virtualization subsystems of MIRALIS, including emulation of all possible instructions, interrupt delivery, and memory protection.

Virtual firmware monitors enable decoupling proprietary logic implemented in closed-source firmware from isolation enforcement, which then becomes the prerogative of the virtual firmware monitor itself. As a result, the proprietary firmware is removed from the TCB, reducing attestation dilution by virtue of the open nature of virtual firmware monitors.

1.3.2 Blocking Transient Execution Attacks on Confidential VMs

A system’s attestation also gets diluted when isolation mechanisms are insufficient to protect from a given threat — *i.e.*, the attestation is not sufficient to reason about the system’s security. The initial discovery of Meltdown [184] and Spectre [156] in 2018 demonstrated that hardware-enforced isolation boundaries were, in fact, porous, challenging the core assumptions of system design. Later attacks have demonstrated that TEEs are similarly affected by transient-execution attacks [47, 203], leading to leaked hardware keys enabling attackers to sign fraudulent attestations [50], thus shattering the foundation upon which trust is established.

Therefore, in contexts where transient-execution attacks are a concern, such as the public cloud [325], the attestation should carry evidence of protection from these threats, the same way it carries evidence of adequate memory protection. Because transient-execution and side-channel attacks rely on shared micro-architectural state between the victim and attacker [326], we remark that pinning security contexts on different cores prevents most of those attacks. We propose a new technique called *core gapping*, which provides the attestable guarantee that an attacker and a victim never share the same physical CPU core. In particular, that the host hypervisor never executes on cores dedicated to tenant VMs.

We present the design and implementation of core-gapped confidential VMs on ARM CCA [20]. We propose new techniques and optimizations to minimize changes to the host hypervisor and execution overhead, such as interrupt list virtualization and re-purposing LINUX’s CPU hotplug mechanism. In total, our implementation requires a 1030-line modification to the host hypervisor, and no changes to the guest VM. Further, we deploy and evaluate our implementation on a hyperscaler-grade ARM server, demonstrate negligible overhead, and the ability to scale to large core counts.

Because core-gapping is enforced by the security monitor whose hash is part of the attestation, core-gapping provides attestable isolation from all attacks that require the attacker and the victim to be co-scheduled on the same physical CPU core. Although not sufficient to pro-

tect from all possible side-channel threats (such as last-level caches leakage), core-gapping offers protection for the vast majority of transient-execution attacks, therefore significantly reducing the attack surface, TCB, and attestation dilution.

1.3.3 Attestation-Isolation Co-Design

In some cases, attestation dilution is caused by a mismatch between the use case and available TEE abstractions. For instance, when an enclave would be the ideal abstraction, but the platform offers only confidential VMs [323, 324], or when the platform operator imposes the use of a paravisor [197]. Because the requirements vary across use-cases, significant efforts have been invested in working around the limitations of one-size-fits-all solutions [30, 40].

Instead of providing point solutions, we argue that systems should focus on exposing low-level mechanisms that enable the creation of tailored abstractions and their integration within the system’s attestation. Capability-based micro-kernels are typical examples of systems that embrace the separation of mechanisms from policies [88], yet there is no such equivalent in the context of TEEs.

We present the design and implementation of TYCHE, a capability-based security monitor targeting x86-64 and RISC-V. We discuss how micro-kernel capabilities need to be extended in order to track confidential memory, and how to perform attestation on capability-based systems. We demonstrate that capability-based security monitors can support existing abstractions and runtimes, and further support new use-cases, such as shared confidential memory, nesting of TEEs, or multi-TEE attestations.

By co-designing attestation and isolation primitives, capability-based security monitors empower system designers to tailor isolation to specific use-cases. They enable nesting of TEE abstractions, sharing of confidential memory, and attestation of multiple components simultaneously. Through fine-grained control of the TCB and resource attestation, capability-based security monitors simplify the design of systems with low or no attestation dilution.

1.3.4 Holistic System Design for Undiluted Attestation

Combining the techniques developed in this thesis, we can design systems with undiluted attestation. Virtual firmware monitors ensure that all system software in the TCB can be included in the attestation and audited. Core-gapped confidential VMs provide a practical approach to protect the most sensitive workloads from pervasive side-channel and transient-execution attacks. Finally, capability-based security monitors provide the flexibility to design and attest systems with tailored isolation abstractions.

1.4 Thesis Statement

In summary, the authenticity and security properties of a computer system are determined by examining the attestation evidence. Reviewing the attestation should be sufficient to evaluate the system's correctness, however this is not the case today. The presence of proprietary firmware, the threat of transient-execution attacks bypassing protections, and the mismatch between isolation requirements and TEE abstraction result in risks that are not captured by the attestation.

Thesis Statement

Through software-only changes it is possible to design systems for which correctness can be derived solely from examination of their attestation, assuming only the correctness of hardware and of software that can be audited.

1.5 Thesis Contributions

This thesis makes the following contributions:

Secure and efficient sandboxing of untrusted firmware:

- We propose the concept of *virtual firmware monitors*, a new kind of system based on the theory of classical virtualization. Virtual firmware monitors enable secure and efficient de-privileging of unmodified vendor firmware.
- We describe the design and implementation of MIRALIS, and demonstrate by construction that the RISC-V ISA is compatible with virtual firmware monitors. As a practical application, we demonstrate the virtualization of unmodified firmware on three RISC-V platforms.
- We propose a new framework for automated verification of the functional correctness of firmware mode virtualization, and verify the virtualization subsystem of MIRALIS.

Protecting confidential VMs from same-core attacks:

- We propose *core-gapped confidential VMs*, a set of software-only modifications that ensure that no distrusting code shares a core, thus protecting confidential VMs from all same-core side-channels and transient-execution vulnerabilities.
- We show how to enable core-gapped confidential VMs on ARM CCA with minimal changes to the host hypervisor and security monitor, leveraging existing mechanisms such as CPU hotplug and by proposing new techniques such as interrupt list virtualization.
- We propose performance optimizations, such as asynchronous calls and interrupts delegation, and demonstrate that not only does core-gapping offer performance competitive

with existing VMs, the greater locality achieved by avoiding shared cores improves performance for CPU-intensive workloads.

Attestable and Composable Isolation Primitives:

- We explore the design space between micro-kernels and security monitors, and propose a new design that provides the flexibility of capability-based isolation with the rigor of attestation necessary for TEEs.
- We discuss the design and implementation of TYCHE, a capability-based security monitor. We show how such a design can be adapted to different execution environments, such as an x86-64 hypervisor or a RISC-V firmware, and how to leverage capability-based isolation with existing runtimes, such as KVM or enclaves library OSes.
- We demonstrate that capability-based security monitors can not only support existing TEE abstractions, such as enclaves and confidential VMs, but can also compose them in new ways. Capability-based security monitors further enable new use-cases, such as shared confidential memory and multi-TEE attestations.

1.6 Thesis Organization

This thesis is organized as follows. In [Chapter 2](#) we go through the necessary background on trusted execution environments that we depend on in this thesis. The rest of the thesis is organized in three parts:

1. [Part I](#) discusses the problem of untrusted firmware running at the highest privilege level on the CPU. [Chapter 3](#) introduces the concept of virtual firmware monitor, and discusses the implementation of MIRALIS. [Chapter 4](#) presents a framework for automated verification of firmware mode virtualization, and discusses the verification of MIRALIS.
2. [Part II](#) discusses the problem of TEE abstractions and isolation guarantees. [Chapter 5](#) discusses the problem of side-channels and transient-execution attacks, and proposes a new technique to protect confidential VMs from same-core transient-execution attacks. [Chapter 6](#) discusses the diversity of TEE abstractions, and describes the design of TYCHE, a capability-based security monitor that supports multiple abstractions within a single security monitor.
3. [Part III](#) describes future research directions and concludes the thesis.

1.7 Bibliographic Notes

This thesis primarily builds upon the ideas presented in the following publications:

- **TYCHE: Composable Isolation as a Foundation to Manage Trust in the Cloud** EuroS&P'26
Charly Castes^{*}, Adrien Ghosn^{*}, Neelu S. Kalani, Yuchen Qian, Marios Kogias, Edouard Bugnion
- **The Design and Implementation of a Virtual Firmware Monitor** SOSPP'25
Charly Castes, François Costa, Neelu S. Kalani, Timothy Roscoe, Nate Foster, Thomas Bourgeat, Edouard Bugnion
- **Lightweight Hypervisor Verification: Putting the Hardware Burger on a Diet** HotOS'25
Charly Castes, François Costa, Nate Foster, Thomas Bourgeat, Edouard Bugnion
- **Sharing is leaking: blocking transient execution attacks with core-gapped CVMs** ASPLOS'24
Charly Castes, Andrew Baumann
- **Creating Trust by Abolishing Hierarchies** HotOS'23
Charly Castes^{*}, Adrien Ghosn^{*}, Neelu S. Kalani, Yuchen Qian, Marios Kogias, Mathias Payer, Edouard Bugnion

In addition, parts of the motivation, discussion, and future works build or expand on ideas presented in the following workshop publications:

- **On Attestation Dilution** SysTEX'26
Charly Castes
- **Dynamic Linkers Are the Narrow Waist of Operating Systems** PLOS'23
Charly Castes, Adrien Ghosn
- **Attestable Software Versioning for Stateful Confidential Computing** SysTEX'22
Charly Castes, Edouard Bugnion

2 Background

This thesis focuses on reducing attestation dilution through software-only changes. In this chapter we provide the general background and context in which the thesis takes place. In particular, we provide an introduction to the main concepts we will encounter through the following chapters: the notion of zero-trust ([Section 2.1](#)), the attestation mechanism ([Section 2.2](#)), security monitors ([Section 2.3](#)), and a short history of trusted execution environments ([Section 2.4](#)).

2.1 A Zero-Trust Computing Infrastructure

The goal of this thesis is to enable the creation of computer systems with undiluted attestation, which can serve as the foundation for a secure computing infrastructure. In this section we define what we mean by secure computing infrastructure. In particular, we explain how the concept of zero-trust can be extended through attestation, provide an overview of the threat model, and provide examples of use cases for a truly zero-trust infrastructure.

2.1.1 Definition: Zero-Trust Computing Infrastructure

The concept of *zero-trust* is a design principle used in IT security, which can be summarized as “never trust, always verify” [248]. Traditional IT systems rely on a security perimeter, such as a physical data center or VPN, within which any device can interact with any other device. The idea of zero-trust is to remove ambient authority within the security perimeter, and instead mandate authentication before any two devices can start interacting, to the point that the security perimeter becomes optional. [Figure 2.1](#) illustrates the comparison between ambient authority and zero-trust environments. A zero-trust IT architecture provides stronger guarantees: an attacker gaining access to the security perimeter will be blocked from compromising further systems. Google’s BeyondCorp [221] is an example of a zero-trust IT architecture.

In the context of traditional IT infrastructure, zero-trust is enforced at the application layer. The implicit assumption is that system software is correct and behaving accordingly, an assumption that is incompatible with the strict security requirements of TEEs.

In the context of this thesis, we aim to lay the foundations for a stricter zero-trust model in which authentication is replaced by the verification of the correctness of a system through

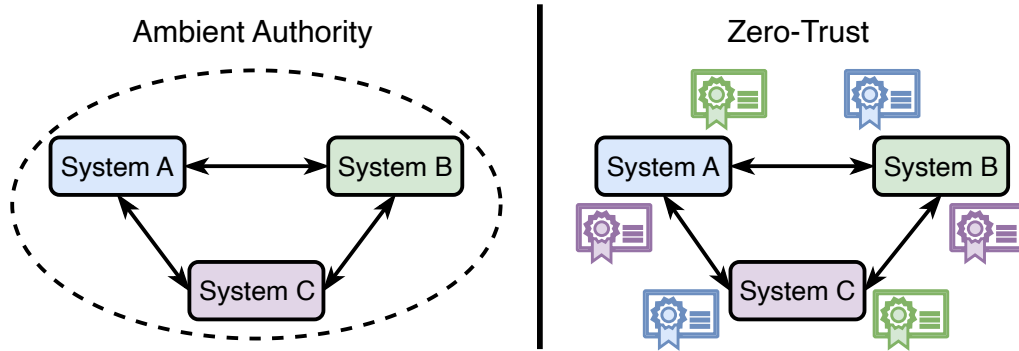


Figure 2.1 – Comparison between ambient authority (left), and zero-trust environments (right). With ambient authority, all systems within a security perimeter can freely interact with each other. In zero-trust environments, systems must prove their authenticity before interacting with others.

the attestation process, as described in [Section 2.2](#). Because zero-trust now encompasses the whole computer system, rather than only the application layer, we refer to that concept as a *zero-trust computing infrastructure*. As discussed in the introduction, we make two fundamental assumptions: that *hardware is correct*, and that *software that can be audited is correct* too. From there, we study how to design a computing infrastructure that requires no further assumptions to formally verify its correctness from inspection of the system’s attestation.

2.1.2 Threat Model for a Zero-Trust Computing Infrastructure

The purpose of a zero-trust computing infrastructure is to enable a *challenger* to verify the correctness and security properties of a set of TEEs through the inspection of their attestations. The challenger might be a cloud tenant renting infrastructure, or the owner of the machines themselves.

We consider a strong adversary with the ability to load arbitrary programs on the machine: the purpose of attestation is to reject insecure configurations. This includes the ability to load arbitrary firmware, system software, or applications. However, once correct software is installed — any software that can be audited by the challenger according to our hypothesis — the adversary cannot hijack the control of that program. In other words, vulnerabilities such as remote code execution or privilege escalation are out of scope. The adversary has full control over the network, and can exploit hardware side-channels. Physical attacks are out of the scope of this thesis: memory encryption provides some guarantees, but it is hard to protect from all physical attacks in practice [[193](#), [194](#)].

Per our assumptions, we take the hardware to be correct, *i.e.*, to respect its specification, although side-channels can exist. In particular, we assume the hardware root of trust to be correct, which will serve as the anchor for the attestation process. Furthermore, as discussed above, all software that can be audited by the challenger is considered to be bug-free and to adhere to its specification.

Example 1: Confidential VMs in the Cloud

Consider the case of deploying confidential VMs in the cloud. In this case the challenger is the tenant deploying VM images, while the adversary encompasses the cloud provider and other tenants. The adversary can insert backdoors in the firmware, security monitor, or VM image. It can leverage the hypervisor to control VM scheduling and collocate adversarial VMs, trigger interrupts, or amplify side-channels. The adversary can control the network and carry out replay attacks.

Example 2: Corporate Data Center

Consider a private data center. In this case the challenger is the data-center owner, and the adversary encompasses any software providers, such as VMWARE or MICROSOFT, compromised machines (*e.g.*, rootkits), or attackers on the network.

2.1.3 Use Cases

The primary use cases for a zero-trust computing infrastructure fall into two categories: new algorithms enabled by the verifiability of the system's operations, and high-stakes environments where security is paramount.

Verifiable computation — through inspection of a system's attestation — enables the design of new distributed algorithms, and can improve the efficiency of existing ones. The canonical example to benefit from TEEs is secure multiparty computation, which traditionally relies on advanced and computationally expensive cryptography [110], but can trivially be achieved through TEEs [72, 186]. The Signal messenger application's private set intersection for contact discovery [267] is an example of deployment of such algorithms. Similarly, TEEs also benefit BFT consensus algorithms [120, 202] and smart contracts [312]. While promising, the algorithmic guarantees assume the absolute correctness of the underlying systems, instead of relying on established cryptography or BFT guarantees.

Another important use case is the need for highly secure and confidential infrastructure, either because of regulatory requirements, or because of the value or sensitivity of data and IP being processed. For instance, the European Union's Digital Operational Resilience Act [92] mandates the use of encryption in use (as seen in some TEEs) for sensitive industries, while mandatory attestation of system software is already widely deployed within hyperscalers' infrastructure [39]. AI and machine learning applications also show growing interest in zero-trust computing infrastructure. Private AI inference, such as seen in APPLE's Private Cloud

Compute [18] and competing services [207, 282], leverages zero-trust infrastructure and TEEs to guarantee the confidentiality of users' input. On the other hand, AI model providers rely on third-party infrastructure to train and serve their models, and require strong guarantees of confidentiality for the model weights, which can be worth north of hundreds of millions of dollars [271].

For all use cases, the correctness of the underlying computer infrastructure is crucial, either for the correctness of algorithms, or to guarantee the security of data and assets. As a result, there is a pressing need to provide strong—ideally mathematical—guarantees of a system's correctness, from low-level firmware all the way to the application layer.

2.2 Attestation

The goal of the attestation process is to provide evidence that can be used to evaluate a system's correctness, and thus serves as the backbone of a zero-trust computing infrastructure. In this section we provide the necessary background on attestation, discuss the problem of attestation dilution, and provide an example of real attestation.

2.2.1 The Attestation Process

Before describing the attestation process we must first understand the content of an attestation. Typically, an attestation is composed of three sections: a hardware identity token, a list of hashes of the system software TCB (also called *measurements*), and a report written by the security monitor:

- **Hardware identity:** The hardware token identifies a specific machine, or at least the hardware manufacturer and the machine model. This is done through a per-machine private key fused at manufacturing time, with the support of a public key infrastructure (PKI) [74, 132].
- **System Software TCB:** The TCB comprises all software in the boot chain until—and including—the security monitor. The boot process starts from read-only memory (ROM), and builds a chain of measurements with each stage computing the measurement of the next. Hashes are stored in a tamper-proof append-only hardware log, typically a trusted platform module [285] (TPM), and signed with the hardware key on demand.
- **Security monitor report:** The security monitor report details the specific configuration of the system or TEE being attested. Typically, this includes a measurement of the initial TEE content or boot drive, as well as the specific configuration such as the isolation and security measures in place.

During the attestation process, a *challenger* tries to verify the correctness of a particular system. The challenger can be either a person or a program. Sometimes, a distinction is made between remote and local attestation, depending on whether the challenger is a program

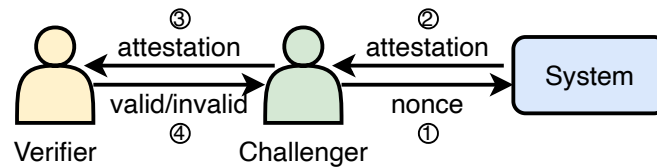


Figure 2.2 – Overview of the attestation process.

executing on the same machine or not. In the context of this thesis we will not make the distinction, unless explicitly stated.

In addition, the attestation process might involve one or more *verifiers*. Verifiers are external services with the ability to verify a subset of the attestation evidence. Hardware vendors such as INTEL [139] and platform providers such as MICROSOFT AZURE [196] offer verification services. Verifiers are not strictly needed; the hardware vendor root certificate is enough to verify the validity of an attestation [6]. Verifiers can be used for convenience, such as keeping up-to-date with the latest firmware patches, or because part of the attestation cannot be evaluated by the challenger itself, *i.e.*, when the attestation is diluted, as we will discuss shortly.

Figure 2.2 provides an overview of the attestation process. The challenger initiates the attestation process by sending a nonce to the system (step ①). The nonce serves to ensure freshness; it prevents replay attacks based on old but valid attestations. The system then builds an attestation by creating a security monitor report and passes it to the hardware. The hardware concatenates the hardware identity token, the system software measurements, and the report, signs the resulting document with the hardware key, and sends it to the challenger (step ②). The challenger might be able to evaluate the attestation on its own, or rely on an external verifier to assess part or all of the attestation evidence (steps ③ and ④).

2.2.2 Attestation Dilution

An attestation is not always sufficient to fully reason about the behavior of a computer. The challenger might not be able to verify some measurements—for instance in the case of closed source software—or the system might not offer sufficient isolation to protect an application from other software. In such cases the challenger has to make additional unverifiable assumptions. This is what we call *attestation dilution*.

In the case of closed source software, the software authors can either publish the expected measurements, or act as external verifiers, allowing the challenger to verify the software identity even without access to the source code. In both cases, the software authors must be trusted, preventing deployments where such assumptions are not acceptable. Confidential computing in the cloud is one such example, where the cloud provider is both untrusted and the verifier for proprietary software [38, 198]. To enable undiluted attestation, there are only

two options: to open-source the proprietary software, or to re-design systems in such a way that the proprietary software no longer needs to be trusted.

The other main cause of attestation dilution is the lack of sufficient isolation guarantees. For instance, some applications require nested trusted execution environments [133]. Others impose threat models that cannot be accommodated by existing systems, such as protection from transient execution attacks. Ensuring the attestation remains meaningful for those applications requires co-designing systems software and attestation to support new kinds of isolation guarantees.

2.2.3 Example: ARM CCA

To give a concrete example of attestation dilution, let us consider the attestation from an ARM CCA [20] confidential VM. For reference we provide a complete attestation in annex, in Figure A.1 on page 133. We also illustrate the deployment in Figure 2.3.

In total, a CCA attestation contains nine measurements and four configuration hashes [21, 252], which we summarize in Table 2.1. Of the nine measurements, four are hashes of software running on the application processor (*i.e.*, the CPU running the confidential VMs).

Of the four measurements on the application processor, only one — RMM¹, a small trusted monitor — corresponds to software that is typically auditable by the challenger; the others are proprietary firmware. The AP_BL1 and AP_BL2 measurements correspond to boot loaders, while AP_BL31 is the EL3 runtime firmware and security monitor, which executes with higher privileges than the RMM, and has therefore full access to the confidential VMs' memory. As

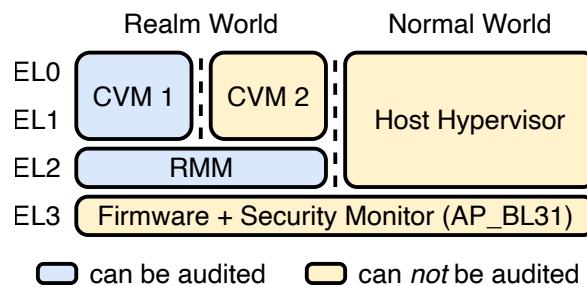


Figure 2.3 – Overview of a confidential VM (CVM) deployment on Arm CCA, from the point of view of the tenant of CVM 1. Software components in blue can typically be audited by the tenant and are measured as part of the attestation. Software components in yellow can not be audited by the tenant, and only the EL3 software (AP_BL31) is part of the attestation.

¹Realm Management Monitor

Table 2.1 – System software measurements in the Arm CCA attestation.

Category	Hashes
Application Processor (AP)	AP_BL1, AP_BL2, AP_BL31, RMM
Runtime Security Engine (RSE)	RSE_BL1_2, RSE_BL2, RSE_S
System Control Processor (SCP)	SCP_BL1, SCP_BL2
Configuration	HW_CONFIG, FW_CONFIG, TB_FW_CONFIG, SOC_FW_CONFIG

a result, the attestation is diluted: it does not enable a challenger to verify the correctness of the system because the untrusted AP_BL31 software can breach any security guarantee.

But that is not the only issue: in case of confidential VMs in the cloud, transient-execution attacks are a real concern [326], including from a malicious host hypervisor. Yet, the attestation provides no guarantees against those. Similarly, the RMM report (`cca-realm-delegated-token` in Figure A.1 in the annex) is limited to a single confidential VM. It does not support finer-grained isolation policies, such as a nested enclave in the confidential VM to protect highly sensitive keys; the whole guest OS needs to be trusted even when that is not desirable. The lack of appropriate isolation policy limits the security guarantees that can be deduced from the attestation, and therefore also contributes to dilution.

The remaining five system software measurements correspond to software executing on co-processors within the SoC. The runtime security engine is a security co-processor that is responsible for producing the attestation itself, while the system control processor manages power and reset. Despite being part of the TCB, the software running on those processors is typically closed source. This is not an ARM-specific problem, AMD and INTEL both rely on similar co-processors (the PSP and ME, respectively). In the context of this thesis we will simply assume this software to be correct, by considering those co-processors to be part of the hardware itself. We will revisit this problem as part of future research directions in Part III.

2.3 Security Monitors

Trusted execution environments rely on a *security monitor* to enforce security policies and write the attestation report. Security monitors serve as the software root of trust. They play a key role in the context of TEEs, and are one of the focal points of this thesis. In this section we provide a high-level overview of security monitors, presenting the underlying design principles and the primitives they rely on.

2.3.1 Principles

The design of security monitors follows three fundamental principles: *verifiability*, *compatibility*, and *minimality*.

Verifiability: The purpose of security monitors is to enable a challenger to verify the correctness of a system, which is the most basic requirement for a TEE. As such, security monitors must be designed so that the security properties of a system can be audited. This often requires the security monitor itself to be source-available to the challenger, but also that configuration and operations of the security monitor can be audited through the security monitor report in the attestation.

Compatibility: Security monitors are designed to extend *existing systems* with TEEs. As such, they can be seen as an alternative to a costly from-scratch system re-design, as has been done for security-critical embedded systems [14, 170, 222, 274]. By compatibility, we mean compatibility with software running *outside* of the TEE. In other words, a system that does not use the security monitor's TEE should run as if no security monitor was present on the system. Compatibility with software running *inside* the TEE, such as enclaves and VMs, is helpful, but can be dealt with using dedicated runtimes [30, 40, 114], and as such is often left outside of the security monitor as per the minimality principle.

Minimality: The minimality principle comes from the need for uncompromising security. The security monitor acts as the software root of trust, and therefore any claim of security guarantee relies on the security monitor correctness. Not only does minimality reduce the attack surface and simplify formal verification [154], it plays a critical role in making the system easier to reason about.

Sometimes, these design principles can be in tension. For instance, compatibility and minimality can be hard to reconcile without dedicated hardware support. When there is a tension, verifiability always takes precedence over the other two principles, as it is a strict requirement for TEEs. Throughout this thesis we will see how the different principles come into play, and how to re-design systems software in accordance.

2.3.2 Primitives

Borrowing inspiration from Liedtke's definition of micro-kernels [176] and in accordance with the minimality principle, we can define the primitives that a security monitor should retain control over as *the minimal set of primitives required to attest and enforce the desired TEE guarantees*. In other words, a primitive should be implemented in a security monitor only if implementing it outside would prevent enforcing the required TEE guarantees. In most cases this boils down to four basic primitives: *measurement*, *attestation*, *memory isolation*, and *context switching*.

Measurement: A security monitor must be able to measure software and configuration. Often, this means having the ability to hash code and data in memory. The measurements are included in a security monitor report that is transmitted through attestation.

Attestation: To create an attestation, the security monitor must interact with the hardware root of trust, which concatenates the hardware identity token and system software measurements, as discussed in [Section 2.2](#). The hardware root of trust *must be accessible only to the security monitor*, otherwise fake attestations can be forged and authenticity can no longer be verified. In systems where this is not possible, the hardware root of trust must require authentication before generating an attestation, such as with TPM 2.0 [\[285\]](#). In those cases, the credentials must be stored securely within the security monitor.

Memory Isolation: Security monitors enforce memory isolation to protect TEEs from untrusted software on the system. To be effective, isolation must protect from direct memory accesses, but also from indirect accesses through devices or co-processors. The exact mechanisms to enforce memory isolation vary from one implementation to the other: second-level page tables, I/O MMU, encryption, or dedicated hardware support.

Context Switching: Context switching between two security domains must always be handled by the security monitor, both for synchronous (e.g., traps or system calls) and asynchronous exits (interrupts). This is required to ensure the security monitor remains in control of the state shared across security domains — both architectural and micro-architectural — in order to enforce the required isolation and agreed-upon ABI. An important consequence is that the security monitor itself cannot be forcefully preempted by any other software on the machine.

2.4 A Short History of Trusted Execution Environments

Establishing trust in a machine is a long-standing problem that had a profound impact on the design of systems — both on hardware and software. A comprehensive history of the impact of the problem of trust on computer design would have to go back to at least the early time-sharing machines [\[75, 246\]](#), and take the necessary detours to mention the plethora of interesting systems that ensued, such as the broad family of capability-based machines [\[308, 310\]](#).

In the context of this thesis we will focus on the narrow slice of computer history that has led to the development of trusted execution environments as we know them today. We will give some background preceding the introduction of TEEs, and then discuss the evolution of TEEs over time: from software-based systems, to hardware-managed TEEs, and finally the hardware-software co-designs that emerged in the last few years. [Figure 2.4](#) provides an overview of the three generations of TEE-capable systems.

2.4.1 Inception and Historical Context

We choose to start our history in the early 80s, during a major effort by the US Department of Defense to synthesize the core ideas and requirements of the time for trusted computer systems [183]. One of the outcomes is the *Trusted Computer System Evaluation Criteria* [293], also known as the Orange book, which heavily influenced the design of systems of that time.

The effort defines a trusted computer system as one that “employs sufficient hardware and software integrity measures” [280], echoing the hardware-software co-design we see in the TEEs today. On the software side, the Orange Book mentions the idea of a *security kernel* [17], a kind of software responsible for the security-related functions, such as access control and validation—the clear counterpart of today’s security monitors. The concept of security kernel does not prescribe a particular implementation, and both a kernel and a hypervisor-based implementation are considered at the time. In both cases, the reports already put emphasis on the need to keep the TCB minimal and verifiable. As we will see shortly, those ideas will serve as the basis of modern TEEs.

Fast forward to the 90s, another category of systems of interest for this thesis is the L4 micro-kernel family, which introduced the modern micro-kernel design principles [88, 176]. Security monitors share the principle of minimality with micro-kernels, but evolved a different set of features. In particular, security monitors are not intended to be used as a kernel replacement (whether micro- or monolithic), they instead rely on less privileged software to implement kernel and OS abstractions². Indeed, micro-kernel features such as time sharing and communication across TEEs might in some circumstances be detrimental to the system, in particular for workloads requiring strong isolation close to physical partitioning [57, 328]. We will present one such case in Chapter 5, and how to bridge the gap between security monitors and micro-kernels in Chapter 6.

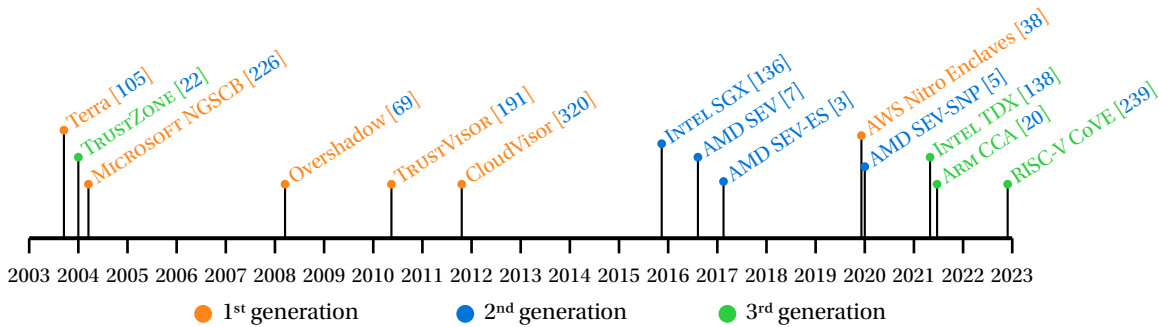


Figure 2.4 – A selection of notable academic and industrial TEEs in recent history.

²Similar to the original exokernel [89] design.

2.4.2 – 1st Generation: Hypervisor-based TEEs

The first systems to exhibit all the characteristics of TEEs were implemented using hypervisor-based security monitors. This enabled preserving compatibility with existing operating systems, despite the lack of a dedicated privilege mode for security monitors at the time. AWS Nitro Enclaves [38] is a modern example of large-scale deployment of this category of TEEs.

The first implementations were built on top of off-the-shelf hypervisors, due to the complexity of implementing a dedicated hypervisor without hardware support. The Terra security monitor [105], for instance, is built on top of the VMWARE hypervisor [49]. The size and complexity of hypervisors of the time created a tension between the compatibility and minimality principles. The advent of hardware support for virtualization [1, 292] put an end to that tension by drastically lowering the software complexity of hypervisors. This opened the door to small security monitor implementations [191, 320] while preserving full compatibility with existing OSes.

Micro-kernels similarly benefited from hardware support for virtualization, enabling running off-the-shelf OSes as micro-kernel tasks, effectively behaving as a security monitor. The process of porting an existing system as a VM on top of a micro-kernel, and progressively moving sensitive services into isolated micro-kernel tasks is sometimes called *incremental cyber-retrofit* [124]. SEL4 [154] and NOVA [274] are two example micro-kernels supporting those kinds of retrofit. However, those systems often lack an attestation mechanism, as the flexibility of micro-kernels can make it hard to reason about isolation guarantees, and therefore fail to uphold the verifiability principle. We propose a solution to this design problem in Chapter 6.

2.4.3 – 2nd Generation: Hardware-based TEEs

While the first generation of TEEs was completely software-based, the second generation relies on hardware implementations of security monitor functionalities. The two most notable members of this category are INTEL SGX [76, 136] and AMD SEV [4]. INTEL and AMD implementations diverge in appearance, but rely on the same core ideas. INTEL SGX exposes an enclave abstraction, and relies on micro-coded instructions to implement the security monitor. AMD SEV exposes a confidential VM abstraction, and relies on a separate secure processor called the PSP³ to implement the security monitor, with which the application processor can communicate over memory mapped I/O. To illustrate the typical implementation of a hardware-based security monitor, let us consider the implementation of the security monitor primitives (discussed in Section 2.3) for INTEL SGX and AMD SEV.

³Platform Security Processor

Measurement: During TEE creation physical pages are added and measured one by one. On INTEL SGX the EEXTEND instruction updates the measurement digest by chunks of 256 bytes, and must be called 16 times per 4K page, while AMD relies on the PSP for updating the digest.

Attestation: Both INTEL SGX and AMD SEV rely on software for the attestation logic. AMD relies on the PSP, while INTEL SGX uses a set of dedicated *architectural enclaves*, *i.e.*, SGX enclaves running signed INTEL software with elevated privileges compared to standard enclaves. Note that with the SGX approach any enclave vulnerabilities can lead to leaked platform keys [51, 257], whereas on AMD keys remain protected on the PSP.

Context Switching: On TEE exits, both synchronous and asynchronous, the security monitor must perform a context switch to protect TEE registers and apply side-channel mitigations if appropriate. On INTEL SGX the hardware implements the context switching logic and saves register state in a dedicated enclave memory region. The original AMD SEV implementation would save the VM state in plaintext in a non-protected memory region, leaking guest state. To correct the flaw AMD later introduced SEV-ES [3] (for Encrypted State), which extends the host CPU hardware to encrypt VM state before returning control to the host⁴.

Memory Isolation: Enforcing memory isolation is the difficult part with hardware-based TEEs. Software-based security monitors rely on a separate privilege mode (such as hypervisor mode) and a dedicated memory protection mechanism (such as second-level page tables) under the sole control of the security monitor itself. Hardware-based TEEs instead rely on the host page tables but enforce integrity, *i.e.*, aborting if the host modifies the layout of the TEE's virtual address space. The protection of the metadata and page life-cycle management is, however, challenging. The lack of a dedicated page table integrity mechanism in the original AMD SEV and SEV-ES led to successful compromise of confidential VMs [204], and the introduction of a Reverse Map Table (RMP) structure to protect integrity in SEV-SNP. Yet, protecting the integrity of the RMP itself remains challenging and a source of attacks [261]. Similarly, INTEL SGX relies on dedicated metadata to protect integrity. In SGX v1, integrity metadata was stored in a dedicated CPU area, limiting the maximum enclave size (usually to 128 MB on commercial CPUs). SGX v2 moved the metadata in RAM to lift the enclave size limitation.

INTEL SGX and AMD SEV are both commercially successful systems and interesting experiments in how far the hardware can go in enforcing high-level security properties. Yet, the choice of relying mainly on hardware for implementing security policies has been questioned [41]. Hardware implementations are not intrinsically more secure [98], are harder to extend with new features or to patch when new vulnerabilities are found. As a result,

⁴Note that this prevents the host hypervisor from performing emulation when required. To work around that, SEV-ES adds a new instruction and memory area for the guest to selectively share state, effectively delegating parts of the security monitor prerogatives to the guest VM itself.

hardware vendors have moved toward a hardware-software co-design for the next generation of TEEs.

2.4.4 – 3rd Generation: Hardware-Software Co-Design

The third and current generation of TEEs is characterized by the introduction of a dedicated privilege mode and memory protection mechanism for security monitors. INTEL TDX [138], ARM CCA [20], and the RISC-V CoVE proposal [253] all fit this hardware-software co-design pattern. The ARM TRUSTZONE [22] extension also fits that pattern and is an early example of third generation TEE. Starting with ARMv9-A, TRUSTZONE is being consolidated into ARM CCA.

Figure 2.5 illustrates the architecture of ARM CCA, as a representative example of a third generation TEE. The security monitor executes at the highest privilege level, EL3. The security monitor controls a structure called the Granule Page Table (GPT) which assigns each physical frame to one of the four execution states (called *world* in CCA): normal world, realm world (for confidential VMs), secure world (for TRUSTZONE), and root world (for the security monitor itself). The GPT checks are cumulative with the existing first and second-level page table checks, effectively providing a third level of protection⁵.

RISC-V CoVE similarly relies on a similar structure (the Memory Tracking Table, or MTT) in M-mode, while INTEL TXT relies on per-confidential VM encryption keys controlled by a new SEAM (SEcure Arbitration Mode) privilege level.

This thesis takes place in the context of third generation TEEs, and discusses how to design system software for a zero-trust computing infrastructure. The addition of dedicated hardware support for attestation and software security monitors creates new opportunities to rethink how systems should be designed with zero-trust and verifiability as guiding principles.

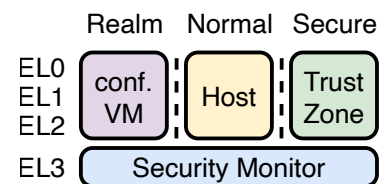


Figure 2.5 – Overview of Arm CCA

⁵Three levels of protection, but only two levels of virtual addresses. The GPT does not allow re-mapping physical frames like page tables do, it only impacts access control.

Part I Virtual Firmware Monitors

Low level software is often granted high privilege, yet this need not be the case. Although vendor firmware plays a critical role in the operation and management of the machine, most of its functionality does not require unfettered access to security critical software and data. In this part of the thesis we demonstrate that vendor firmware can be safely and efficiently deprivileged, decoupling its functionality from isolation enforcement and reducing attestation dilution.

In [Chapter 3](#) we introduce a new class of systems, called *virtual firmware monitors*, that run unmodified vendor firmware in user-space. To do so, we demonstrate software-based virtualization of the highest privilege mode of the application CPU. We describe the implementation of MIRALIS, a RISC-V virtual firmware monitor. MIRALIS can be used to implement security monitors that do not rely on firmware correctness to enforce security policies. We implement three security policies on MIRALIS, to protect the OS, enclaves, and confidential VMs from malicious firmware. Finally, we demonstrate that MIRALIS can effectively virtualize unmodified vendor firmware for two hardware platforms with no performance degradation compared to native execution.

In [Chapter 4](#) we demonstrate the use of lightweight formal methods for verifying functional correctness of virtual firmware monitors. We conduct a top-down analysis of software-based virtualization, and propose two simple correctness criteria on the lock-step execution of the virtual and host machines. By relating the two executions, these criteria transform the task of verifying higher-level properties, such as memory isolation, into simpler conditions that can often be discharged automatically. We demonstrate the applicability of our approach by developing a verification framework for MIRALIS, leveraging the KANI RUST model checker and a SAIL specification of the RISC-V architecture. Using our tool, we identified and corrected 21 bugs and proved several properties, including memory isolation, with minimal human effort.

The work presented in this part of the thesis is based upon the following publications:

- **The Design and Implementation of a Virtual Firmware Monitor** SOSP'25
Charly Castes, François Costa, Neelu S. Kalani, Timothy Roscoe, Nate Foster, Thomas Bourgeat, Edouard Bugnion
- **Lightweight Hypervisor Verification: Putting the Hardware Burger on a Diet** HotOS'25
Charly Castes, François Costa, Nate Foster, Thomas Bourgeat, Edouard Bugnion

3 The Design and Implementation of a Virtual Firmware Monitors

The security challenges resulting from multi-tenancy, increased exposure to networks and other external threats, supply chain attacks, and more capable adversaries have forced systems designers to shift from traditional models based on trusted, privileged components to ones based on mutual distrust between components. In particular, *Trusted Execution Environments* rely on a privileged *security monitor* that protects applications or virtual machines from untrusted kernels or hypervisors. All major Instruction Set Architectures (ISAs) now offer TEE hardware extensions [5, 22, 136, 138, 175, 241], and a growing number of software solutions seek to provide similar security guarantees [69, 97, 98, 166, 191, 223, 320].

Security monitors have thus become one of the most important software components running on modern systems, acting as the root of trust supporting TEEs. One might expect such critical components to be well-isolated from other software on the machine, but this is not the case today: in most systems, *security monitors are co-located with vendor-supplied firmware* running at the highest privilege level on the application CPU, with no isolation between them. Bugs or vulnerabilities in the tens-to-hundreds of thousands of lines of firmware code can compromise the security of the entire platform [66, 81, 95, 177, 260].

In this chapter, we show how to apply the principle of least-privilege to vendor firmware to strengthen the guarantees provided by a system’s attestation. We propose a new class of system software called *Virtual Firmware Monitors* (VFM) that de-privilege vendor firmware by virtualizing the highest privilege mode on the application CPU. A VFM runs vendor firmware in user-space and intercepts access to privileged resources and configuration registers. Figure 3.1 provides a comparison of VFMs with existing systems.

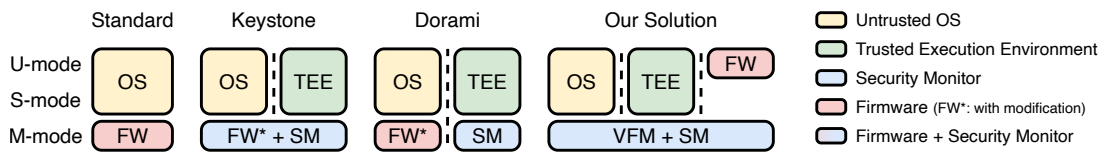


Figure 3.1 – Comparison of TEE deployments on RISC-V. With Keystone [166] and most TEEs the security monitor is co-located with the vendor firmware. Dorami [159] is a recent attempt at privilege separation that requires firmware refactoring and binary scanning. Our solution achieves privilege separation with no firmware modification.

A VFM has two components: (1) a required *firmware virtualization subsystem* and (2) an optional *fast path offloading subsystem* for better performance. The former relies on classical software virtualization techniques without requiring specific hardware extensions. Under the classical Popek and Goldberg definition of virtualizability [228], we find that RISC-V’s M-mode is virtualizable but that ARM’s EL3 is not. The fast path offloading subsystem bypasses the deprivileged firmware for common operations, and is only required on platforms which rely heavily on software emulation of optional hardware features.

We present the implementation of MIRALIS, the first virtual firmware monitor. MIRALIS is written in RUST, runs on RISC-V, and is extensible, supporting custom isolation policies. We demonstrate the virtualization and sandboxing of unmodified vendor firmware for multiple hardware platforms including VISIONFIVE 2, PREMIER P550, and popular open-source firmware. We further extend MIRALIS with support for secure enclaves and confidential VMs by porting the KEYSTONE [166] and ACE [223] security monitors respectively. Unlike previous security monitors and other attempts at privilege separation, MIRALIS *requires no vendor firmware modifications*. We evaluate MIRALIS’s performance on a wide range of applications and report no degradation compared to native execution.

In this section we define our key terms and concepts and motivate the need for sandboxing unmodified firmware.

3.1 The Anatomy of Firmware

“Firmware” is often an umbrella term for a variety of software running on a physical machine: devices, management controllers, and security co-processors all run different flavors of firmware. In this chapter we use “firmware” to mean software running at the highest privilege level on the application CPU, such as ARM’s EL3 and RISC-V’s M-mode, although other kinds of firmware are the subject of complementary research efforts [99, 101]. We exclude AMD and INTEL x86-64 from our discussion because of their heavy reliance on microcode for system management [41] and the complex, vendor-specific, and often undocumented privilege modes required for system operations (such as SMM, SEAM, Architectural Enclaves, or the AMD Secure Processor [5, 70, 76]).

In contrast, the ARM and RISC-V architectures implement system management in software using well-documented privilege levels, and on high-end ARM and RISC-V platforms the process of building firmware is standardized around an open core with vendor-specific extensions. The core infrastructure is provided by open-source firmware libraries, such as ARM TF-A [286] and RISC-V OPENSBI [242]. Those libraries are high-quality and well-audited, although vulnerabilities in them do appear [287]. Above the core, vendors add their own management code: drivers, debugging tools, configuration, telemetry, and more. To take one

example, the vendor firmware of the CAVIUM THUNDERX-1 ARM CPU is more than a million lines of C, including a custom standard library, scheduler, and LUA interpreter to customize firmware operations [195]. Often, vendor firmware is provided without source, as an opaque binary.

3.2 Vendor Firmware Dilutes the Attestation

While firmware vulnerabilities often make the headlines [85, 112, 149, 160, 211] and security researchers continue to find new flaws in vendor firmware [66, 81, 95, 177, 260], the threat of a buggy or malicious firmware has been largely ignored by system software designers so far. Even high-assurance systems like seL4 [154] simply assume the firmware to be correct – not out of negligence, but because firmware by its nature is a highly privileged, non-portable, and proprietary artifact.

However, this is particularly problematic in the context of TEEs such as enclaves [35, 77, 97, 98, 166] and confidential VMs (CVMs) [175, 223, 253], all of which rely on a *security monitor* to enforce isolation between trusted and non-trusted environments. This security monitor currently runs at the same privilege level as the vendor firmware.

Figure 3.1 shows TEE deployments on RISC-V, where the security monitor is co-located with vendor firmware [35, 166, 223]. This means that the whole vendor firmware image is part of the security monitor's Trusted Computing Base (TCB). For RISC-V, the problem will get worse as RISC-V cores and vendor firmware become more complex, particularly with an increased emphasis on server platforms. The ARM architecture provides separation of vendor firmware and security monitor via additional privilege modes, such as TRUSTZONE [22] and realm EL3 [20], but this fails to address the fundamental security issue: vendor firmware remains more privileged than security monitors, and the latter must trust the former.

Not only does the firmware increase the TCB, in the common case of proprietary firmware the system software measurements within the system's attestation can no longer be verified by an independent challenger. This dilutes the value of a system's attestation, and precludes the use of the system in a zero-trust computing infrastructure.

3.3 Threat Model

We therefore define our threat model as follows: we assume an adversary with full control over vendor-supplied firmware *and* the OS or hypervisor. Furthermore, we assume that the vendor firmware binary is closed and cannot be modified by the system operator.

We exclude physical attacks, denial of service, or attacks from firmware running on auxiliary cores or other devices in the platform. Further, transient execution attacks [52, 156, 184, 203, 232] are also out of scope, although standard mitigations can be applied [42]. We also assume that platform-specific instructions, hardware registers, and MMIO regions are known and documented (although the vendor firmware itself may be closed), and that the hardware adheres to its specification.

Under this threat model, we guarantee that the adversary can neither gain code execution capability (integrity) at the highest privilege level nor gain access to memory used by code at this level (confidentiality). In Section 3.6 we further discuss how to extend this protection to other components of the system, such as the OS, enclaves, or confidential VMs.

3.4 Virtual Firmware Monitors

In this chapter we show how to make security monitors secure in the presence of untrustworthy vendor firmware without requiring new hardware or changes to firmware binaries. We introduce a new category of system software, the *Virtual Firmware Monitor* (VFM), to enforce the principle of least privilege all the way down to firmware.

VFMs rely on *classical virtualization* to de-privilege *only* vendor firmware, achieving this using software-based virtualization techniques without requiring any additional hardware extension.

3.4.1 Theory of Classical Virtualization

In 1974 Popek & Goldberg formulated the formal architectural requirements to support virtual machines safely controlled by a *Virtual Machine Monitor* (VMM) using a technique called *trap & emulate* [228]. The approach is purely software-based and predates the introduction of CPU virtualization extensions [1, 292] and modern hypervisors. The resulting requirements provide sufficient criteria for virtualizable ISAs: all *sensitive* instructions must

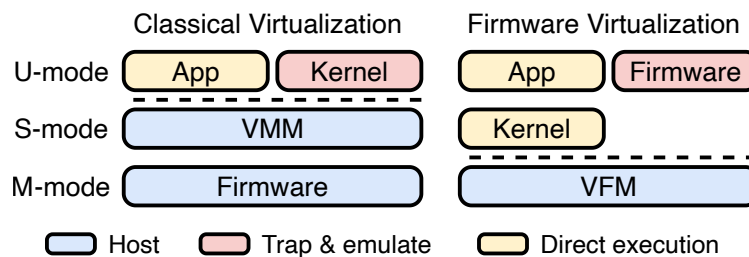


Figure 3.2 – Classical vs. firmware virtualization.

be *privileged*. An instruction is said to be sensitive if it modifies or depends on the privileged state or privilege level. Famously, not all ISAs are classically virtualizable, *e.g.*, x86_32 is *not* classically virtualizable as instructions such as `popf` are “sensitive” and leak privileged state (*e.g.*, the interrupt flag) [49].

Figure 3.2 (left) illustrates classical virtualization on a RISC-V architecture, where the unmodified guest kernel runs in user-mode (U) rather than supervisor-mode (S). Upon executing a *privileged instruction*, the guest kernel traps to the VMM, which then emulates the trapping instruction and resumes the kernel in U-mode, effectively creating a virtual S-mode in user space.

3.4.2 Firmware Virtualization

Firmware Virtualization extends the concept of classical virtualization to the highest privilege level, which is used to execute firmware, *i.e.*, machine-mode (M-mode) on RISC-V.

Figure 3.2 (right) shows how, by analogy with classical virtualization, a VFM creates a virtual M-mode (vM-mode) by running firmware in user-space with trap & emulate. Importantly, the host OS (both S- and U-mode) runs natively without interference from the VFM. As VFMs are used solely for isolation they do not require the complexity of traditional hypervisors needed to multiplex physical resources like memory management, scheduling, and device drivers. VFMs can co-exist in M-mode with security monitors that implement TEE abstractions, and do not interfere with hypervisors running in S or HS-mode.

The classical virtualization requirements of Popek & Goldberg [228] also provide a good lens to reason about the construction of VFMs. A manual review reveals that RISC-V’s M-mode is virtualizable, while ARM’s EL3 is not. Indeed, just as the x86_32 `popf` instruction is “sensitive”, the operation of the ARM `cpsid` instruction used to disable interrupts also depends on the current privilege level but does not trap: instead, it is treated as a no-op in userspace.

Although ARM’s EL3 is not currently virtualizable, this does not completely preclude the implementation of an ARM VFM. Similar to x86 hypervisors, there exist two options to virtualize ARM’s firmware: binary patching, or hardware modification. Binary patching consists in replacing the instructions violating the virtualization requirements (the `cpsid` instruction family for ARM) by a trapping instruction. In the general purpose case of OS virtualization binary patching can be tricky due to dynamically loaded code. The original VMWARE monitor relies on runtime dynamic binary translation for that purpose [49]. However, most firmware is not self-modifying, and closed-sourced firmware binaries could be patched ahead of time through a static analysis pass. The second option is to extend the ARM architecture with one extra configuration bit, which when turned on restores the trapping behavior of the `cpsid` instruction family.

On x86_64 platforms the system management mode (SMM) can not be virtualized through trap & emulate because faulting instructions cannot trap to SMM, instead SMM is entered upon reception of a system management interrupt (SMI). As a result, SMM is not compatible with the machine model proposed by Popek & Goldberg, and other avenues must be explored for de-privileging SMM software [83].

3.4.3 Access Control to System Resources

A VFM enables enforcing fine-grained access control to system resources beyond main memory. Vendor firmware configures and operates the hardware through two means: control registers to configure CPU cores, and Memory Mapped I/O (MMIO) for the rest of the system. By design, a VFM intercepts all accesses to control registers by trapping privileged instructions and can decide to either forward the write to the hardware register, filter the value, or simply ignore it. Similarly, a VFM leverages the memory protection unit (PMP on RISC-V) to trap accesses to MMIO regions.

3.4.4 Fast Path Offloading

An explicit requirement of the VFM is to minimize the impact on OS performance. A VFM introduces two kinds of overhead: emulation of privileged operations during firmware execution, and world switches on a transition between the firmware and the OS. Notably, a VFM introduces no overhead during OS execution. The impact on OS performance is therefore proportional to the number of – and duration of – world switches to the virtualized firmware.

To better understand the performance implications of a VFM we measure the interactions between LINUX and the vendor firmware on the VISIONFIVE 2 RISC-V single board computer and plot the results in [Figure 3.3](#). The figure represents the proportion of traps from the OS to the firmware grouped per category during 500ms windows throughout the kernel boot, including the bootloader, early kernel initialization, and idling in user-space.

On this platform 99.98% of all traps to firmware have one of five causes: reading the time register, configuring the timer deadline, misaligned load & store, IPIs, and remote fences. Those traps all share an important property: they are software emulation of unimplemented hardware features specified in the RISC-V architecture [243, 302]. Any hardware that does not implement this functionality will require the same generic emulation. For instance, the supervisor timer deadline is part of the Sstc RISC-V extension, while IPIs and remote fences require a compatible interrupt controller.

Due to this portable nature, and because of the high trap frequency (5500 trap/s during boot), we can offload the handling of most traps to the VFM itself, bypassing the virtualized firmware. This is practical because the features are part of the open RISC-V standard and are

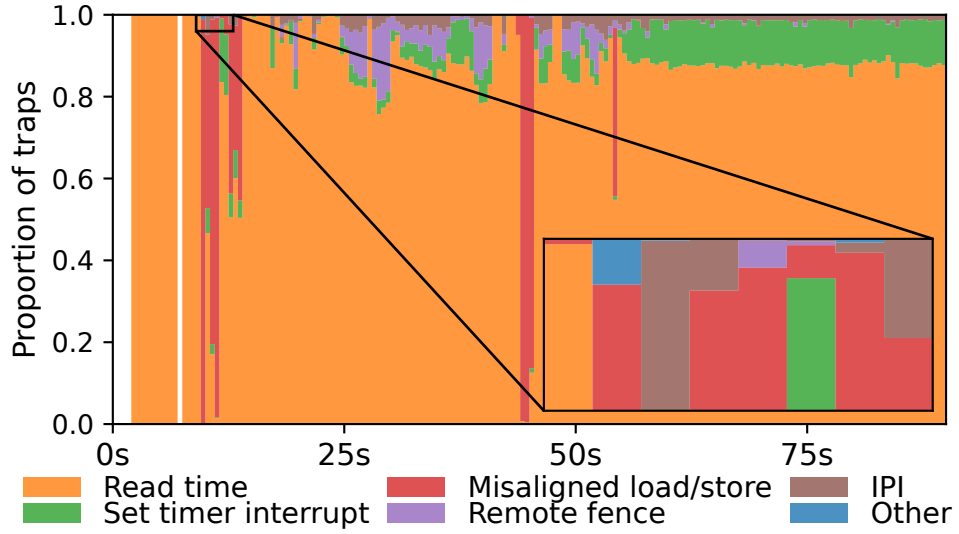


Figure 3.3 – Distributions of M-mode trap causes over time during Linux boot on the VisionFive 2. Only five causes account for 99.98% of all traps.

portable across RISC-V platforms and vendors. In our experience, each can be implemented in 10 to 100 lines of code. We emphasize that this does *not* involve porting code from vendor firmware, nor does it require firmware to be open-source as long as it adheres to the RISC-V standard SBI [240]. Doing so reduces the number of world switches to just 1.17 per second during the boot sequence, and therefore has no noticeable performance overhead compared to the baseline.

We revisit and measure the impact of this design decision in the evaluation section Section 3.8. In particular, we find that support for reading the time CSR and the supervisor timer (Sstc) extension would remove the need for fast path offloading. For instance, fast path offloading is not required on CPUs implementing RVA23 or a more recent RISC-V profile [244].

3.5 M-mode Virtualization with MIRALIS

This section presents the design of MIRALIS, a virtual firmware monitor for RISC-V platforms. MIRALIS is written in RUST, executes in M-mode, and can run unmodified firmware in user-space by simulating a virtual M-mode through trap & emulate. In the following we describe the virtualization of three main aspects of the RISC-V architecture: the CPU, memory protection unit, and interrupts.

3.5.1 CPU Virtualization

MIRALIS executes in M-mode with interrupts disabled and follows a simple execution model where trap handlers always run to completion. [Figure 3.4](#) provides an overview of the design of MIRALIS. At any time, a CPU core (*i.e.*, *hart* on RISC-V) executes in one of two worlds: vM-mode for the virtual firmware, and direct execution for the OS. The trap handler dispatches traps to different subsystems based on the world the trap comes from. A trap from the virtual firmware will result in software emulation, whereas traps from the OS are either re-injected in the virtual firmware or handled directly by MIRALIS as a fast path for common operations. MIRALIS additionally virtualizes M-mode interrupts, such as timer and Inter-Processor Interrupts (IPI), by intercepting and re-injecting corresponding physical interrupts.

During virtual firmware execution all S- or M-mode instructions trap to MIRALIS for emulation. The instruction emulator is the biggest subsystem in MIRALIS, with 2.1k lines of code to implement the decoder and execution engine for privileged instructions. On RISC-V the number of privileged instructions is small, MIRALIS has support for 12. The complexity, however, lies in the emulation of the Control and Status Registers (CSRs), which occupies most of the RISC-V privilege architecture specification [\[302\]](#). The number of CSRs is platform specific; MIRALIS has support for 84 of them. MIRALIS maintains a shadow copy of the CSRs on which the instruction emulator operates. Those virtual CSRs are never installed in the physical registers while the virtual firmware is executing, although MIRALIS will configure the physical CSRs in accordance for emulation purposes. Memory traps are handled by different subsystems, as explained in [Section 3.5.2](#) and [Section 3.5.3](#), while all other traps are simply re-injected in vM-mode.

After processing each trap, MIRALIS checks for vM-mode interrupts and which execution mode (firmware or OS) to return to. A virtual interrupt must be injected if it is both pending and enabled. This check must be done *after* emulation as traps and privileged instructions can mask or unmask interrupts (*e.g.*, mie CSR). The last step is to check for *world switches*. A world switch is when MIRALIS switches from vM-mode (firmware mode) to direct execution (OS mode) or vice versa, usually because of a trap from the OS or the firmware executing the `mret` instruction. Each direction requires a different handling: from firmware to the OS MIRALIS installs the virtual CSRs into the physical registers, except for CSRs required for emulation or isolation such as PMP and mie, and conversely from the OS to firmware MIRALIS loads the content of the physical CSRs into the virtual copies and installs well defined values in physical registers. As a world switch involves changing memory permissions, it also requires a TLB flush. Finally, MIRALIS restores the general purpose registers and returns control to the appropriate mode.

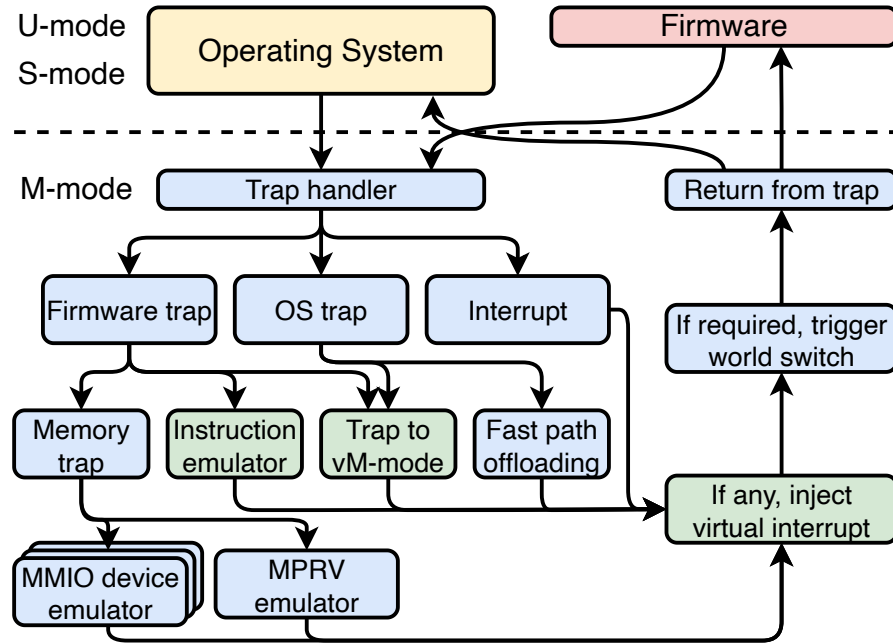


Figure 3.4 – Overview of Miralis: Firmware executes in virtual M-mode and OS traps and associated devices are emulated. Green boxes are verified via symbolic execution (see Chapter 4).

3.5.2 Physical Memory Protection Virtualization

On RISC-V, M-mode software does not use an MMU for memory management and protection but instead relies on Physical Memory Protection (PMP). Unlike virtual memory systems, PMP virtualization does not require shadow or multi-level page tables, thus simplifying emulation.

PMP rules are defined using up to 8 configuration registers and 64 address registers, each protecting a contiguous memory segment. Rules are evaluated in order of priority, with the first matching entry determining access permissions. MIRALIS provides virtual PMP CSRs to the firmware by multiplexing the physical PMP registers, as shown in Figure 3.5 in the case of 8 physical PMP entries. The virtual PMP entries are installed in the physical registers with *lower priority* than the entries protecting MIRALIS to ensure MIRALIS’s PMP rules always take precedence. As PMPs do not take effect in M-mode unless locked, MIRALIS sets read-write-execute permissions on unlocked vPMP entries while executing the firmware to mimic hardware behavior.

In addition to reserving PMP entries to protect MIRALIS’s own memory and MMIO devices, MIRALIS reserves three extra entries to emulate M-mode memory protection. First, by default

		Firmware (MPRV off)	Firmware (MPRV on)	OS
0	SYSTEM memory	---	---	---
1	Virtual CLINT	---	---	---
2	MPRV emulation	---	--X	---
3	ToR null entry	---	---	---
4		vPMP 0	vPMP 0	vPMP 0
5	Virtual PMP	vPMP 1	vPMP 1	vPMP 1
6		vPMP 2	vPMP 2	vPMP 2
7	Default access	RWX	RWX	---

Active
 Inactive
 Managed by firmware

Figure 3.5 – Physical PMP access rights configuration for memory virtualization in the different execution modes. RWX stands for read-write-execute, --- for no permission.

M-mode is granted access to all memory whereas S- and U-mode are granted no access. When running the virtual firmware in vM-mode (*i.e.*, physical U-mode) MIRALIS configures read-write-execute permissions for all memory using the last PMP entry to emulate this behavior. On world-switches, the last entry is disabled to match S- and U-mode semantics. Second, with the Top of Range (ToR) addressing mode, the starting address of the segment is defined by the preceding entry. When the PMP 0 entry uses ToR addressing mode, the starting address is hardwired as 0. To ensure *virtual* PMP 0 behaves accordingly in ToR addressing mode, the physical PMP entry preceding the PMP entry that hosts vPMP 0 (*e.g.*, entry 3 on Figure 3.5) must be configured with the address 0. Finally, the `mstatus.MPRV` bit (Memory PRiVilege) modifies the effective privilege mode for loads and stores *but not* for instruction read, in M-mode. This allows the firmware to read or write data using the kernel or user virtual addresses without software page table walk. We emulate this behavior by setting a PMP entry with execute-only permission on all memory when the firmware enables MPRV. This causes all loads and stores to trap to MIRALIS, which then installs the page tables and perform the access on behalf of the firmware using MPRV itself.

3.5.3 Interrupts, Devices, and MMIO Virtualization

MIRALIS does not virtualize devices such as disks or network interface cards, as such devices are usually managed by the kernel rather than the firmware. Therefore, MIRALIS forces the delegation of all non-M-mode interrupts by hard-wiring corresponding bits to one in the virtual `midelleg` CSR, as allowed by the RISC-V specification [302]. This only leaves M-mode interrupts to be virtualized.

On the platforms we consider, the only MMIO device which needs emulation is the CLINT (Core Local INTerruptor). In contrast, we found that other devices such as the PLIC, cache controllers, or embedded NPU and GPU do not need emulation because they are either not used by the virtual firmware, or used only during initialization and access can be safely

revoked afterward (see sandbox policy [Section 3.6.2](#)). A virtual MMIO device is created by protecting the corresponding MMIO region with a PMP entry. Then on a memory trap, MIRALIS checks if the access is within the bounds of an MMIO device and calls into the corresponding device emulator. MIRALIS’s virtual CLINT multiplexes timer and software interrupts between itself and the virtual firmware. MIRALIS also has experimental support for virtualizing M-mode external interrupts through a virtual PLIC, although it is not needed on the platforms we support as vendor firmware delegates all external interrupts to the OS.

Finally, MIRALIS needs to restrict direct memory accesses (DMA) by the firmware. On platforms with IOPMP [\[245\]](#) support, MIRALIS would virtualize the IOPMP to restrict which memory regions can be accessed through DMA by the firmware, similarly to how MIRALIS restricts direct memory accesses through PMP virtualization ([Section 3.5.2](#)). As with PMP virtualization, this would only incur a small overhead on IOPMP modification, and a minimal runtime overhead if IOPMP was not already enabled by the platform’s firmware [\[309\]](#). However, as IOPMP is not widely supported yet, MIRALIS defaults to preventing firmware DMA by blocking firmware access to all MMIO regions controlling DMA-capable devices.

3.5.4 Comparison with Hypervisors

In our experience developing MIRALIS, VFMs are drastically simpler software than traditional hypervisors, even with hardware support for virtualization. Indeed, virtualizing firmware is a simpler problem in many aspects: (1) there is a single virtual firmware, removing the need for memory, CPU, and devices multiplexing; (2) firmware-level instructions and features are easier to virtualize, such as the PMP compared to an MMU; and (3) there is no need for complex drivers beyond interrupt controllers. [Table 3.1](#) provides a breakdown of MIRALIS’s implementation.

3.6 Policies

So far, we have discussed how to virtualize a firmware, but not how to isolate the rest of the system from a buggy or malicious firmware. This section describes how to enforce isolation policies with MIRALIS, leveraging the same ideas behind security monitors built on top of traditional hypervisors [\[38, 69, 128, 130, 191, 320\]](#). As VFMs are useful across a wide variety of

Table 3.1 – Miralis lines of code decomposition.

Emulator	2.7k LoC	Fast path offload	190 LoC
Hardware interface	1.1k LoC	Other	1.8k LoC
MMIO devices	430 LoC	Total	6.2k LoC

scenarios and requirements, we designed MIRALIS to be extensible, with support for custom isolation policies.

3.6.1 Policy Modules

An isolation policy is defined in terms of a *policy module*, *i.e.*, a RUST struct that implements the policy module interface. The interface consists of seven optional methods: three are called on `ecall`, `trap`, and `world switch` from the firmware, three others from the OS, and one called on interrupts. The methods either complement or override MIRALIS's implementation based on the return value. In addition, policy modules can get assigned PMP entries with higher priority than the virtual PMPs to configure memory protection for both the OS and firmware. Policy modules are meant to implement security monitor features and decouple M-mode virtualization from use case-specific isolation requirements. In the following we present three examples of policy modules.

3.6.2 Firmware Sandbox Policy

The firmware sandbox policy isolates the whole OS from an untrusted firmware. More precisely, the policy allocates a small memory range for the firmware and blocks access to the rest of the memory, including OS memory, the PCIe address space, and most other MMIO devices. In addition, the policy saves and restores general purpose registers and S-mode CSRs to prevent unintended leakage. In the case of explicit calls to the firmware (SBI calls), the policy module allows a well-defined set of registers to be passed as SBI call arguments. We automatically generate the per-SBI call register allow-list from the SBI specification [243]. Finally, some firmware needs access to platform specific CSRs or MMIO regions to operate and configure the hardware. Access to those resources is denied by default but can be granted assuming that the hardware vendor accurately documents the effects of those resources and that they do not allow the firmware to bypass the sandbox policy restrictions. In our implementation, MIRALIS will stop the machine with an error message if the firmware performs an illegal action. In production systems, we imagine that MIRALIS would instead log the invalid action and return arbitrary values. This would enable the detection of malware, such as rootkits, without impacting the OS execution.

Threat model: The sandbox policy considers an adversary with full control over the firmware whose goal is to violate the integrity or confidentiality of the OS. Physical and side channel attacks are out of scope of the policy. Assuming the hardware behaves as documented, the sandbox policy guarantees the integrity and confidentiality of the OS.

On the two platforms used in our evaluation sandboxing the firmware had surprisingly little consequences, all of which could easily be worked around. First, the firmware needs access to OS memory during initialization to load the S-mode bootloader (U-BOOT in our case). As

a result we allow access to all OS memory until the first jump to S-mode, at which point the policy locks down the OS memory on all harts until the machine is powered off and then computes a hash of the initial S-mode image. Second, on some platforms the firmware is used to emulate misaligned loads and stores. We thus simply implemented the misaligned loads and stores emulation directly in the policy. Finally, the LINUX `printk` early console was configured to use an SBI call requiring shared memory between the OS and firmware.

3.6.3 KEYSTONE Policy

The KEYSTONE [166] policy is a re-write of the KEYSTONE security monitor in RUST as a MIRALIS policy module. The KEYSTONE policy adds support for enclaves to MIRALIS, a common TEE abstraction [77, 136, 191] that protects a user application from the OS and hypervisor. The policy exposes the same SBI interface as the original KEYSTONE security monitor, such as the ability to create, run, and terminate enclaves, although our current implementation lacks attestation capabilities. Therefore, we re-use the existing KEYSTONE infrastructure including the enclave runtime, libraries, and kernel module. The policy protects the enclave using policy PMPs that take priority over the virtual PMPs, protecting the enclave from both the OS and firmware.

Threat model: The KEYSTONE policy follows the same threat model as the original KEYSTONE security monitor [166] *except* that the vendor firmware is no longer trusted and follows the same attacker model as the OS.

3.6.4 ACE Policy

The ACE policy provides support for confidential VMs (CVMs) by porting the ACE monitor [223] to MIRALIS. With CVMs the host hypervisor is responsible for managing and scheduling VMs, but has no access to the underlying memory, making the content of the VM confidential from the hypervisor point of view. The firmware, however, can get access to the VM's content. All major ISAs have support or plan to support CVMs [5, 138, 175, 253]. The ACE monitor is an M-mode security monitor that enforces isolation between the host hypervisor and CVMs through PMPs. To run CVMs, ACE leverages the RISC-VH (hypervisor) extension which introduces the HS and VS-mode. Note that supporting HS and VS-mode in MIRALIS requires no special treatment compared to any other S-mode extension, it suffices to implement support for saving and restoring the new CSRs on world switches and creating shadow copies.

Threat model: The ACE policy follows the same threat model as the original ACE monitor [223], *except* that the adversary additionally has full control over the virtual firmware.

To minimize changes to the ACE code base we opt for a co-location approach, where the ACE policy takes control over M-mode while running the host hypervisor and confidential VMs by handling traps directly without MIRALIS intervention, but yield back to MIRALIS when trapping to the firmware. As ACE is significantly more complex than the KEYSTONE monitor and already written in RUST, we port the existing monitor as a policy module rather than attempting a full re-implementation.

3.7 Security Analysis

Before discussing the evaluation of MIRALIS, we propose to review the security guarantees of our implementation. MIRALIS enforces its own integrity and confidentiality by protecting itself from accesses by the firmware and the OS. This includes direct loads and stores, as well as DMA from devices. DMA protection relies on IOPMP or an equivalent mechanism. If no such mechanism is available, as is the case on the platforms we evaluate, MIRALIS revokes firmware accesses to DMA-capable devices, but is vulnerable to device accesses from the OS. As we will discuss in [Chapter 4](#), the core virtualization logic has been verified for functional correctness, which covers 43% of the code base. The rest of the code, including assembly, device drivers, and fast-path offloading, is part of the TCB.

MIRALIS is intended to be used with at least one policy module, which is responsible for providing the desired isolation guarantees for the OS, enclaves, or confidential VMs. Policy modules play a similar role to traditional security monitors, such as KEYSTONE [166] or KOMODO [98]. In comparison, MIRALIS provides the necessary infrastructure to extend the security policies to the vendor firmware, which would otherwise need to be trusted. It is the responsibility of the policy authors to verify policy modules. We did not verify the sandbox and KEYSTONE policies, but ported verified code for the ACE policy [223].

Table 3.2 - Characteristics of our evaluation platforms

	VISIONFIVE 2	PREMIER P550
Number of Cores	4	4
Frequency	1.5GHz	1.8GHz
RAM	4GB	16GB
LINUX kernel version	5.15	6.6

3.8 Evaluation

In this section we seek to validate the hypothesis underlying the design of VFMs. Specifically, we seek to answer the following questions:

- Q1** Can a VFM virtualize unmodified vendor firmware?
- Q2** Does a VFM induce overhead on OS execution?
- Q3** Is fast path offloading necessary on the current generation of RISC-V CPUs?
- Q4** Can a VFM be used to implement custom isolation policies, such as enclaves and confidential VMs?

For this purpose, we evaluate MIRALIS on two platforms from different vendors. We demonstrate the virtualization of two unmodified firmware images, evaluate the overhead of MIRALIS through a comprehensive suite of micro and macro-benchmarks with and without fast path offloading, and port two existing security monitors as MIRALIS policy modules.

3.8.1 Experiment Setup

For our evaluation, we port MIRALIS to two platforms from different vendors: STARFIVE’s VISIONFIVE 2 and SiFIVE’s PREMIER P550. The VISIONFIVE 2 is powered by U74 in-order RISC-V cores, while the PREMIER P550 uses the high-performance P550 out-of-order super-scalar cores with H-mode support. [Table 3.2](#) summarizes the characteristics of both platforms. Unless stated otherwise MIRALIS is configured with the firmware sandbox policy ([Section 3.6.2](#)), results are averaged over 5 runs, and number of traps are reported per core. “MIRALIS no-offload” designates the configuration *without* fast path offloading. When network is involved we use a remote client with 8 cores at 3.5 GHz on the local Ethernet network.

3.8.2 Virtualizing Unmodified Firmware

To answer **Q1** we evaluate MIRALIS’s ability to virtualize unmodified vendor firmware on the VISIONFIVE 2 and PREMIER P550 boards. Both vendors rely on a two-stage M-mode firmware.

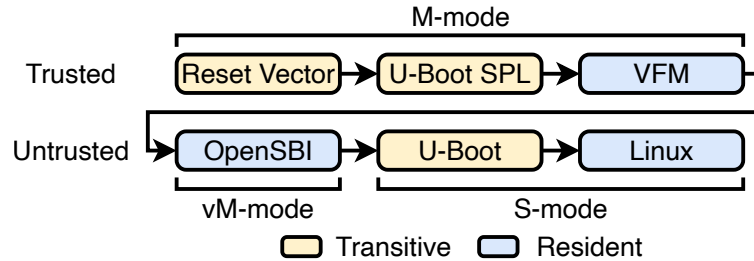


Figure 3.6 – Modified boot-flow with the Miralis VFM.

The first stage is responsible for initializing DRAM and loading the second stage in memory, but is no longer used afterward. The second stage is the firmware used at runtime, and stays resident in memory until system reset. Both second-stage firmware images are open-source, and account for 31.8k and 24.7k lines of code on the VISIONFIVE 2 and PREMIER P550, respectively. We insert MIRALIS in-between the two firmware stages, ensuring that MIRALIS gets full control over the machine and that the second stage firmware never executes in M-mode. [Figure 3.6](#) illustrates the boot flow on both platforms. The PREMIER P550 further exposes four non-standard but documented CSRs for controlling speculation and error reporting, MIRALIS explicitly allows writes to these CSRs for that platform. With those configurations the user sees no difference in machine behavior besides additional logs during system boot.

As both vendor firmware are based on OPENSBI, we further exercise MIRALIS’s virtualization capabilities with two independent open-source firmware images: RUSTSBI and ZEPHYR. RUSTSBI [251] is an alternative to OPENSBI, written from scratch in RUST. ZEPHYR [318] is a Real Time OS (RTOS) designed for micro-controllers, it consists of an M-mode kernel and user applications. Both RUSTSBI and ZEPHYR pass their respective test suite while being virtualized by MIRALIS, and are in fact part of the test pipeline. Finally, to demonstrate that the firmware does not need to be open-source, we port MIRALIS to a third platform, the STAR64, for which the vendor does not provide firmware sources. Upon receiving the board, we extracted the 164kB firmware image from the flash and successfully ran it virtualized with MIRALIS.

We conclude that the answer to **Q1** is *yes*: MIRALIS can virtualize unmodified vendor firmware.

3.8.3 Impact on OS Performance

We then proceed to evaluate the impact of a VFM on OS operations to answer **Q2** and **Q3**.

3.8.3.1 Cost of MIRALIS Operations

A VFM introduces overhead when the OS traps to the virtualized firmware. The incurred cost falls in two categories: firmware instruction emulation and world switches. [Table 3.3](#) reports both costs measured with a minimal firmware and kernel. Specifically, we measure the cost of emulating the “`csrw mscratch, x0`” privileged instruction, including the overhead of trapping to M-mode and jumping back to U-mode (*i.e.*, vM-mode). The cost of a world switch is reported for a full round trip, *i.e.*, sequence OS → VFM → firmware → VFM → OS, where the firmware returns directly to the OS. % We typically observe around 10 firmware instruction emulation per world switches, in the order of 7 000 cycles or 5 μ s per trap from the OS to the virtual firmware. On average, we observe around 10 firmware instruction emulations from

Table 3.3 – Overhead of Miralis operations in cycles.

	Instruction emulation	World switch
VISIONFIVE 2	483	2704
PREMIER P550	271	4098

the virtual firmware to handle a trap from the OS. This results in a world switch round trip cost of about 7 000 cycles or $5\mu s$.

On platforms that heavily rely on the firmware for software emulation of unimplemented hardware features, the overhead of firmware virtualization might become prohibitive. To keep the overhead low, we implement a fast path for the five most common operations (Section 3.4.4) directly in MIRALIS, bypassing the virtual firmware for those operations. We compare the implementation in MIRALIS and the vendor firmware (based on OPENSBI) of two such operations, reads to the timer CSR and IPIs, by instrumenting the LINUX kernel to execute 100k of each operation in a tight loop. Table 3.4 reports the results for the VISIONFIVE 2. We found MIRALIS’s implementation to be slightly faster, a difference that might be caused by OPENSBI’s heavy use of indirect function calls that prevent cross-functions inlining and compiler optimizations, although this is an implementation detail rather than a fundamental result. In addition, we report the cost of the two operations with fast path disabled as “MIRALIS no-offload”. Without fast path, the operations trap to the virtualized firmware, which in turn requires emulation for each privileged instruction and adds up to an order of magnitude of overhead.

3.8.3.2 Microbenchmarks

We evaluate the impact of MIRALIS on OS CPU and I/O microbenchmarks. We report the results for the VISIONFIVE 2 but observe similar patterns for the PREMIER P550.

CPU: We use COREMARK-PRO [87] running on all 4 cores as a CPU-bound microbenchmark. We report the relative performance compared to running without MIRALIS (Native) in Figure 3.7.

Table 3.4 – Cost of timer read and IPI

	read time	IPI
Native (OPENSBI)	$228ns$	$3.96\mu s$
MIRALIS	$208ns$	$3.65\mu s$
MIRALIS no-offload	$7.26ns$	$39.8\mu s$

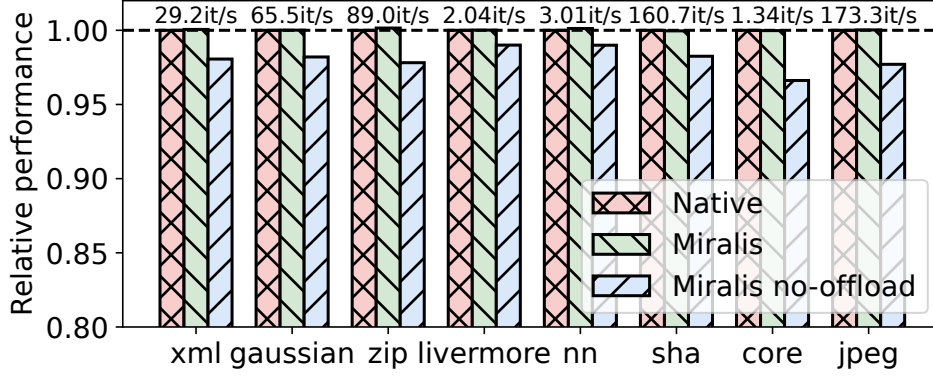


Figure 3.7 – Relative CoreMark-Pro scores.

Disk I/O: We use IOZONE [212] with O_DIRECT to evaluate the impact on disk I/O. We report the read and write throughput for a 128K record size on the visionfive in Figure 3.8. The absolute performance depends on the underlying hardware: we measure close to 10x the throughput on the PREMIER P550.

Network latency: Finally we evaluate the impact of MIRALIS on the latency of MEMCACHED using MEMENTIER [162]. MEMENTIER is a closed-loop latency benchmark; we open 32 concurrent connections with 100k requests each. We report the latency distribution in Figure 3.9.

On all three microbenchmark suites, MIRALIS causes no overhead compared to the baseline (“Native”). Indeed, we measure 0.479 world switches per second on average across all

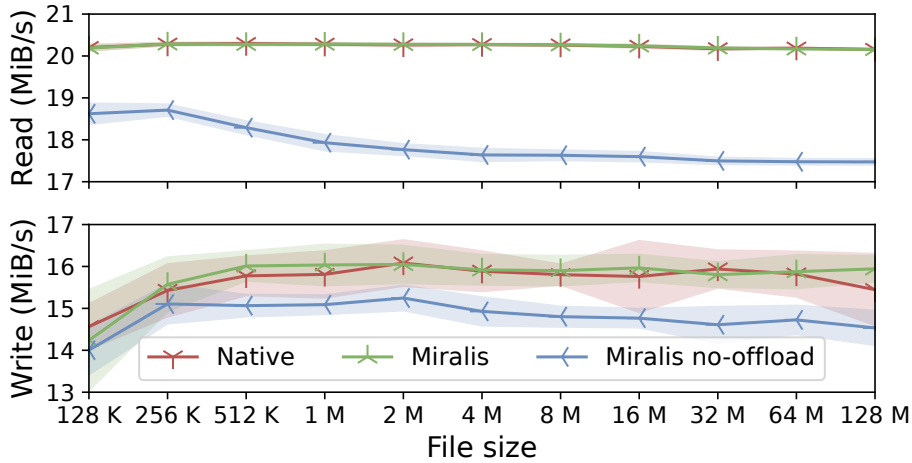


Figure 3.8 – IOzone throughputs on the VisionFive 2.

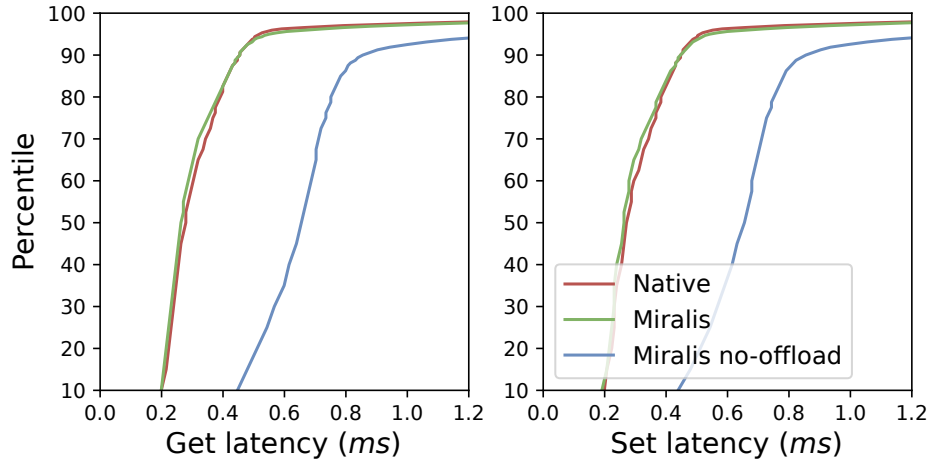


Figure 3.9 – Memcached latency distribution.

microbenchmarks, a negligible amount. In fact we observe that MIRALIS performs slightly better on IOZONE write and on MEMCACHED under the 95th percentile, $263ns$ vs $279ns$ at the median for instance. The difference is due to MIRALIS’s fast path implementation being slightly more efficient than the vendor firmware’s. However, with the fast path disabled (“MIRALIS no-offload”), we observe an overhead that scales with the frequency of traps to M-mode. The CPU benchmark causes the least traps to M-mode, $11k/s$, while MEMCACHED causes the most at $388k$ trap/s. As a result, we observe on average 1.9% overhead on CORE-MARK-PRO, 10.6% on IOZONE, and $2\times$ the latency on MEMCACHED.

We observe similar results on boot time, measured from board power-on to login prompt display on the screen, during which the firmware plays a major role. We measure 48.0s boot time with MIRALIS against 47.5s for the baseline (1% overhead), whereas removing the offload increases boot time to 61.3s, *i.e.*, 29% overhead.

3.8.3.3 Application Benchmarks

We then proceed to evaluate the impact of MIRALIS in realistic application scenarios. We consider four workloads: two in-memory key-value stores, REDIS and memcached, an SQL database (MySQL), and GCC. Figure 3.10 reports the relative performance (higher is better) for both the VISIONFIVE 2 and the PREMIER P550.

Key-value stores: We evaluate the popular REDIS (v7.0) and MEMCACHED (v1.6) in-memory key-value stores on YCSB workload A [73] with one million operations each. REDIS executes

as a single threaded application, while MEMCACHED uses the 4 available cores on both platforms.

SQL workload: We evaluate MySQL (v8.0) as a mixed CPU, network, and disk application. We use the OLTP read/write workload from Sysbench [158] with 128 concurrent clients over 5 minutes.

Compilation: Finally we measure the time to compile REDIS with GCC v12.2.

In line with the microbenchmarks evaluation, we observe very few world switches, 0.486/s on average on the VISIONFIVE 2 and none at all on the PREMIER P550, and therefore no overhead due to MIRALIS. As network workloads tend to generate more traps to M-mode, up to 272k trap/s for REDIS and 389k trap/s for MEMCACHED, MIRALIS performs better on those with up to 7.6 and 1.2% improvement on the VISIONFIVE 2 and PREMIER P550, respectively. Once again, this is due to MIRALIS's fast path being slightly faster, although the vendor firmware could be tuned to achieve comparable performance. Disabling fast path offloading drastically increases the overhead: up to 259% on REDIS for the PREMIER P550, although compute heavy workload is less dramatically affected.

Overall, we conclude that a VFM introduces no overhead on OS execution due to the low frequency of world switches (Q2). However, considering the overhead incurred when disabling fast path offloading we conclude that the fast path for the most common operations must be implemented within the VFM (Q3). We expect this to change with the next generation of RISC-V CPUs. Indeed, implementing support for reading the time CSR plus the Sstc extension would remove 96.5% of all world switches on our application benchmarks.

3.8.3.4 Support for Isolation Policies

Finally, to answer Q4 we test the suitability of MIRALIS for enforcing custom isolation policies by deploying the three policies described in Section 3.6.

All benchmarks presented so far use the firmware sandbox policy (Section 3.6.2), and therefore prevent the firmware from accessing OS memory, devices, and registers with no overhead.

In addition, we demonstrate the KEYSTONE policy (Section 3.6.3) by reproducing the RV8 benchmark from the KEYSTONE paper [166] using a re-implementation of the KEYSTONE security monitor as a MIRALIS policy module. We nonetheless re-use the rest of the KEYSTONE infrastructure without modification, including the kernel driver, enclave runtime, applications, and loader. Figure 3.11 reports the results on the VISIONFIVE 2, with an average overhead of 1% when running inside an enclave, in line with the results reported by KEYSTONE.

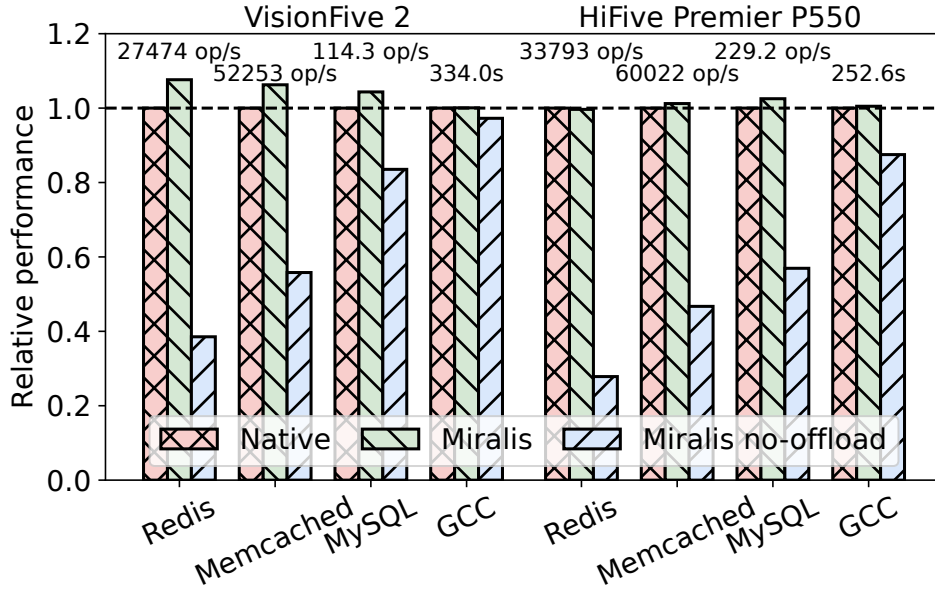


Figure 3.10 – Relative performance comparison for various application workloads.

Lastly, with the ACE policy (Section 3.6.4), we reproduce the configuration demonstrated by the ACE authors. Specifically, we run a confidential LINUX VM with 1GB of RAM, a virtio NIC and a drive. We run the VM through the ACE API, where we further strengthen confidentiality by excluding the firmware from the TCB. As ACE does not support available hardware platforms at the time of writing, we reproduce the ACE example on QEMU and are therefore unable to provide a meaningful performance comparison.

Overall, we conclude that VFMs are a suitable host for TEE security monitors (**Q4**), as they do not require vendor firmware modification and efficiently de-privilege untrusted firmware while removing it from the TCB. We further built MIRALIS around a modular architecture, making it simple to experiment with new isolation policies.

3.9 Discussion and Related Work

In this section we discuss related work and elaborate on firmware virtualization.

Privilege Separation: DORAMI [159] is the first system to propose a practical privilege separation between the security monitor and the firmware. DORAMI sandboxes the firmware directly within M-mode by leveraging ePMP (Smepmp [302] extension) through a combination of firmware modifications and binary scanning. DORAMI is closely related to systems such as Nested Kernel [80] for kernel-level privilege separation. In comparison, MIRALIS

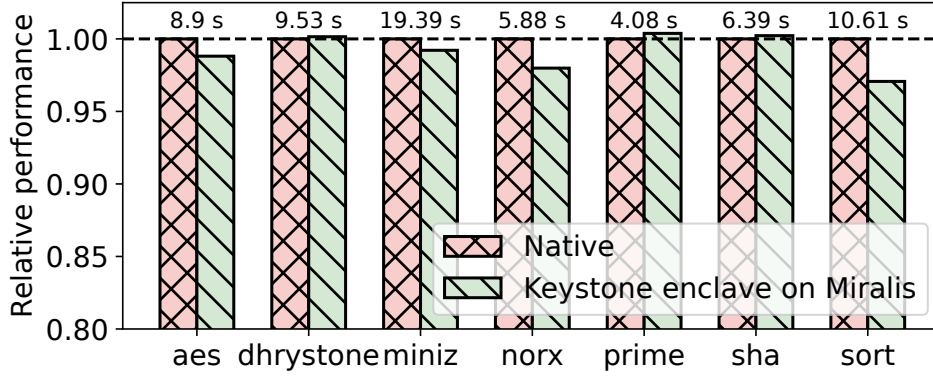


Figure 3.11 – Relative performance of Keystone enclaves implemented by Miralis on RV8.

requires no firmware modification and runs on commercially available hardware. MIRALIS instead is more closely related to micro-kernels [154, 170, 176] and privilege-separation hypervisors [172, 173], with the firmware virtualization subsystem inspired from classical VMMs [44, 48, 49].

Security Monitors: The growing popularity of TEEs has led to booming research in building security monitors [35, 97, 98, 166, 175, 223]. In particular, the idea of leveraging virtualization to deprive an untrusted kernel has been widely explored [22, 69, 107, 128, 130, 191, 320]. MIRALIS provides the mechanisms needed to push this idea one step further, down to the firmware, and can serve as the basis for the next generation of security monitors. Beyond security considerations, the biggest benefit of MIRALIS is maybe the ease of maintenance and portability. Indeed, maintaining N security monitors across M platforms currently requires $\mathcal{O}(N \times M)$ engineering effort, whereas developing security monitors as MIRALIS policy modules reduces that effort to $\mathcal{O}(N + M)$.

Non-Virtualizable ISAs: As discussed in Section 3.4.2, ARM’s EL3 is not virtualizable due to the ISA design. This does not preclude the sandboxing of firmware with an ARM VFM, however the virtual EL3 mode would behave slightly differently, potentially affecting the execution of the virtualized firmware. We envision two potential solutions to this problem: (1) modifying the vendor firmware to call into the VFM for emulation of instructions that violate the virtualization requirements [228], *i.e.*, paravirtualization [37]; or (2) add a configuration register in the next iteration of the ARM architecture to enable trapping on those instructions.

Running firmware in S-mode: Although we only discussed running vendor firmware in U-mode, running vendor firmware in S-mode is possible under some conditions. Per the Popek & Goldberg requirements [228] all sensitive instructions must trap, yet a firmware in S-mode gets access to many more instructions. On RISC-V it is possible to force all sensitive instructions to trap, such as accesses to the page table pointer or `sret`, and thus possible

to run a firmware in S-mode. That is because RISC-V has been designed so that HS-mode can be emulated in software. Yet there are few benefits for doing so: it complicates the VFM implementation with limited performance benefits as most S-mode instructions should be intercepted.

3.10 Conclusion

In this chapter, we described how to safely and efficiently deprive and isolate unmodified vendor firmware. We introduced the concept of virtual firmware monitor (VFM), a new class of system software that virtualizes the firmware privilege mode. We described MIRALIS, and demonstrated that VFMs can be deployed on commercially available RISC-V platforms and introduce no performance overhead. Finally, we demonstrated that existing security monitors can be ported to MIRALIS, removing the vendor firmware from the TCB, and thus reducing attestation dilution.

4 Lightweight Virtual Firmware Monitor Verification

In [Chapter 3](#) we presented the concept of virtual firmware monitors, which enable securely and efficiently de-privileging unmodified firmware. As a result, the virtual firmware monitor now occupies the highest privilege mode on the machine, and becomes one of the most security-critical pieces of software on the machine.

Per the assumptions we presented in the introduction—that software that can be audited is correct—we can simply consider MIRALIS to be correct by hypothesis. Indeed, virtual firmware monitors are designed to be devoid of proprietary IP, unlike vendor firmware, and can therefore be open-sourced without restrictions. Yet, this is not entirely satisfying, and therefore in this chapter we discuss how to partially verify virtual firmware monitors in order to provide a robust foundation for trusted execution environments.

In the course of developing MIRALIS, we designed a new automated framework for verifying key components of VFMs, including instruction emulation and hardware configuration. While we do not aim for full formal verification of security properties [\[154\]](#), we do provide pragmatic, automated tools to find bugs and improve assurance using lightweight formal methods [\[11, 46, 148\]](#). Our key insight is to express the VFM specification in terms of existing, high-quality executable ISA specifications [\[29, 236\]](#). We apply our method to verify instruction emulation and memory isolation in MIRALIS using the KANI [\[294\]](#) RUST model checker. This methodology allowed us to identify and correct 21 bugs in development, including loss of virtual interrupts, PC overflow, and out-of-bounds accesses.

4.1 Background

This section gives background on the two key technologies that underpin our approach: virtualizable architectures and machine-readable architecture specifications.

4.1.1 Trap & Emulate Hypervisors

In [Chapter 3](#) we already discussed the requirements for virtualization [\[228\]](#), in which Popek and Goldberg describe how to construct a hypervisor on an abstract virtualizable ISA. Under this model the VM can execute each of its instructions in one of two modes: *emulation* for

privileged instructions and *direct execution* for the others. In this chapter rather than focus on a particular hypervisor or implementation, we take a top-down approach and simply assume a trap & emulate hypervisor as described by Popek and Goldberg. We describe two correctness criteria in [Section 4.3](#), which enable verifying a hypervisor’s functional correctness in the case of instruction emulation and direct execution, respectively.

As a result, we describe a general framework applicable to all trap & emulate hypervisors, including virtual firmware monitors such as MIRALIS that we described [Chapter 3](#), but also traditional hypervisors. Indeed, nowadays essentially all major architectures are effectively virtualizable, either because they were designed with virtualization in mind, or because they have been extended to support virtualization [[1](#), [292](#)]. For instance, INTEL’s x86-64’s root and non-root modes [[292](#)] have been designed to satisfy the virtualization requirements. As a result, modern x86-64 hypervisors are implemented as trap & emulate hypervisors, in contrast with the original VMWARE hypervisor [[49](#)] on older, non-virtualizable x86-32 architectures.

In the rest of this chapter we will focus on the special case of VFMs and choose our terminology accordingly. We refer readers interested in the more general case of traditional hypervisor verification to our original publication [[61](#)].

4.1.2 Lifting Architecture Specifications

Formal specifications are the crux of any verification effort; they constitute the foundation that supports any formal claims. The smallest mismatch between a specification and the real system can void the proofs altogether, or at least severely limit their applicability. As a consequence, writing correct specifications remains a challenge even for lightweight formal methods.

In general, there is no way to escape the need to write specifications—at a minimum, one must at least specify the exposed interfaces of the system being verified [[98](#), [118](#), [208](#)]. But VFMs are different. Indeed, the interfaces to a VFM are well defined, as it executes on hardware and exposes virtual hardware, as depicted in [Figure 4.1](#). Fortunately, in recent years we have seen the emergence of high-quality, well-maintained, and machine-readable specification for most architectures [[256](#)]. For instance, the ARM architecture is formalized in

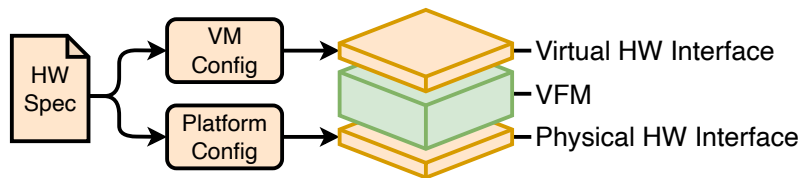


Figure 4.1 – The host and virtual hardware interfaces follow the same hardware specification, but with different configurations.

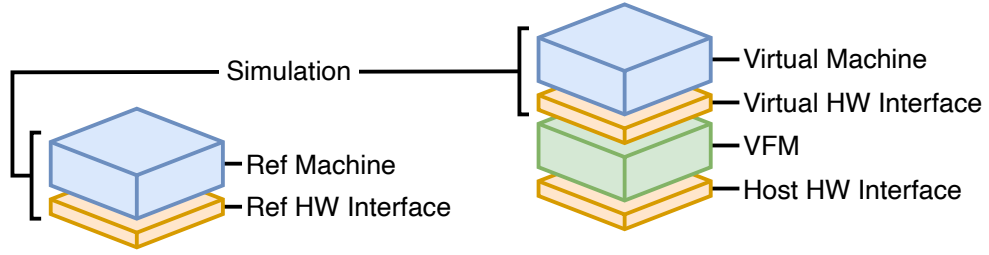


Figure 4.2 – The reference (left) and host (right) machines.

ASL [236] while RISC-V has an official model in SAIL [29]. This is not only an opportunity to free VFM developers from the burden of writing formal specifications, but also a guarantee that the specifications will stay up to date as the hardware evolves. In the rest of this chapter, we therefore assume the availability of a machine-readable architecture specification.

4.2 A Simulation Problem

Many of the most important properties that a VFM might satisfy, such as correct emulation and memory isolation, are the consequences of a simpler simulation problem: proving that the virtualized program, *i.e.*, the *virtual machine* (VM), is a simulation of a separate *reference machine*. Figure 4.2 illustrates the host, virtual, and reference machines. The main complication is that while the host and reference machine usually share the same architecture, they often have different *configurations*, *e.g.*, memory, number of cores, interrupt IDs, or hardware extensions. To illustrate, consider memory isolation: assuming the reference machine is configured with a subset of the host memory, then proving that the VM is a simulation of the reference machine implies that it cannot access the rest of the host memory. In particular, because any such access would trap (for invalid address) on the reference machine, it will also trap on the VM.

To prove the simulation relation, we first need a notion of *observability*, after which it suffices to prove that observable states evolve in lock-step. The Popek and Goldberg formalization offers a natural notion in the form of *privileged state* that is useful for defining observability. The privileged state is all architectural state that can only be accessed through privileged instructions⁶, and by opposition we refer to all other state as *unprivileged*. Because unprivileged instructions are executed directly by the VM, without interference from the VFM, the unprivileged state is what can effectively be observed by the VM. Parts of the privileged state can selectively be made observable during privileged instruction emulation by the VFM, for

⁶In the context of modern hypervisors leveraging virtualization extensions, kernel-level instructions are not necessarily privileged, only instructions that trap outside of the VM (*e.g.*, in root mode on INTEL x86-64) are.

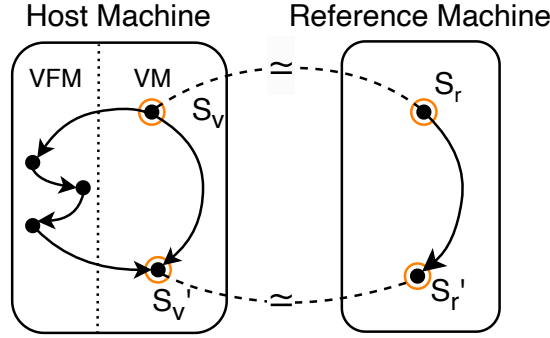


Figure 4.3 – Lock-step execution between a VM and the corresponding reference machine. For each instruction, the VM can either execute the step directly or trap to the VFM for emulation.

instance by loading the content of a virtual privileged register into a general-purpose register. With this definition of observability, we define lock-step execution as follow:

Definition (Lock-step execution).

The virtual and reference machines execute in lock-step if: (i) their initial observable states are equal, and (ii) after each executing an instruction, their observable states remain equal.

To prove lock-step execution we need to proceed disjunctively by cases, as illustrated in Figure 4.3. If the instruction is privileged it traps to the VFM for software emulation. Note that the whole emulation process counts only for a single instruction executed in the VM. Otherwise, it executes directly on the hardware.

4.3 Correctness Criteria

The difficult problem of verifying high-level properties such as memory isolation is now reduced to the simpler problem of proving lock-step execution. In this section, we formalize two criteria, *faithful emulation* and *faithful execution*, that guarantee lock-step execution during privileged instruction emulation and direct execution of unprivileged instructions. Table 4.1 summarizes the notations used in this section.

4.3.1 Modeling the Architecture

An instruction set defines the transition function of the system's state machine. The transitions depend on the configuration $c \in C$ of the platform (accessible memory ranges, available hardware extensions, number of cores, interrupts IDs, etc.) and the current state $s \in$

Table 4.1 – Symbol definitions

Symbol	Definition	Symbol	Definition
C	Set of platform configurations	S_u	Set of unprivileged machine states
c_r	Configuration of the reference machine	s	Machine state
c_h	Configuration of the host machine	p_v	Privileged VM state
I	Set of instructions	u	Unprivileged machine state
I_p	Set of privileged instructions	vfm	VFM's privileged instruction emulator
i	Instruction	hw	Executable hardware specification
S	Set of machine states	$hw _u$	Restriction of the hardware specification to S_u
S_p	Set of privileged machine states	cfg	Hypervisor hardware configuration function

S of the machine (registers and memory). For the purpose of verification, we also make the next instruction $i \in I$ explicit — *i.e.*, we encode the instruction fetch in the model. Note that interrupts can be modeled as special instructions. We can formalize the transition function as follows:

$$\text{hw} : C \times S \times I \rightarrow S. \quad (4.1)$$

Here, the hw function encodes the whole architecture. For the official RISC-V SAIL model, it runs to 16k lines of code. If we fix a configuration c , it can also be used as a simulator. Indeed, C and OCAML simulators can be generated from the official RISC-V SAIL model. In short, the hw function is a high-quality specification of the architecture that already exists. In the following we leverage hw to specify the host, virtual, and reference hardware interfaces (see [Figure 4.2](#)), instead of using custom specifications.

4.3.2 Faithful Emulation

Privileged instructions (and interrupts) executed by the VM trap to the VFM for software emulation. The virtual hardware interface constitutes the biggest attack surface that is directly exposed to the software running in the VM. Bugs in the emulation logic can at best disrupt the VM workload, and at worst lead to privilege escalation. We designate the emulation function of a VFM as vfm, where $I_p \subset I$ is the set of privileged instructions:

$$\text{vfm} : S \times I_p \rightarrow S \quad (4.2)$$

The vfm function corresponds to one iteration of the trap, emulate, resume loop that is found in VFMs. Note that the function can include arbitrary side effects, including scheduling other VMs. Using these definitions, we can define faithful emulation, which states that the VFM implements a correct virtual hardware interface:

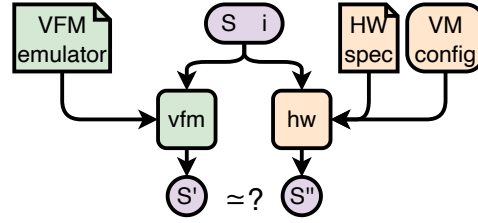


Figure 4.4 – The faithful emulation criteria. A VFM implementation properly emulates the virtual hardware interface if for any state and instruction the resulting state is equivalent to what a reference machine would produce.

Definition (Faithful emulation).

$$\text{let } c \in C, \forall (s, i) \in S \times I_p, \text{vfm}(s, i) \simeq \text{hw}(c, s, i) \quad (4.3)$$

In plain English, the VFM and ISA specifications produce the same output for the same input, at least for all privileged instructions. Figure 4.4 illustrates the criteria. The comparison ($\simeq$) implicitly takes into account differences in internal representation of the system state. The faithful emulation criteria has been used in previous works to generate test cases [16, 190], in this chapter we demonstrate it can scale to exhaustive symbolic execution of a VFM emulation subsystem.

4.3.3 Faithful Execution

To enforce lock-step execution during direct execution the VFM must configure the host hardware to behave like the reference machine. The difficulty comes from the difference in configuration between the host and reference machine, as well as the need for the VFM to maintain its own privileged state. To illustrate this with loads and stores, *i.e.*, prove memory isolation, we present a concrete example involving the RISC-V PMP in § 4.5.3.

To make reasoning easier it is helpful to partition the machine’s state between privileged and unprivileged state, *i.e.*, $S = S_p \times S_u$. Further, as per the virtualization requirements, the privileged state cannot be modified by unprivileged instructions, thus we consider a restriction of hw to the unprivileged state:

$$\text{hw}|_u : C \times S_p \times S_u \times I \rightarrow S_u. \quad (4.4)$$

Faithful execution is linked to a notion of configuration of the host privileged state. During privileged instruction emulation the VFM might update the VM’s privileged state $p_v \in S_p$, which is often kept in in-memory data structures, and gets a chance to modify the host’s own privileged state $p_h \in S_p$, which is installed in the hardware during direct execution. We use $\text{cfg} : S_p \rightarrow S_p$ to denote the abstract VFM function which given a virtual privileged state

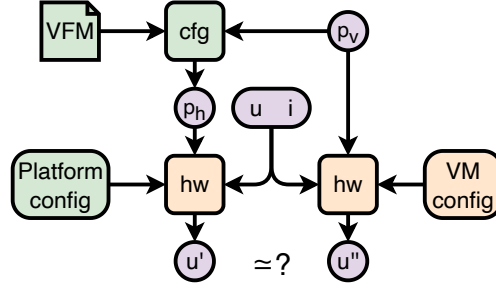


Figure 4.5 – The faithful execution criteria. A VFM must configure the host machine such that direct execution produces observable results similar to a reference machine.

returns the host privileged state. In practice this function is often incremental: given a change in one virtual privileged register the VFM updates the corresponding physical register. For the purpose of verification, capturing the incremental changes is enough to reconstruct the `cfg` function. With this notation, we define faithful execution:

Definition (Faithful execution).

$$\text{let } (c_h, c_r) \in C \times C, \forall (p_v, u, i) \in S_p \times S_u \times I, \quad (4.5)$$

$$\text{hw}|_u(c_h, \text{cfg}(p_v), u, i) \simeq \text{hw}|_u(c_r, p_v, u, i)$$

In plain English, the host hardware must be programmed to execute as if the VM was running on a machine with a different configuration. The verification of the host machine programming therefore requires instantiating two hardware interfaces, one with the VM platform configuration and privileged state, and another with the host platform configuration and the privileged state derived from the VM virtual state by the hypervisor. We illustrate the criteria in Figure 4.5.

While the faithful emulation criterion often catches standard implementation bugs, faithful execution can detect more subtle hardware misconfiguration that cannot be easily caught by language-level analysis or with an incomplete hardware specification. In practice it might be impractical to verify faithful execution for all instructions, but verifying even a subset of instructions can be sufficient to derive desirable properties. For instance, verifying faithful execution of loads and stores is sufficient to ensure memory isolation.

4.4 Checking the Criteria

Section 4.3 presented the two correctness criteria against an abstract VFM model. The rest of this chapter discusses how to apply the criteria to verify concrete VFM implementations.

The first step is to embed both the specification and the VFM implementation into a common representation. Hardware specification languages often support compilation into other languages, for instance the SAIL compiler can generate C, OCAML, ROCQ, and ISABELLE, and is easily extensible.

The next step is to develop adapters to convert between the internal representation of the machine state of the VFM and the specification in order to check for equality. In our experience the adapters tend to be straightforward mappings from one structure layout to another.

Finally, the choice of verification framework can be adapted to the needs of each specific project. We found software model checking to be especially effective because architecture specifications and VFM implementations tend to avoid complex code patterns that complicate model checking, such as unbounded loops. However, we believe other approaches such as symbolic execution or even fuzzing would also yield good results.

4.5 Automated Verification of MIRALIS

We built an automated verification framework for MIRALIS using the KANI [294] model checker for RUST. By translating the SAIL specification to RUST, the faithfulness criteria are written as simple unit tests that can be executed either with concrete or symbolic values using KANI. This makes verification simple to set-up and comprehend for RUST software engineers. We verified faithful emulation and partial faithful execution (memory isolation) and found and corrected 21 bugs in the MIRALIS code base, and proved critical properties such as memory isolation.

4.5.1 Preparing the Architecture Specification

To analyze the specifications of the architectures using KANI, we needed to translate the SAIL code to RUST. Unfortunately, the SAIL compiler does not yet support compiling to RUST, therefore we developed a new SAIL back-end in 2K lines of OCAML. In the future we hope that support for generating RUST will exist upstream and be maintained in the same way as the C back-end is. Using our RUST back-end we generated 6k lines of RUST code from the SAIL specification, with full support for all privileged instructions, interrupts, and memory protection. The model is configured by importing external functions, for instance the number of PMP registers is controlled by providing a `sys_pmp_count` function. We selected one set of parameters and fixed them for the purpose of the verification. Finally, we wrote adapters to convert between the internal representation of the RISC-V state of MIRALIS and the model, as well as checks for equality. In total the TCB includes the SAIL model and compiler, KANI, and the RUST compiler, all maintained externally, as well as our RUST back-end and the adapters.

4.5.2 Checking Faithful Emulation

First, we proved faithful emulation of all privileged instructions and of interrupt delivery. We used the faithful emulation criteria from [Section 4.3.2](#) with arbitrary (symbolic) state and privileged instruction, executing the instruction in both the MIRALIS emulator and the reference RUST model. The symbolic execution covers instruction decoding, emulation, and virtual interrupt delivery, but ignores the assembly trap handler which is part of the TCB. This amounts to a total coverage of 2.4k lines of code, or just above 40% of the MIRALIS code base. We found bugs throughout the whole emulation pipeline, with the biggest concentration in CSR emulation. Thanks to RUST we did not find security flaws due to undefined behavior, although we found multiple opportunities for the VM to crash MIRALIS and mismatch with the RISC-V specification.

During verification we found it especially useful to progressively increase the coverage, making the process of retrofitting formal methods to an existing system much simpler. Indeed, while we can now verify end-to-end emulation correctness, from arbitrary instruction bytes, to the decoder and emulator, we started by verifying instructions one at a time. There are two levers that enable this: the first is simply to verify smaller functions, for instance the decoder only, the second is to constrain the (symbolic) input. We report the verification time for some of the components of the emulation pipeline in [Table 4.2](#), reported on a M3 MacBook Pro. End-to-end verification takes 118 minutes, making it practical as a nightly CI job to catch regressions, while verifying components individually only takes a few minutes, allowing interactive debugging sessions.

As a caveat, we did have to slightly modify the RISC-V model to verify MIRALIS. In total we changed 43 lines of code, or 0.25% of the code base. The two reasons for these changes are the discrepancy between extensions available on the hardware platform and in the model, and the lack of configurability for some aspects of the model. We also fixed and upstreamed two SAIL compiler bugs, and one bug in the RISC-V model. We expect these limitations to disappear as the RISC-V model improves.

Table 4.2 – Verification time of the emulation pipeline.

Verification task	Time	Verification task	Time
mret instruction	68s	wfi instruction	28s
sret instruction	56s	instruction decoder	45s
CSR read	99s	interrupt virtualization	94s
CSR write	9min	end-to-end emulation	118min

4.5.3 Checking Faithful Execution of Loads and Stores

Second, we proved memory isolation using the faithful execution criteria — *i.e.*, we proved faithful execution for load and store instructions. MIRALIS relies on PMP (physical memory protection) registers for its own memory protection but also exposes virtual PMP to the VM. As there can be at most 64 PMP registers, MIRALIS exposes fewer than the host machine. To proceed, we instantiated two hardware interfaces, one with the reference configuration (*i.e.*, fewer PMP registers) and one for the host. We also removed the memory used by MIRALIS and MMIO from the reference machine, as those should not be accessible from the VM.

The main difficulty in proving faithful execution is to reconstruct the configure function `cfg` from the VFM source code. As a reminder, the `cfg` function returns the target host configuration from a virtual machine state. Indeed, while the `vmf` function is often directly implemented in VFMs, the host state is usually configured incrementally. To reconstruct the `cfg` function in MIRALIS we recorded the writes to privileged registers—functionality that already existed for the purpose of unit-testing. We first recorded the writes during VFM initialization, then we provided an arbitrary (symbolic) VM state and recorded the writes that occur in response. The first step was necessary as some of the privileged registers are modified at initialization-time only — *e.g.*, the PMP entries protecting the MIRALIS and MMIO memory. We then installed the VM symbolic state in the reference hardware interface and the state produced by MIRALIS in the host interface, performing a memory access at a symbolic address with both interfaces and querying KANI for equality. We notably discovered an exploitable PMP configuration error. In total memory isolation took 23 minutes to verify.

4.6 Bugs Found

Table 4.3 lists the bugs detected during the verification of MIRALIS. The exact descriptions and fixes for the bugs can be seen in the git history with the following command:

```
git log --grep="^SailBugfix"
```

MIRALIS was developed without initial concerns for verification, beside good engineering practices such as testing and modularity. When the verification effort started MIRALIS was already running on hardware platforms and successfully virtualizing a production firmware. However, the virtualized firmware would sometimes crash after minutes of running, taking the machine down in the process. We identified the cause to be mismatches between the RISC-V specification and MIRALIS' implementation, leading to silent state corruption within the virtual CPU and eventually crashing the firmware. This category of bugs are not a security concern, and are labeled with low severity in Table 4.3: they only affect the integrity of the firmware, not of MIRALIS or the trusted execution environment. Yet, these bugs are especially hard to track down, and make the system unstable or even completely unusable.

Table 4.3 – List of bugs found during verification. Bugs of high severity enable privilege escalation, bugs of medium severity can crash Miralis, bugs of low severity corrupt the state of the VM but do not compromise security.

Registers/Instructions	Description	Severity
mret	Should set MPP to U-mode after mret	Low
mret	Wrong default MPP with U-mode disabled	Low
mvendorid, mcounteren	Registers are 32 bits	Low
mepc, sepc	Bits [1:0] should be hardwired to 0	Low
mtvec, stvec	Restrict to valid vector modes (direct or vectored)	Low
medeleg	Bit 11 is read-only zero	Low
satp	Discard writes with invalid mode	Low
vstart	Invalid write mask	Low
mcause, scause	Allow any value	Low
pmpcfg	Odd pmpcfg register should be invalid	Medium
pmpaddr	Invalid legalization mask	High
pmpaddr	W = 1 & R = 0 is reserved	High
mepc, sepc	Enforce alignment based on C extension	Low
CSR read	Return 0 instead of panicking on unknown CSR	Medium
CSR write	Ignore writes to unknown CSRs	Low
mie	Writes were discarding	Low
sie, sip	Filter based on mideleg	Low
mtvec	Overflow in PC computation in vectored mode	Medium
pmpcfg	Out of bound access	Medium
Interrupts	Were not issued with the correct priority	Low
Compressed loads/store	Off-by-one error in immediate	Low

Figure 4.6 provides an example of such a bug, where an off-by-one error in the computation of the immediate offset of compressed load instructions leads to memory corruption during MMIO and misaligned-writes emulation. Such memory corruption does not immediately cause the firmware to crash, the crash can happen millions of instructions later. This makes this category of bugs very hard and time consuming to debug.

Applying formal methods to MIRALIS enabled us to fix all mismatches between the RISC-V specification and MIRALIS' implementation at once. After fixing the issues found during verification the virtual firmware went from running for a few minutes before crashing to running for two weeks, until we manually turned the machine off.

```

1 C_SW => {
2 -   let imm = (raw >> 3) & 0b100 | (raw >> 7) & 0b111000 | (raw << 1)
   & 0b1000000;
3 +   let imm = (raw >> 4) & 0b100 | (raw >> 7) & 0b111000 | (raw << 1)
   & 0b1000000;
4     LoadInstr {
5         rd,
6         rs1,
7         imm,
8     }
9 }

```

Figure 4.6 - Off-by-one error in the immediate decoding of compressed store instructions.

```

1 -self.csr.mepc = value & !0b11
2 +if self.get(Csr::Misa) & misa::C != 0 {
3 +    self.csr.mepc = value & !0b1
4 +} else {
5 +    self.csr.mepc = value & !0b11
6 +}

```

Figure 4.7 - Adding support for the C (compressed instructions) extension changes valid bit patterns in some configuration registers.

Another remarkable use of our framework is for keeping MIRALIS up-to-date with the RISC-V standard. New features added to the RISC-V spec can change the behavior of older features in subtle ways that can be easily missed by developers. [Figure 4.7](#) provides such an example, where adding support for the compressed instruction extension changes valid bit patterns in existing systems registers. Because we use the official SAIL specification, upgrading MIRALIS is now as simple as pulling the latest SAIL specification, running the verification, and fixing all the reported bugs.

Of all bugs detected, only two could lead to privilege escalation. The two bugs are listed with high severity in [Table 4.3](#) and are related to the virtualization of the PMP registers used for memory protection. [Figure 4.8](#) shows one of those bugs, in which the PMP “lock” bit is not properly removed from the virtual PMP registers before writing it to the physical registers. This enables the virtual firmware to overwrite memory protection configured by MIRALIS, and thus escalate privileges.

We believe the use of RUST reduced the total amount of vulnerabilities in the MIRALIS code-base. In particular, we see in [Table 4.3](#) that some bugs of medium severity that cause MIRALIS

```

1 let reg_idx = idx / 8;
2 let inner_idx = idx % 8;
3 let shift = inner_idx * 8; // 8 bits per config
4 -let cfg = (pmpcfg[reg_idx] >> shift) & 0xff;
5 +let cfg = (pmpcfg[reg_idx] >> shift) & 0x7f; // Remove the lock bit
6 self.set_pmpcfg(idx + offset, cfg as u8);

```

Figure 4.8 – The “lock” bit (0x80) is missing from the bitmask used to convert virtual PMP registers into the physical registers.

to crash would have lead to undefined behavior in some systems languages. For instance the out-of-bound access to the PMP registers array or the integer overflow in the PC computation.

To conclude our discussion, we note that our verification framework only covers the functional correctness of the virtualization subsystems of MIRALIS. In particular, our framework does not cover low-level assembly code such as trap handlers and context switching, which are particularly delicate to get right. We believe that the SAIL specification can also be used for the purpose of verifying low-level assembly and how it interacts with the surrounding RUST code. We developed an initial prototype that enables inline assembly verification by lifting assembly into equivalent RUST, using the SAIL model for emulating assembly instructions. We achieved encouraging early results, including finding and correcting bugs in one of MIRALIS trap handlers, and believe this direction to be promising for future work on verification of low-level systems software.

4.7 Discussion

In this chapter we talked about *lightweight verification*, in opposition to traditional, interactive theorem prover-based formal verification. In this section, we clarify the scope of our verification efforts, the guarantees our tools can provide, and how they compare to other techniques.

4.7.1 Lightweight Formal Methods

By lightweight formal verification, we refer to lightweight formal methods, defined as techniques that can provide immediate benefits at relatively low cost [11, 148]. This is to be contrasted with “standard” formal methods, which traditionally require significant effort and departure from traditional software engineering techniques. Indeed, our approach remains close to standard RUST development practices, and requires little to no prior experience in formal methods: it is, in fact, very close to writing traditional unit tests.

Lightweight formal methods usually rely on heavy automation and can struggle to scale to bigger systems, although progress is being made [64]. As a result, they are often associated with weaker forms of correctness, such as bounded model checking. However, in some special cases such as systems with finite interfaces [208], they can be used to formally prove valuable properties. The verification of the virtualization subsystem of MIRALIS is one such case. Because we verify one instruction execution at a time, and each instruction is either executed directly or emulated in a bounded amount of time, lightweight formal methods such as model checking can be used for exhaustive exploration, and can therefore be used to prove properties about any possible execution of the system. In the case of MIRALIS, we prove that executing one instruction of the virtual firmware will always result in the same observable state as executing the same instruction on a reference machine.

Another limitation of lightweight formal methods is that they often lack a way to express high-level properties, such as security properties. As a result, higher-level properties might need to be expressed in another formal system, such as through pen-and-paper proofs or through interactive theorem provers. In the case of MIRALIS, the faithfulness criteria have been explicitly designed to be within reach of model checkers by limiting the scope to a single instruction execution, rather than an arbitrary length sequence. Indeed, KANI cannot directly be used to prove high-level properties such as *“until the next privileged instruction, all unprivileged instructions will produce the same observable effect on the virtual machine as on the reference machine”*. Such properties, if desired, must be expressed through separate proofs outside of KANI. Assuming faithful execution, it is easy for one to convince oneself that it is the case, but such a proof would live outside of the formal framework used to verify the faithfulness criteria in the first place.

4.7.2 Formal Guarantees and Limitations

Formally, we proved that:

- Any privileged instruction is emulated properly (faithful emulation). This means that the firmware cannot observe behaviors that cannot be observed on the reference machine. Therefore, it cannot read or write data outside of the allocated memory region through privileged instructions. Further, KANI ensures that no `panic!` statement is reachable, meaning that the firmware cannot crash MIRALIS either.
- Any unprivileged load or store can only access the memory region available on the reference machine (faithful execution of loads and stores). This means that the firmware cannot access memory outside of its allocated range through direct loads and stores.
- Virtual interrupts are delivered exactly like they would on the reference machine (faithful emulation for interrupts). Although this is not necessary for security purposes, it is an important liveness property for the firmware to execute as intended by its developers.

Note that the proof does not cover inline assembly snippets and trap handlers, although ongoing work is looking into addressing this limitation. Timing and side-channels are similarly outside the scope of the proof. Finally, and as discussed before, the verification only guarantees the faithfulness criteria for the execution of one instruction, not for an arbitrary sequence. Although a paper proof is straightforward, it is not part of the current formal system.

4.7.3 Comparison with Other Tools

Our approach is based on model checking, and inherits some of its benefits and limitations compared to other tools.

Symbolic execution is another popular type of lightweight formal method. As a representative member of that category we tested SOTERIA [33], a symbolic execution engine with a KANI-compatible RUST front-end, for the verification of MIRALIS. In simpler cases, such as the verification of `mret` instructions, SOTERIA tends to be faster than KANI. However, in some cases symbolic values can lead to path explosions, such as symbolic registers in `csrrw <reg1>, <CSR_reg>, <reg2>`, which causes SOTERIA to explore paths sequentially and thus not terminate in a reasonable amount of time. In contrast, KANI encodes the whole RUST program as an SMT query, which allows the solver to recognize when computations are shared among multiple possible execution paths.

Another category of tool requires manual annotations of post- and preconditions, and can be more or less automated depending on the goals. Examples of such tools include FLUX [168], VERUS [165], and PRUSTI [31] in the context of RUST. Those tools can offer an experience closer to interactive theorem provers at the cost of manual annotation. An explicit goal of our approach is to reuse the existing official SAIL ISA specification as is, which enables automated updates when the upstream specification changes. Using the SAIL specification with those tools would require manual code annotations, which is time-intensive considering the size and complexity of the RISC-V model. As MIRALIS exposes a finite interface, those tools would not provide additional benefits over KANI to verify the faithfulness criteria, except potentially faster verification time once annotations are in place.

4.8 Related Work

This work builds on ideas developed as part of previous efforts in automating system verification [64, 227, 268, 317]. The methodology to automatically verify critical components of MIRALIS was inspired by the findings of past work that verification of systems with *finite interfaces* [208, 209] is often amenable to fully automated verification by SMT solvers [205, 283]. Indeed, a VFM exposes an ISA interface which (at least on RISC-V platforms) is finite.

Our approach leverages the specificities of VFMs to go even one step further by expressing the specification of a VFM as a function of an already existing ISA specification, lifting the burden of writing both the proof and the specification. In particular, this guarantees that the VFM specification stays up-to-date as the ISA evolves.

The idea of applying hardware development tools to hypervisor development has been explored in the past, such as leveraging a CPU vendor test suite to test the KVM implementation [16], or by leveraging high-fidelity emulators to generate test cases [190]. Our approach is similar in many ways, but leverages a formal specification and performs exhaustive symbolic execution, rather than generating test cases.

Prior work has shown that it is possible to verify high-level properties for hypervisors using traditional formal methods [13, 154, 169, 174, 279, 295]. While these results are impressive, they have typically required a huge effort and significant input from verification experts and hand-written hardware specifications that might overlook architectural features. We believe our approach to be complementary to those research efforts, by providing low-effort verification of low-level functionalities against official ISA specifications, and could be enhanced with the formulation and verification of higher-level security guarantees.

4.9 Conclusion

This chapter demonstrates how to apply lightweight formal methods to virtual firmware monitors with minimal human effort. We observe that many high-level VFM properties, such as isolation and platform compatibility, are consequences of proper simulation of a reference machine. We propose two criteria, faithful emulation and faithful execution, to model correct simulation in VFMs. Not only can the criteria be efficiently checked by existing lightweight verification tools but they can also use existing architecture specifications, which removes the need for manual specifications. We built a verification tool for MIRALIS and used it to discover 21 bugs, prove functional correctness of the virtualization subsystems, as well as memory isolation.

Part II Systems Software and Attestation Co-Design

In [Part I](#) we discussed how to decouple security policies from firmware logic. Virtual firmware monitors guarantee that no untrusted firmware can override the security policies, reducing attestation dilution by removing the firmware from the trusted software components.

In the second part of the thesis, we focus on designing and integrating security policies with a system's attestation. The ultimate goal is to enable a challenger to logically deduce the security of a system from the facts presented in the attestation. This excludes any assumptions regarding software executing on the machine that is either not auditable by the challenger, or not part of the attestation. To do so, we propose solutions to two main challenges of TEEs: transient execution attacks, which break traditional isolation boundaries, and fine-grained isolation and attestation by tailoring TEEs to specific use-cases.

In [Chapter 5](#) we discuss how transient-execution vulnerabilities and CPU bugs have repeatedly violated the promises of greater security and isolation from hypervisor-level attacks on platforms such as INTEL TDX, AMD SEV and ARM CCA. At the root of this problem lies the need to multiplex CPU cores with all their complex microarchitectural state among distrusting entities. We propose *core-gapped confidential VMs*, a set of software-only modifications that ensure that no distrusting code shares a core, thus removing all same-core side-channels and transient-execution vulnerabilities from the guest's TCB.

In [Chapter 6](#) we discuss the problem of tailoring TEE abstractions to specific deployment use-cases. Current systems focus on providing one point solution, such as enclaves, or confidential VMs, or containers, rather than providing mechanisms that empower users to define tailored abstractions. We propose a shift toward *capability-based security monitors*, and implement TYCHE as a proof of concept. We discuss how to co-design capability-based isolation with system attestation, how to retrofit existing abstractions on top of capabilities, and new use-cases, such as nesting TEEs and shared confidential memory.

The work presented in this part of the thesis is based upon the following publications:

- **Sharing is leaking: blocking transient execution attacks with core-gapped CVMs** ASPLOS'24
Charly Castes, Andrew Baumann
- **TYCHE: Composable Isolation as a Foundation to Manage Trust in the Cloud** EuroS&P'26
*Charly Castes**, Adrien Ghosn*, Neelu S. Kalani, Yuchen Qian, Marios Kogias, Edouard Bugnion
- **Creating Trust by Abolishing Hierarchies** HotOS'23
*Charly Castes**, Adrien Ghosn*, Neelu S. Kalani, Yuchen Qian, Marios Kogias, Mathias Payer, Edouard Bugnion

5 Blocking Transient Execution Attacks with Core-Gapped Confidential VMs

Since Spectre [157] and Meltdown [184] in 2018, CPU security isolation has been repeatedly broken by a series of transient-execution vulnerabilities [23, 36, 50, 53, 55, 113, 125, 129, 140, 141, 153, 199, 203, 213, 230, 231, 233, 237, 258, 259, 262, 263, 273, 284, 305, 307, 322] and CPU bugs that break isolation [47, 142, 219, 220, 277, 321] and the flood of vulnerabilities shows no sign of stopping (see Figure 5.3 in Section 5.1.2). For the worst problems, mitigations have been developed in the form of microcode fixes and software patches, but these often induce substantial performance overheads. Because these vulnerabilities stem from microarchitectural designs that seek to maximise single-threaded performance through aggressive speculation and out-of-order execution [299], we have no reason to doubt that they will continue.

The same period has seen increasing adoption by cloud providers and customers of *confidential VMs* (CVMs) on platforms like AMD SEV-SNP, INTEL TDX, and ARM CCA. These architectures encrypt guest memory and enable the removal of the hypervisor and host stack from a cloud guest's trusted computing base (TCB). In principle, the guest TCB is reduced solely to the CPU hardware and remotely-attested security monitor. However, in practice, transient-execution vulnerabilities and CPU bugs are a significant threat to confidentiality; somewhat ironically, the features offered to *increase* security of guest VMs place them at the greatest risk from this new class of attacks. Because the *untrusted* hypervisor controls resource allocation and scheduling of CVMs, any leaks that do exist can be effectively amplified and exploited. For example, a malicious hypervisor may interrupt guest execution at inopportune moments to attempt to leak microarchitectural state, or it may co-schedule attacker and victim vCPUs on different hardware threads of a shared core.

Rather than the status quo of waiting for the development of ad-hoc mitigations to new transient-execution vulnerabilities as they arise, we seek a principled and robust fix. In this regard, the most promising approach is to isolate distinct security domains on distinct cores, a technique we refer to as *core gapping*. Core gapping avoids the vast majority of shared microarchitectural state and (as we will discuss shortly in Section 5.1.2) mitigates all but two of the processor vulnerabilities known to have broken cloud VM security isolation. For a typical VM isolation scenario, core gapping requires scheduling vCPUs of guest VMs on distinct *physical* cores, thus preventing leaks to other VMs through either time-slicing context switches or sibling hardware threads. This is necessary but unfortunately not sufficient for

VM isolation; to prevent leaks across the trust boundary with privileged code (*i.e.*, hypervisor and host), we must ensure that they too execute on distinct physical cores [126]. However, the naive implementation of this approach for CVMs runs into an immediate and fundamental limitation: because the hypervisor is untrusted, the guest cannot rely on it to implement core isolation.

Our goal is to address this limitation, enabling core gapping of confidential VMs *without* a trusted hypervisor and *without* further hardware changes. We leverage the convergence of hardware support for confidential VMs to a model where a security monitor that is attested and trusted by both host and guest executes on all transitions to/from the execution of confidential vCPUs. We modify this security monitor to constrain the physical CPUs on which a confidential vCPU is dispatched and limit the causes and frequency of interrupts. Our modified security monitor also delegates all VM exit events to a remote core; we thus prevent the possibility of leaks across the trust boundary, by avoiding the context switch between a confidential VM and untrusted hypervisor.

In this chapter, we present our design for core-gapped confidential VMs (Section 5.2) on platforms such as ARM CCA, INTEL TDX, and RISC-V CoVE [253]. We implement a prototype for ARM CCA (Section 5.3) by modifying ARM’s RMM security monitor, showing that it is practical on a simulator implementing the forthcoming architecture extensions for CVMs, and evaluate its performance on a current ARM platform (without hardware CVM support). Our prototype uses novel techniques to temporarily dedicate host cores to confidential VM execution (Section 5.3.2), support fast cross-core RPCs without relying on constant busy-waiting (Section 5.3.3), and minimise VM exits by delegating interrupt management to dedicated cores (Section 5.3.4). A performance evaluation (Section 5.4) shows that thanks to these optimizations, core-gapped CVMs offer performance competitive with shared-core VMs. Finally, we discuss in Section 5.5 how the same approach could be applied to competing CVM platforms such as INTEL TDX.

5.1 Background

In this section we cover background and further motivation for our work. We present the common principles underlying emerging ISA extensions for confidential VMs, then discuss the impact and limitations of transient-execution attacks and how recent proposals address such vulnerabilities before concluding with our threat model.

5.1.1 Confidential VMs

Confidential VMs have gained traction in recent years to the point where all major architectures now include ISA extensions to support them [5, 20, 138]. CVMs offer the ability to

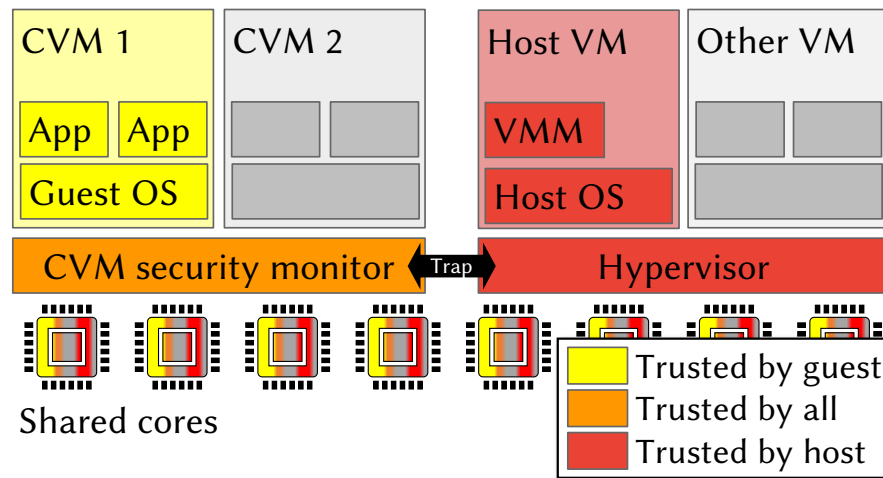


Figure 5.1 – Generic architecture support for confidential VMs [20, 138, 253]. A trusted security monitor protects CVMs, which may run on any core. Cores context switch between trusted and untrusted domains in response to VM entry/exit.

“lift and shift” existing workloads to an untrusted cloud environment while providing the capabilities and guarantees typical of more restricted trusted execution environments such as remote attestation, confidentiality, and integrity. In particular, CVMs promise to remove the hypervisor (and hence the platform provider) from the guest’s TCB.

The design of CVM ISA extensions has mostly converged toward a model in which isolation is enforced through cooperation between hardware and software components, with significant functionality being placed in platform firmware that is attested by both host and guests, and which can be replaced with a trusted virtual firmware monitor (see Chapter 3). Indeed, ARM CCA/RME [20, 175] (specified as part of ARMv9), INTEL TDX [138] (commercially available since 2022), and CoVE [253] (a leading RISC-V extension proposal) all follow a very similar design, depicted in Figure 5.1. Those extensions introduce a new privileged CPU mode which grants access to the physical memory backing confidential VMs, and in which a *CVM security monitor* runs. Physical memory ranges can be configured as confidential (and thus inaccessible to host software) using hardware mechanisms under the control of trusted firmware

Table 5.1 – Cost of timer read and IPI

	ARM CCA	INTEL TDX	CoVE [253]
Confidential VM	realm VM	TD VM	TVM
Security monitor	RMM	TDX module	TSM
Privileged mode	realm	SEAM	confidential

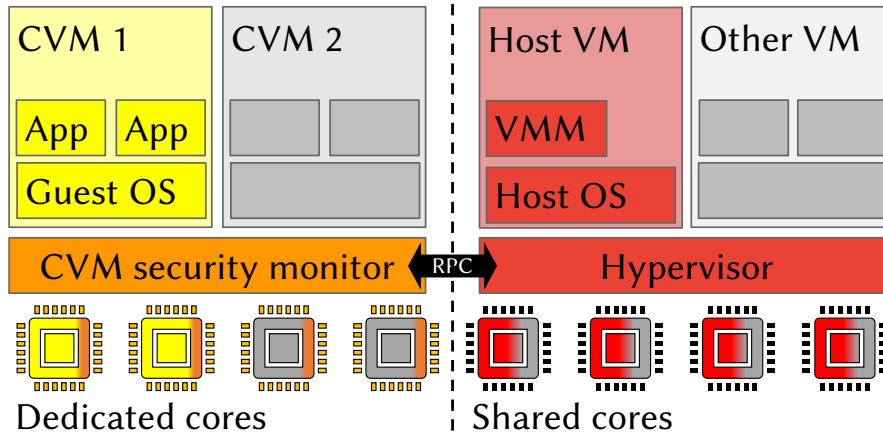


Figure 5.2 – Core-gapped confidential VMs (compare to Figure 5.1). Physical cores are bound to specific CVMs; VM exits are handled by cross-core RPCs rather than context switching.

within the confidential mode. The CVM security monitor enforces isolation between CVMs using second-level page tables, similar to how a traditional hypervisor provides isolation between VMs. Table 5.1 maps each extension’s unique terminology into our unified CVM model.

Besides managing physical memory and CVM second-level page tables, the CVM security monitor also plays the role of the arbiter between CVMs and the host hypervisor. In particular, the security monitor interposes on all transitions between the CVMs and the host to save and restore contexts, validate host-provided configuration, and selectively share the register state of guest VMs. For all situations where it would access guest context, modify guest page translations, or simply initiate guest execution, the hypervisor is required to invoke the security monitor. A call to the security monitor always implies a transition to confidential mode, and a transition back to normal mode requires explicit intervention of the security monitor to save/restore context. This prevents the host from observing guest register state on interrupts or exceptions, although it retains overall control of the CPU.

During transitions to/from the host, the security monitor may attempt to mitigate transient-execution attacks by flushing microarchitectural state; for example, INTEL’s TDX module flushes branch history on return to the host [143]. However, such flushing carries an inevitable cost, and requires that the relevant microarchitectural structures be exposed to software control; as we will discuss shortly in Section 5.1.2 it is often applied only retroactively. Moreover, flushing cannot protect against *architectural* leaks due to bugs in processor implementations [47, 219, 220, 321].

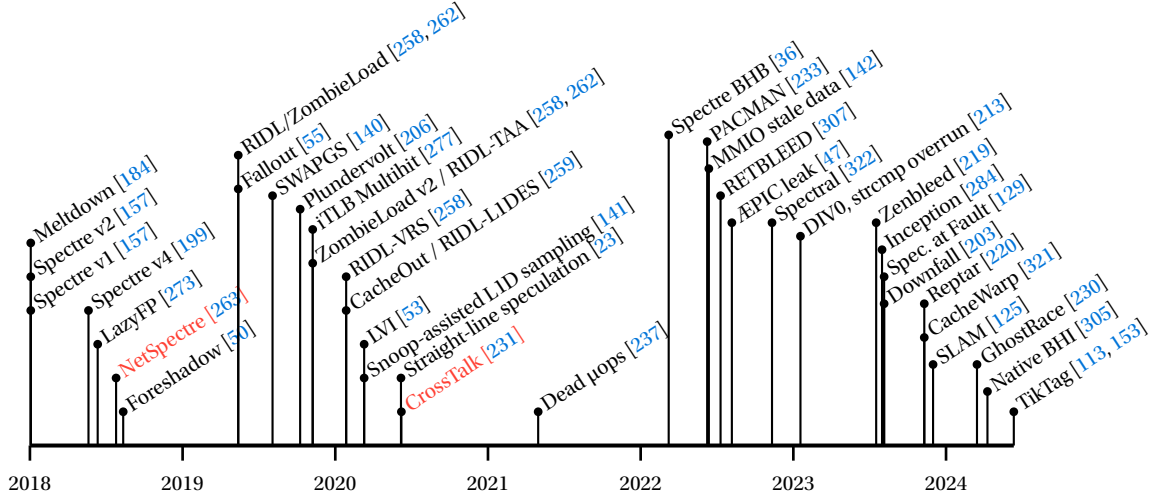


Figure 5.3 – Transient-execution vulnerabilities and CPU bugs that break security isolation. Only NetSpectre [263] and CrossTalk [231] demonstrated cross-core leaks in typical cloud VM settings.

As a notable exception to the architecture depicted in Figure 5.1, AMD’s confidential VM extension, SEV-SNP [5], relies more heavily on the CPU ISA to directly enforce isolation. Rather than a new privileged mode, SEV-SNP places responsibility for protecting guest register context on the implementation of the VMRUN instruction used to launch confidential VMs. Furthermore, the page tables for SEV-SNP VMs remain directly accessible to the host, with a shadow reverse-map structure (“RMP”) consulted by the processor on each page walk to ensure integrity of a confidential VM’s address space. A secure coprocessor manages the lifecycle of confidential VMs, supports attestation, and controls the contents of the RMP table, but is *not* directly involved in the ongoing execution of CVMs. We will discuss the challenges of implementing core-gapped CVMs on AMD SEV-SNP later, in Section 5.5.2, but for now we will not consider it further in our design.

5.1.2 Transient-Execution Vulnerabilities

While we must defer to prior work for a more thorough treatment of past vulnerabilities [54, 126, 311], we present in Figure 5.3 a timeline of disclosed vulnerabilities (including transient-execution vulnerabilities and architectural bugs) that have broken processor security isolation on mainstream CPUs (INTEL, AMD, and ARM) since 2018. Not all vulnerabilities affected all CPUs and isolation mechanisms; for example, some only affected INTEL SGX enclaves [47, 206], while others targeted features specific to APPLE’s ARM CPUs [67, 233, 298]. What is generally true, however, is that each such vulnerability represents a defeat of the same forms of hardware security mechanism that are now being relied upon to isolate CVMs.

For example, the $\text{\AE} \text{PIC}$ leak [47] (a bug in INTEL’s local APIC that exposed the processor’s internal state) was demonstrated by extracting secrets from an SGX enclave, but the same vulnerability arising today would be just as likely to leak from a TDX VM.

Across all these vulnerabilities, only *one* demonstrated a cross-core leak severe enough to warrant a security advisory from the processor vendor and mitigation by cloud providers. That bug was 2020’s CrossTalk [231], which exploited a speculative leak in a microarchitectural staging buffer used in the implementation of instructions such as `CPUID` and `RDRAND` and, crucially, that was shared by multiple cores. By contrast, the recent GhostRace [230] attack relies on multiple cores to steer transient execution, but assumes a setting with a shared kernel between attacker and victim cores; it would be mitigated by core gapping.

Notwithstanding the impact of CrossTalk, that the vast majority of vulnerabilities (30+) were *not* exploitable across core boundaries (even though many were exploitable by a sibling hardware thread) is significant, yet unsurprising. The nature of CPU microarchitecture design dictates that performance-critical structures be kept as close as possible to the execution units of a core and thus be duplicated (rather than shared) across cores; first-level (and often second-level) caches, TLBs, register files, reorder buffers, store buffers, and more are all per-core resources. Fewer shared resources leads to fewer opportunities for leaks and exploits; moreover, for the remaining shared resources located further from the core (and thus, with higher request latencies) such as last-level caches, memory controllers, and on-chip I/O devices, it becomes practical to implement explicit partitioning and resource management to close the remaining side channels [185].

To be clear, cross-core exploitation of transient-execution vulnerabilities remains conceptually possible as long as an interface to trigger the vulnerability and a side channel through which to observe the results of transient execution exist. Such side channels may include contention in the on-chip interconnect [79] or memory controller, or coherence states of memory shared between attacker and victim cores [322]. NetSpectre [263] even demonstrated a remote attack that relies solely on the timing of network traffic, although in its current form the leakage rate is extremely low (fewer than 10 bits per hour in a cloud setting).

To date, the known vulnerabilities have been mitigated (albeit after sometimes lengthy pre-disclosure delays [203]) by a combination of microcode patches, software workarounds, and new hardware. Besides the cost and engineering complexity of developing such mitigations, they can also have a significant performance and functionality impact. Past mitigations have imposed dramatic overheads [43], disabled features such as transactional memory [144], and rendered hyperthreading quasi-unusable [214] on systems that host SGX enclaves [50, 145]. We believe that the status quo of ad-hoc mitigations for each discovered attack is neither secure nor sustainable, and seek a more comprehensive solution.

5.1.3 Core Gapping as the Basis of Isolation

We are not the first to observe that cores offer a natural boundary on which to build stronger security isolation: two recent systems leverage the same observation in different contexts (we will cover other related work later, in [Section 5.6](#)). The first, Core Slicing [328] also uses isolated cores to avoid transient-execution vulnerabilities and side channels, but replaces guest VMs with statically-partitioned bare-metal tenants and hence requires hardware support. The second, Quarantine [126], runs VMs on distinct physical cores from their hypervisor as we do, but does not consider the unique challenge of confidential VMs. Quarantine is designed to protect the host from malicious guests, but not to protect guests from a malicious host. Applying Quarantine’s design to CVMs cannot give guests any better security guarantees than the status quo, as it relies on the hypervisor to implement core isolation. We seek to work within the constraints of existing hardware designs for confidential VMs, while guaranteeing meaningful (attested) isolation to CVM guests.

Like the above systems, our use of core gapping changes the way cores are allocated and CPU time is accounted for. Because host CPUs are statically bound to guest vCPUs, it becomes impractical to overcommit host CPUs by time-slicing vCPU execution. Furthermore, virtualization tasks performed by the host (VM exit handling, I/O emulation, etc.) must run on a distinct CPU from the guest vCPUs. Naively, this would seem to require more host resources, or come at an inherently large cost in performance, since fewer vCPUs can run on the same host. In reality, however, the impact is not so significant. In the specific case of infrastructure-as-a-service cloud VMs, [328] already noted that (a) host CPUs are not generally overcommitted, and (b) I/O is becoming fully virtualised through SR-IOV devices, with minimal time spent in the host handling VM exits. Indeed, while private clouds heavily rely on overcommitment, public clouds are converging to a “slice of hardware” model for IaaS VMs that offers guaranteed non-overcommitted resources for predictable performance. We anticipate that the same will also be true of confidential VMs in the cloud: they will have guaranteed CPU resources and will evolve towards fully hardware-virtualised I/O [9, 146], so we target such a setting in our design, but unlike Core Slicing we do not require it. In particular, we measure the overhead of software-emulated I/O in our evaluation.

More generally, as per-socket core counts steadily increase, the relative cost of dedicating cores to specific purposes decreases. Rather, the dominant performance bottleneck becomes off-chip bandwidth (the “memory wall”); in this regard we can expect remote exit handling to improve performance, because by running the host on a separate core we avoid evicting the guest’s working set from per-core caches and the TLB [270]. Nevertheless, in our experiments, we will follow the lead of [126] and fairly account for the resource overhead of core gapping by using the same number of *physical* cores in all comparisons.

5.1.4 Threat Model

We adopt the confidential computing threat model, which seeks to guarantee the integrity and confidentiality of guest CVMs from a malicious host, but we strengthen its handling of CPU vulnerabilities and side channels.

We consider distinct TCBs from the perspective of the host (such as the cloud provider), and guests (who run in confidential VMs). Host and guests distrust each other, but both must trust the CPU hardware, platform firmware, the CVM security monitor, and the systems that remotely attest them; software and hardware vulnerabilities in those components are generally out of scope, with an important exception: hardware side channels and transient-execution vulnerabilities that leak state (breaking confidentiality) or affect execution (breaking integrity) *only within a single core* are in scope and are mitigated by core gapping. For example, side channels in the last-level cache are out of scope (although we recommend the use of hardware cache partitioning mechanisms as an effective mitigation), but attacks exploiting the L1 cache, TLBs, store buffers, reorder buffers, etc. are all defended against. Other lower-level hardware side channels including power and electromagnetic leaks are out of scope.

Guests trust the contents of their confidential VMs and the attestation mechanisms (*e.g.*, rooted in a certificate from the platform vendor) that give them confidence in the integrity of their CVMs, firmware, and CVM security monitor. Guests can execute arbitrary binary code within their CVMs, seeking to exploit transient execution vulnerabilities in the host or in neighbour VMs. We do not aim to harden isolation boundaries within CVMs; for example, the strength of address-space isolation within a CVM remains as in the status quo. The host controls the hypervisor and thus resource allocation and execution of guest CVMs; denial of service of CVMs is therefore out of scope.

5.2 Design

The essential properties that core gapping seeks to maintain are: (a) all instructions of a confidential VM's vCPU are executed by the same host core, and (b) from the first to the last instruction of the vCPU, only guest-trusted code (*i.e.*, the security monitor) may run on that core. (From a security perspective, it is not strictly necessary to guarantee that vCPUs cannot migrate *within* the set of trusted cores used by a given CVM, but relaxing this condition adds complexity for no obvious benefit.)

To achieve these properties, as illustrated by [Figure 5.2](#) we make two changes to the basic CVM software architecture:

- The security monitor maintains and enforces a binding of CVM vCPUs to host cores. Any attempts by the hypervisor to dispatch a vCPU on the “wrong” core fail, which prevents a malicious guest’s vCPU from being dispatched on the same core as a victim VM.
- When a running vCPU exits the VM because of an exception or external interrupt and requires the service of the host hypervisor (for example, to emulate untrusted I/O devices), rather than saving context and exiting from the privileged mode back to the host hypervisor, the security monitor writes the exit cause information to shared (non-confidential) memory, permitting the hypervisor to handle it *on another core*.

The second point is the more invasive of the two changes, yet it is critical: if the security monitor were to permit a switch back to the hypervisor, it would lose control of the core, and would have no way of knowing what had executed and what secrets may have leaked through microarchitectural state. However, never returning to the hypervisor has significant implications – it starves the kernel scheduler on the affected cores, but it is difficult for a general-purpose host such as KVM to ensure that no other threads are scheduled nor any interrupts delivered to such cores. Our insight here is that we can take these cores completely “offline” from the point of view of the host scheduler, by using the existing processor hotplug support, and then dedicate them to the security monitor and its guest CVM for the life of that guest. Then, instead of executing the guest VM via a local call to the security monitor, the host is changed to turn all such calls, including running a guest vCPU, into cross-core RPCs that the security monitor handles on a dedicated core. In handling some VM exits, the hypervisor may require further action by the security monitor, for example to manipulate page tables, and these are also implemented by changing the existing APIs to cross-core RPCs.

Replacing local context switches between the hypervisor and security monitor with cross-core RPCs may add latency, but as we will show it also reduces overheads, both direct (saving/restoring register state and flushing microarchitectural structures to mitigate transient execution vulnerabilities) and indirect (cache and TLB pollution, and cold microarchitectural state due to the aforementioned flushing). Such performance tradeoffs between same-core privilege transitions and shared-memory RPCs have been explored in the literature [108, 217, 270, 304] and do not fundamentally differ here; our observation is that the remote case can not only improve performance, but also *strengthen* security isolation.

The final aspect of our design is a user-mode *core planner* component that performs admission control on CVMs, assigns cores to run them, and orchestrates the process of dedicating those cores to confidential mode and later returning them to normal execution. This can be considered on one hand as a complement to the cloud resource allocators that schedule VMs to nodes [122, 297] (logically extending them to consider resource allocation inside a node), and on the other as an extension of existing VM schedulers that seek to maintain affinity of vCPUs for NUMA locality [200, 216] (making explicit longer-term placement decisions). The primary practical change is that whereas previously vCPU threads would be “pinned” to

specific host cores only as a performance optimization, a static allocation of vCPUs to host cores is now enforced from the start of a CVM. To avoid long-term fragmentation of available cores (and thus poor locality), we envisage permitting limited changes of the vCPU-to-core binding at coarse (*e.g.*, 10s of seconds) time scales; this could be implemented in the CVM security monitor, but we defer further investigation of it to future work.

5.3 Implementation

We describe a prototype implementation of core-gapped confidential VMs for ARM CCA and LINUX/KVM. We begin with a brief overview of the relevant architecture mechanisms, before describing the basic implementation of core gapping, followed by optimizations to reduce its overhead.

5.3.1 ARM CCA

ARM’s confidential compute architecture (CCA) follows the design principles outlined in [Section 5.1.1](#). A lightweight *realm management monitor* (RMM) [24] executes in a privileged *realm world* [20] where it manages second-level page tables and CPU context to isolate confidential VMs from each other. The host runs in normal world and cannot access realm world memory. To interact with the RMM, the host must use a *secure monitor call* [25] to trusted firmware, which then switches the CPU into realm world and forwards the call to the RMM which handles the call and ultimately returns to the host, again via trusted firmware.

ARM specifies a standard RMM interface [24] to enable interoperability between host and RMM implementations. The host remains responsible for resource management and virtualization, but the RMM is responsible for validation and enforcement. This interface exposes functions for creating and managing confidential VMs, delegating memory to realm world, requesting updates to guest page tables, and running vCPUs (among others). In this model a guest’s VM exit is perceived by the host as a return from an RMM call to run the vCPU, with the return value including selected guest registers required for exit handling.

We chose to develop our prototype on ARM because, even though hardware that implements the necessary realm-management extension (RME) ISA has not yet shipped, ARM provides both a reference implementation of the trusted software for CCA [289] and an architectural simulator [26] on which to run it. Furthermore, as soon as RME-capable ARMv9 systems ship, we expect they will be able to run our prototype, since it uses only standard architectural features. By contrast, despite the earlier availability of INTEL TDX hardware, it is unsuitable for such research – a hardware-enforced security policy limits the user to running only TDX module binaries signed by INTEL (see discussion in [Section 5.5.1](#)).

Our prototype is based on the reference RMM (version 0.3.0) and LINUX/KVM (version 6.2) with ARM’s modifications to support CCA. In total, we modified a further 1030 lines of code in LINUX and 860 lines (2.7% increase) in the RMM to implement core gapping. We did not change the APIs that the RMM exposes to either host or guests, so no changes are required in the guest VM. Besides running our prototype on the ISA simulator (which is not performance accurate), we also modified it to run on current (ARMv8) hardware, to permit a performance evaluation on real servers. In this mode, the RMM runs in normal world (not the new realm mode) on dedicated cores using memory reserved for it at system boot time, but without any security isolation from the host.

5.3.2 Dedication Cores

The crux of core gapping is to ensure that no two security contexts (no two CVMs, nor a CVM and any host code) are co-located on the same physical core. This requires us to (1) inform the host that some cores can no longer be used for its own operations; (2) prevent control of those cores being returned to the host before the CVM using them terminates; and (3) ensure that the security monitor will never permit two CVMs to be co-scheduled on the same core.

We use the LINUX hotplug interface to gracefully restrict the host OS to a dynamic subset of physical cores. When taking a core offline for hotplug, the LINUX kernel migrates all tasks away from the core, re-configures interrupts to target other cores in the system, and marks it as unusable for future operations⁷. The only modification required to the existing hotplug shutdown procedure is to disable the frequency scaling request initiated by the scaling governor during clean-up, so the “offline” cores continue running at normal power and frequency settings. We then introduced a final call to the security monitor to transition the “offline” cores into realm mode rather than halting.

Once a core is dedicated to realm world, no transitions back to normal world should occur on that core until the CVM it is running terminates. To ensure this, we rely on the architecture that delivers all exceptions and interrupts occurring in realm world to the RMM. The intended use is that the RMM saves and wipes guest register state before (in typical operation) switching back to the host hypervisor to handle the exception. We instead modify the RMM so that it never returns execution to the host, effectively taking control of the dedicated cores.

Additionally, our modified RMM enforces a stricter core assignment policy to prevent the host from time-slicing two mutually distrustful CVMs on a single core. Similarly to baseline CCA operations, the host is responsible for resource management, including vCPU placement. Our prototype exposes an additional `ioctl` for the VMM to act as a user-space

⁷Since none of our target ARM platforms support hardware threads, we use the term “core” here for simplicity. However, on a threaded processor, we would dedicate *all* hardware threads of the same core to confidential VMs, and *all* threads of a single core would be bound to the same CVM.

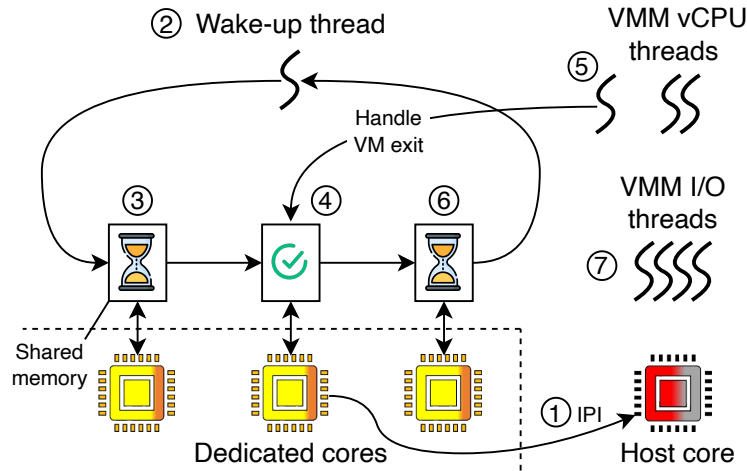


Figure 5.4 – Asynchronous call mechanism. A wake-up thread (new to our design) polls RPC channels to wake runnable vCPU threads in response to an IPI.

core-planner; we also provide a default placement policy to support unmodified VMMs. We block the host from co-scheduling vCPUs by marking a physical core as dedicated from the first entry of a vCPU on that core until the corresponding vCPU is destroyed. Any later attempt to dispatch the same vCPU on another core will also return an error to the host. Only when the CVM is terminated (either by the host, or because it exited gracefully) and its vCPUs are destroyed can the host schedule another CVM or reclaim control of the dedicated cores, whereupon they are brought back “online” through another hotplug event.

5.3.3 Remote RMM Calls

Since host and guest cannot share a physical core, core gapping replaces CPU mode switches with remote procedure calls on shared memory. Practically, the synchronous local RMM call that KVM uses to run a guest vCPU is replaced by an RPC to the RMM on a dedicated core. This call returns when the guest vCPU exits, whereupon KVM (typically) performs some emulation before issuing the next run call. Thus, although KVM is in fact the RPC *client*, an alternative (logical) view of the situation is of the RMM *calling* KVM to handle VM exits. To maximise the utilisation of guest-dedicated cores and thus the performance of guest CVMs, we must optimise for the latter view, i.e. the time between a vCPU exit event that causes a run call to return, and issuing the next run call that resumes VM execution. In the rest of this chapter we refer to this delay as the *run-to-run latency*. Meanwhile, to maximise the scalability of the system where a few host cores service many guest CVM cores, we must also minimise the RPC CPU overhead on the host. Our prototype gracefully integrates with KVM’s thread-per-vCPU design by selectively leveraging asynchronous RPCs to concurrently

manage multiple vCPUs on a single host core, and supports all the same KVM functionality as unmodified ARM CCA VMs.

We make a distinction between vCPU run calls and all other RMM invocations. Because they involve guest execution, run calls are unbounded operations (often taking tens of milliseconds or longer) whereas all other RMM calls are only for quick manipulations of CVM state, such as installing page translations. For the latter short-lived calls, we use a simple synchronous call mechanism with busy-waiting polling on both host and RMM cores. Specifically, the host thread writes the call arguments into shared memory before polling for a response. Meanwhile, on the RMM side, all cores dedicated to RMM execution either run their corresponding vCPU or poll shared memory for incoming host calls (to minimise their latency). There is a one-to-one mapping between host vCPU threads (those controlled by the userspace VMM process) and RMM dedicated cores. Thus, while a VM exit is serviced by a vCPU thread (which is ideally rare and quick) there is no useful work to do on the corresponding dedicated core.

For CVM run calls, the host vCPU thread does not busy-wait for a response, but instead blocks immediately after writing the call arguments into shared memory. [Figure 5.4](#) shows the sequence of events that will unblock it. When a vCPU exits (and thus when the host's run call must return), the RMM writes the exit reason and selected guest registers to a shared page and sends an inter-processor interrupt (IPI) to the host to signal an asynchronous return (①). ARM supports 16 possible IPI numbers, 7 of which are already reserved by LINUX, so we cannot convey information in the IPI itself and allocate only one further IPI as a notification of CVM exits. The handler for that IPI activates a wake-up thread (②) which polls shared memory until a stopped vCPU is found (③–④). The wake-up thread then unblocks the corresponding KVM vCPU threads (⑤) before continuing to poll for other vCPU exits (⑥) and finally suspending itself until the next IPI. To minimise run-to-run latency, the vCPU threads run at high priority (LINUX's FIFO scheduler class), so they typically run until completion whereupon they resume the suspended vCPU and again block until its next exit. Unlike Quarantine [126], we rely on IPIs rather than continuous polling to handle VM exits, because we found that although this increased the run-to-run latency on an uncontended system, polling competed for CPU time with other host threads such as those in the VMM process responsible for I/O emulation (⑦) resulting in worse overall performance on emulation-intensive workloads.

[Table 5.2](#) shows the round-trip latencies of synchronous and asynchronous remote RMM calls on a current ARM server ([Section 5.4.1](#)). This supports our decision to use asynchronous invocations only for the long-running calls, with busy-waiting for the rest, improving both throughput and run-to-run latency. In both cases we change only the RPC transport, but not the API exposed by the RMM to the host.

Table 5.2 – Comparison of null RMM call latencies.

Call	Latency
Core-gapped asynchronous (vCPU run calls)	2757.6 <i>ns</i>
Core-gapped synchronous (e.g., page table update)	257.7 <i>ns</i>
Same-core synchronous	>12.8 μs

Without suitable hardware, we cannot measure the cost of a same-core RMM call, but as a point of comparison we report the latency of a null call to trusted firmware (ARM EL3) on the same system. This is only part of the end-to-end RMM call path, yet takes more than four times longer than a remote call due primarily to mitigations for transient-execution attacks. Although we hope that future hardware will not require such mitigations, experience (Section 5.1.2) suggests that they may accrue over time.

5.3.4 Interrupt Management

The RMM itself executes with interrupts disabled, but intercepts physical interrupts during vCPU execution that cause vCPU exits in accordance with the RMM specification. In addition, the baseline RMM configures the CPU registers related to the local interrupt controller to induce traps, delegating their emulation to the host. We found that interrupt management was by far the most significant source of exits to the host for compute-bound workloads, with exits caused both by physical interrupts and the virtualization of the guest interrupt controller. For example, each tick of the virtual timer induces two exits, and in our experiments over 90% of exits when running CoreMark-PRO [87] are due to local timer interrupts. Such a high exit frequency might be tolerable on a shared core where interrupts are handled locally, but becomes a significant source of overhead with core gapping due to congestion on the remote host core. Moreover, because we dedicate cores to vCPUs, there is less need for a fully virtual interrupt controller.

In our prototype, we address this problem by delegating some interrupt management to the RMM. There is an intrinsic tension between maintaining a small TCB and offloading more emulation to the RMM. However, we note that some interrupt sources can be emulated very simply while significantly reducing the total interrupt count. In particular, we emulate both the virtual timer and inter-processor interrupts directly in the RMM, accounting for 150 and 70 new lines of code respectively. Note that this does not require MMIO emulation (which remains delegated to the host), but rather changes handling of CPU register writes that already trap to the RMM. By doing so we also further isolate the CVM guest from a malicious host; arguably, providing a trusted source of time to a confidential guest is of more importance than some additional emulation logic in the TCB.

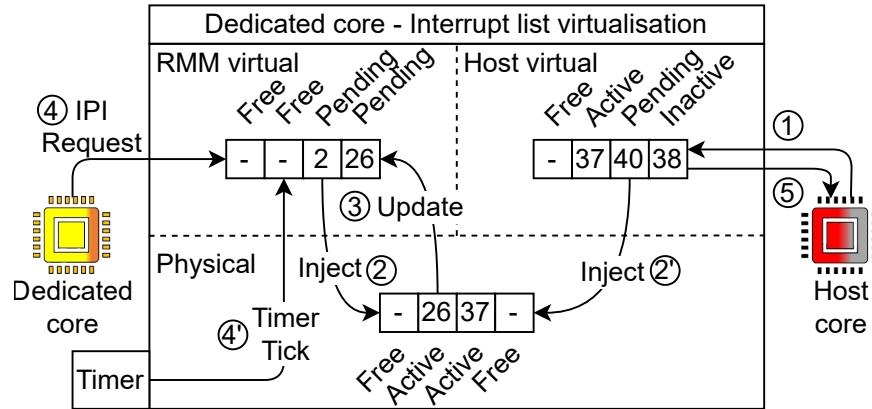


Figure 5.5 – The host observes and manipulates a virtual list containing a subset of the guest interrupts, allowing the RMM to inject interrupts without perturbing host operations.

We modified the RMM’s interrupt handling logic to make the offloading transparent to KVM. On ARM CCA, virtual interrupts are controlled by a list of active and pending virtual interrupts (`ich_lr<n>_el2` registers). The RMM interface specification expects the host to manage virtual interrupts, therefore the run call takes an interrupt list as argument, and returns a potentially updated list of interrupts on exit. We make interrupt virtualization offloading transparent to KVM by exposing a filtered list of virtual interrupts instead, hiding some interrupts from KVM while managing the true list in the RMM. Figure 5.5 illustrates interrupt management on our prototype. As a parameter to the vCPU run call, the host sends a list of virtual interrupts to install on the interrupt controller (①). The RMM then injects new pending interrupts if it can find free slots in the physical list (② & ②’), and updates the state of virtual slots to reflect interrupt life-cycles (③). During guest operation, the RMM can decide to inject interrupts in response to timer ticks or IPI requests *without host interaction* (④ & ④’). The KVM host can still request exits, for instance to inject an interrupt from an emulated I/O device, by sending an IPI as it does for shared-core VMs. On exit to the host, the RMM synchronises the slots one last time and forwards the virtual list to KVM (⑤), which can resume operation transparently.

Table 5.3 – Virtual interprocessor interrupt latency.

Configuration	IPI latency
Core-gapped CVM, without delegation	43.9 μs
Core-gapped CVM, with delegation	2.22 μs
Shared-core VM	3.85 μs

Table 5.3 reports the time to deliver and acknowledge (in shared memory) a virtual IPI; interrupt delegation is particularly effective at reducing this cost, because it avoids VM exits on both sender and receiver cores. In particular, interrupt delegation completely skips the host’s scheduler, which might delay virtual interrupt injections for shared-core VMs.

5.4 Evaluation

In this section we describe a performance evaluation of our ARM CCA-based prototype of core-gapped confidential VMs. We demonstrate that core-gapped CVMs offer competitive performance across a variety of workloads, and sometimes even slightly outperform a shared-core baseline.

5.4.1 Experimental Setup

Since ARMv9 hardware supporting CCA is not yet available and the architectural simulator is not cycle accurate, we evaluate our prototype on a cloud server using an AmpereOne SoC with ARMv8.6 cores at 3 GHz. To run our prototype on this platform without interference with the host OS, we statically reserve system memory for it at boot time (much as the CCA firmware would do), and load the RMM image into it after boot. When dedicating a core to run a CVM, we do not attempt to switch to realm mode but rather jump directly to the previously-loaded RMM running in normal world (which thus cannot enforce isolation from the host). This version of our RMM prototype has been further modified to avoid interaction with trusted firmware (EL3) and to handle all RMM calls from the host OS directly, which suffices to run CVMs without RME hardware.

Overall, we expect our evaluation platform to perform comparably to a true CCA platform, with two notable differences. First, CCA hardware will encrypt the memory of confidential VMs. On other platforms (e.g., INTEL TDX), memory encryption adds a 2–3% overhead to memory-intensive workloads [119], a cost that will be the same with or without core gapping. Second, CCA hardware performs additional physical memory ownership (granule protection) checks in the TLB miss handler. Although we cannot predict the cost of such checks, because core-gapped CVMs avoid interference from the host OS, we expect them to achieve better TLB locality than shared-core CVMs on the same hardware.

More challenging than the core-gapped CVM is the choice of baseline system to which its performance may be compared. Without suitable hardware, we cannot run the RMM in shared-core mode, as it would conflict with the host OS’s control of normal world. Therefore, our experiments use a traditional (non-confidential) shared-core VM as the baseline. This configuration avoids not only the cost of hardware memory encryption and EL3 calls (as in our own CVM configuration), but it also avoids the overhead of world switches, context save/

Table 5.4 – Interrupt delegation effect on CoreMark-PRO.

	Without delegation	With delegation
Interrupt-related exits	33954 ± 161	390 ± 3
Total exits	37712 ± 504	1324 ± 60

restore sequences for transitions to/from realm world, and RMM bookkeeping. Thus, our experiments will unfortunately exaggerate any performance overheads of core gapping, and we can expect core-gapped CVMs to offer relatively *better* performance compared to shared-core *confidential* VMs when such a comparison becomes possible.

We use LINUX 6.2 for the host kernel with the *performance* scaling governor and kvmtool as the VMM, both modified to support ARM CCA [27, 28]. To fairly account for the resource requirements of core gapping, we use the same number of physical cores in all experiments. In particular, for a workload with N cores, we run an N core VM in the baseline, restricting the VMM threads to run on a fixed set of N host cores. In the core-gapped CVM configuration, we instead run an $N - 1$ core VM (on dedicated cores) for the same workload, while pinning all VMM threads on the host to a single additional core. Therefore, the core-gapped CVMs in our experiments always run with one fewer vCPU. Unless specified otherwise, we use 16 cores for all of our experiments, thus 16 vCPUs for shared-core VMs and 15 vCPUs for core-gapped CVMs.

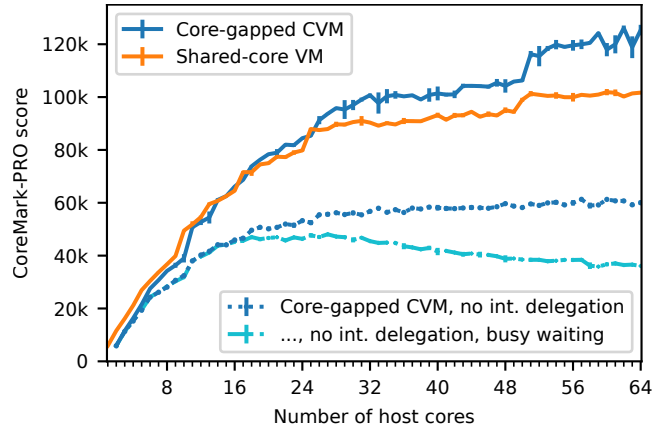


Figure 5.6 – CoreMark-PRO [87] scaling for shared-core (baseline) VMs and core-gapped CVMs.

5.4.2 Scaling a CPU-intensive Workload

We first evaluate the performance implications of core gapping on a CPU-intensive workload, and quantify the impact of our optimizations. We expect to see performance impacts from core gapping due to costlier exits to the host, contention on the host core for both I/O and vCPU threads, and RMM overhead. On the other hand, core-gapped CVMs may benefit from greater locality, potentially boosting performance.

We start by evaluating the impact of scaling up the number of dedicated cores running CoreMark-PRO [87] (a CPU-intensive benchmark), while keeping a single host core running all VMM threads. All guest VMs are assigned 16 GiB of RAM. Figure 3.7 shows that a core-gapped VM can scale to a high number of dedicated cores (here up to 63) without any contention on the host core. Indeed, for this workload run-to-run latency does *not* noticeably increase with the guest core count and remains stable at $26.18 \pm 0.96 \mu\text{s}$. The host overhead can be kept low despite high core counts thanks to the combined effect of interrupt delegation and asynchronous calls: by handling most interrupts locally we reduce the total exit count by $28 \times$ (Table 5.4), and by using IPI notifications rather than Quarantine-style yield-polling [126] we avoid saturating the host core with always-runnable VMM threads (cyan-coloured “busy waiting” lines in the figure).

Figure 5.7 shows that our prototype can also scale to multiple CVMs. Rather than a single VM with an increasing number of vCPUs, we run an increasing number of 4-core VMs and plot the aggregate throughput. Because each instance of CoreMark has only 4 vCPUs, the aggregate scales linearly, and we see that running up to 16 VMMs pinned on a single host core does not harm throughput.

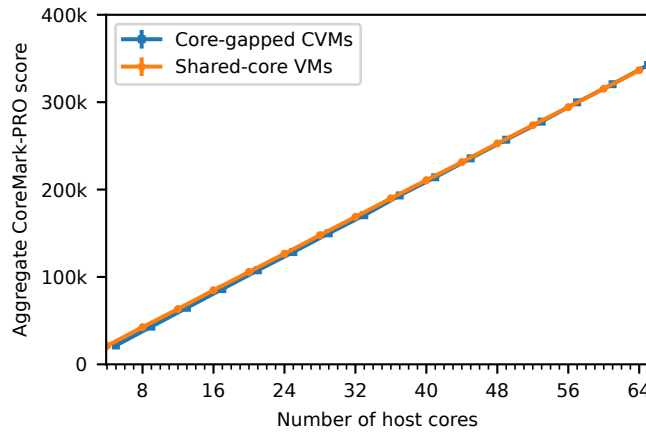


Figure 5.7 – Scaling to multiple VMs. We plot the aggregate CoreMark-PRO score for an increasing count of 4-core VMs/CVMs.

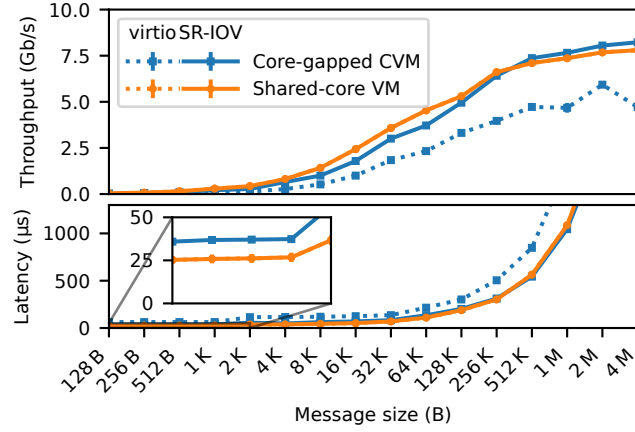


Figure 5.8 – NetPIPE [269] TCP results, 1500-byte IPv6 packets.

5.4.3 I/O Performance

We next investigate I/O performance, where we expect to see the impact (if any) of higher VM exit latencies.

To measure the performance of network virtualization, we configure our guest VMs with two different network interfaces: a virtio interface emulated by kvmtool running on the host, and a virtual function network interface of an INTEL E2000 200GbE IPU, that we assign to the guest VMs using SR-IOV pass-through. Another equivalent but unmodified system serves as a network client. Figure 5.8 shows that the virtio interface (dashed lines) has substantially (up to $2 \times$) higher latency and 30–70% lower throughput in a core-gapped CVM due to the exit-and-emulation-intensive nature of virtual I/O. However, with an SR-IOV network interface, data flows directly between the guest VM and the NIC hardware, with the host serving only to deliver interrupts. The SR-IOV configuration, which is becoming the norm for cloud VMs, achieves latency within 10–20 μ s of the baseline with up to 5% higher throughput for larger message sizes as the workload becomes more compute intensive. The additional interrupt latency can be attributed to a limitation of our prototype, which does not support direct interrupt delivery and must therefore rely on the host to inject interrupts to the VM. Direct interrupt delivery could be supported through further changes to KVM and RMM.

Finally, Figure 3.8 shows that core gapping suffers from the increased latency of virtio block I/O, achieving similar throughput only on large (>10MiB) virtual disk I/Os.

5.4.4 Application Benchmarks

As a sample network-intensive server application, we use redis-benchmark [235] to exercise REDIS v7.0.14 (also using SR-IOV); Table 5.5 reports the results. Core gapping achieves around

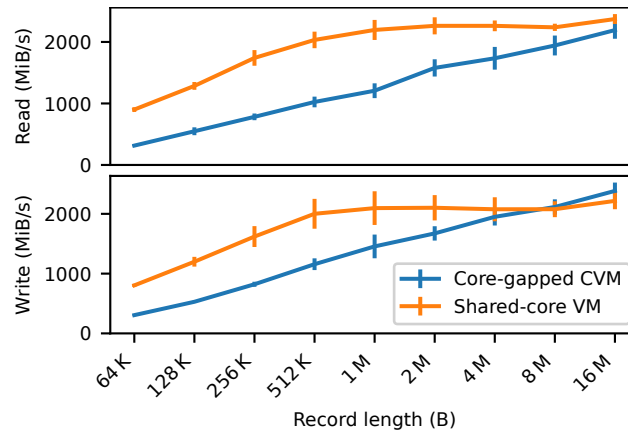


Figure 5.9 – IOzone [212] sync read/write throughput to a virtio block device, using `O_DIRECT` to bypass the guest’s cache.

Table 5.5 – Redis benchmark. 50 clients, 512-byte objects.

		Throughput	Latency (ms)		
		(krps)	Mean	p95	p99
SET	shared core	51.7	0.52	0.60	1.20
	core gapped	56.2	0.63	0.97	1.44
GET	shared core	48.8	0.54	0.64	1.20
	core gapped	55.3	0.57	0.78	1.24
LRANGE 100	shared core	11.6	1.51	2.03	2.38
	core gapped	14.5	1.24	1.56	1.82

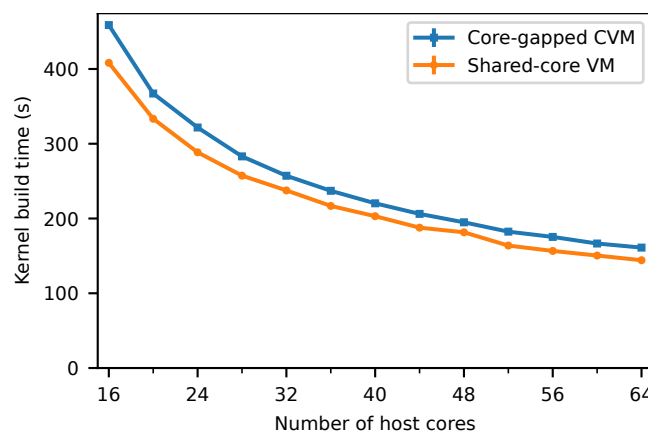


Figure 5.10 – Linux kernel build; virtio disk & 48GiB RAM

10% higher throughput in line with micro-benchmarks, considering that redis saturates the guest CPU, but also higher latency (up to 20% at p99) due to contention on the host. The longer-running and memory-intensive LRange query is an exception, delivering lower latency due to reduced contention.

Lastly, [Figure 5.10](#) shows that kernel builds on a core-gapped CVM scale similarly to a shared-core VM. This benchmark relies on virtio disk, which puts core gapping at a disadvantage due to contention for I/O emulation on the host core as illustrated by [Figure 3.8](#), yet core-gapped CVMs achieve comparable performance despite using one fewer vCPU.

5.4.5 Summary

We showed that our performance optimizations are effective at reducing VM exit rates and that core-gapped CVMs offer competitive performance with shared-core VMs, especially for CPU-intensive workloads that benefit from less host interference. Conversely, core-gapping suffers the highest overhead for emulated I/O due to its exit-intensive nature. We consider these results promising, for two reasons. First, cloud servers are moving towards full I/O offload with SR-IOV for both network interfaces and local storage. Second, confidential VMs on shared cores will have higher VM exit latencies than the non-confidential baseline used in our experiments, because of the added cost of world switches to/from the confidential execution mode. Together with hardware support for direct I/O [[9](#), [146](#)] it is therefore plausible that core-gapped CVMs will outperform shared-core CVMs in the near future.

5.5 Discussion

ARM is not the only CPU architecture at risk of transient-execution vulnerabilities. Indeed, most of our motivating vulnerabilities ([Section 5.1.2](#)) were found in INTEL and AMD systems (a fact we attribute primarily to greater attention from security researchers due to their larger market share to date in server and desktop systems). We therefore discuss how our design might apply to those CVM platforms.

5.5.1 INTEL TDX

As we noted in [Section 5.1.1](#), INTEL TDX is architecturally very similar to ARM CCA's realm-management extension. Both extensions introduce a new privilege mode running trusted firmware that manages the register context and page tables of confidential VMs, and mediates their interactions with the host hypervisor. In TDX, that trusted firmware is known as the *TDX module*, and it provides a host-facing API that is intended to be invoked through a call-gate mechanism and bears a not-unsurprising similarity to the RMM's realm management interface. We therefore expect that our design would apply directly to TDX, with a modified

TDX module that avoids relinquishing control of cores dedicated to CVM execution and supports a shared-memory RPC interface with the host rather than same-core calls.

Perhaps the biggest architectural difference between CCA and TDX is that the latter uses separate secure and insecure page tables for confidential VMs, allowing the host to manipulate untrusted portions of guest address space without calling the firmware. By contrast, the RMM is invoked for all page table modifications; thus we might expect a core-gapped version of TDX to have moderately better relative performance, due to fewer cross-core RPCs.

However, TDX also comes with a major drawback from the point of view of a systems researcher: due to restrictions implemented in the trusted code that configures and loads the TDX module that are ultimately enforced by the processor, *only* TDX module binaries signed by INTEL can be loaded. Moreover, even though INTEL has released source code for the TDX module, there is no documented mode (not even a debug mode) that would permit loading a non-INTEL-signed TDX module. While such restrictions may appear necessary for the security of the overall TDX platform, this is not the case. Just as CCA includes the measurement of the RMM in the chain of trust for remote attestation, TDX also establishes the user's trust in the specific implementation of the TDX module by including its measurement as part of the overall attestation signature. Thus, there is no specific reason that only INTEL-authored TDX modules need be attestable, let alone that non-INTEL-authored TDX modules be prevented from loading on an end user's platform.

Of course, INTEL could add support for core gapping to the TDX module themselves, but we hope they will go further and consider relaxing such restrictions in time for the wider availability of TDX hardware (at present it is available only to some cloud providers). Ultimately, enabling research only strengthens the security of platforms such as TDX.

5.5.2 AMD SEV

AMD SEV-SNP [5] differs notably from the design of other confidential VM platforms, relying solely on the CPU's instruction set (specifically, the `VMRUN` instruction) and its exception handling mechanisms to mediate control transfers between confidential VMs and the host. In particular, no software execution on the CPU is more privileged than the hypervisor, which is not trusted by CVMs. Adapting our design to provide core-gapped CVMs on AMD platforms is thus substantially more challenging. Although we could easily anticipate `VMRUN` restricting the physical cores on which confidential vCPUs could be dispatched, reporting and handling VM exits wholly on a *remote* core without relinquishing control to untrusted host code seems challenging. As a reminder, in our threat model of arbitrary unknown same-core vulnerabilities, the exception handlers that run on the same cores as the CVMs must be trusted by the guest. Thus, even though the code needed to turn a VM exit into a cross-core RPC, such as the shim hypervisor in Quarantine [126], may be simple, because it is not part

of the confidential VM that is measured, attested and protected by SEV, the end user cannot rely on its existence.

One approach to core-gapped SEV VMs might leverage its unique support for multiple VM privilege levels (VMPLs) *inside* a single CVM, and modify the architecture to reflect guest exceptions to a higher privilege level within the CVM. In combination with an extension to the VMRUN instruction to enforce a static binding of vCPUs to host cores, this could enable core gapping. However, it is not clear that such a design could avoid fatal nested exceptions, nor that it would retain sufficient control for the host; for example, the host must be able to terminate a CVM and regain control of its cores.

Overall, this illustrates the limitations of embedding complex trusted execution functionality in a processor ISA [41], even if it may be implemented by a flexible microcode.

5.6 Related Work

The most closely related work is Quarantine [126], which implements core gapping for normal (non-confidential) VMs on x86, with a trusted hypervisor. Although the same design does not directly apply to confidential VMs (see Section 5.1.3), we would expect to see broadly similar performance trade-offs. In this regard, it is interesting to note that Quarantine’s performance evaluation showed a single host core becoming saturated after around 10 guest cores, whereas our prototype scales beyond 64 guest cores. We attribute this difference to our asynchronous RPCs for long-running calls (Section 5.3.3) and delegated interrupt management (Section 5.3.4). Figure 3.7 in Section 5.4.2 supports this: when using busy waiting in place of asynchronous RPCs and disabling interrupt delegation, core gapping suffers from the same scalability bottlenecks as Quarantine.

Core Slicing [328], which seeks to address similar security threats as core gapping, proposes hardware modifications to enforce static allocation of cores and physical DRAM ranges to bare-metal cloud tenants. In contrast, core-gapped VMs work on existing hardware architectures, requiring changes only to software (and trusted firmware) to bind vCPUs to physical cores. Thus, like other VMs but unlike core slicing, core-gapped VMs can support dynamic memory allocation and deallocation, virtual I/O, host-initiated suspend/resume, and live migration (once the RMM supports it). Finally, core gapping retains the familiar VM abstraction and control-path APIs for both the guests and the host, and thus is compatible with unmodified VMs and existing user-space VMMs.

A number of systems have used cross-core RPCs in place of local exceptions to improve locality and performance. FlexSC [270] showed the benefits of handling system calls on a remote kernel core, especially in conjunction with a user-mode thread library to exploit application-level concurrency. Our handling of VM exits on a remote core has similar locality

benefits, but the VM interface precludes concurrent guest execution while handling an exit. SplitX [164] proposed hardware extensions to reduce the cost of remotely-handled VM exits; core gapping may benefit from such extensions, but we suspect they are no longer required since improvements in hardware virtualization support have reduced the rate of VM exits. A similar cross-core technique is also used to mitigate the high direct cost of SGX enclave crossings [108, 217, 304].

A number of efforts have been made to verify safety and correctness properties of the reference RMM [104, 175], but transient-execution attacks and processor vulnerabilities are out of scope; such efforts are therefore complementary to our work.

Our use of the LINUX CPU hotplug facility to dedicate cores was inspired by the LINUX kernel support for AWS Nitro Enclaves [38]. Nitro enclaves allow a VM to donate a subset of its cores to an enclave child at runtime. The donation is mediated by the Nitro hypervisor, which triggers a virtual hotplug event and removes the corresponding vCPUs from the parent VM before starting a new Nitro enclave VM with the same number of vCPUs, effectively donating them from the parent to the child. In this configuration the Nitro hypervisor must be trusted by both parent VM and child enclave. Unlike Nitro enclaves and VMs, the core-gapping host kernel and the RMM run on bare metal. The host kernel directly triggers the hotplug procedure and transfers control to the RMM. Our approach does not depend on a trusted intermediary for donating vCPUs, but requires a modification to the hotplug path to skip the final step that would otherwise physically shut down the cores or put them in a low-power mode.

5.7 Conclusion

A ceaseless tide of CPU bugs threatens the security of confidential VMs. Core gapping offers a practical mitigation to such vulnerabilities whose performance can even exceed that of traditional shared-core VMs, especially as cloud providers move to full hardware offload for I/O virtualization. We evaluated an ARM prototype, and argued for the adoption of core-gapped CVMs on platforms such as INTEL TDX.

6 Composable Isolation as a Foundation to Manage Trust

Cloud infrastructures lack abstractions for managing nuanced trust relationships in complex deployments. Modern workloads often include software components supplied by distinct parties with varying trust levels. This creates a fundamental tension: mutually distrustful components must remain strongly isolated while safely collaborating on shared sensitive data without leakage. Today, cloud customers bear the burden of addressing this tension by creating trustworthy execution environments—designing, maintaining, and either trusting or continuously verifying complex runtimes and software stacks. In this chapter we discuss how to shift that burden back to the infrastructure by providing fundamental primitives to enforce isolation and reason about nuanced trust across the system and its components.

We introduce a unified isolation model that delegates composable and attestable isolation to a single trusted security monitor: TYCHE. TYCHE is a capability-based security monitor; it provides an API for partitioning, sharing, attesting, and reclaiming resources from security domains (SDs). To provide fine-grained security guarantees, SDs can compose isolation by recursively creating and managing sub-SDs. SDs serve as the building blocks for constructing composable enclaves, sandboxes, and CVMs. TYCHE captures these relationships in attestations, allowing cloud tenants to reason about end-to-end security and manage trust in the cloud.

TYCHE preserves compatibility with existing hardware and software stacks while enabling a fundamentally different cloud model centered on fine-grained trust management via composable isolation. It runs on commodity x86-64 hardware without security extensions, and a RISC-V prototype demonstrates portability across platforms. Our SDK runs and composes unmodified workloads as sandboxes, enclaves, or CVMs within SDs with minimal overhead. TYCHE supports complex cloud scenarios, such as confidential inference among mutually distrustful users, model owners, and CSPs.

6.1 Introduction

The cloud lacks abstractions for managing heterogeneous trust. Modern workloads are multi-party: even within a single virtual machine (VM), container, or process, components from parties with varying levels of trust—libraries, frameworks, runtimes, services, and cloud

service provider (CSP) infrastructure—form pipelines with complex trust relationships. Each component holds sensitive assets—some must remain private, such as ML models or proprietary code, while others, such as user information or input data, must be shared with untrusted components that perform essential processing but may leak them to unauthorized parties. We call resolving this tension *controlled sharing*.

Current cloud abstractions, such as containers and VMs, focus on ease of deployment but provide only coarse-grained trust management. Even trusted execution environments (TEEs) only enforce a binary trust model: software inside a confidential VM (CVM) [138, 175, 281] or enclave [22, 136] is trusted and its confidentiality and integrity is guaranteed against the untrusted CSP’s hypervisor or privileged code. This model ignores the heterogeneous trust relationships that exist within the software stack of a single TEE. The burden then falls on tenants to manage the different levels of trust inside their deployments and to implement controlled sharing themselves.

Tenants typically address this burden by adopting one of two approaches: (1) external isolation, running components in separate TEEs and ensuring their functional correctness, or (2) internal (intra-TEE) isolation.

In the external approach, each component runs in its own CSP-provided TEE. The TEEs mutually attest before exchanging encrypted sensitive data over untrusted memory. This protects each TEE from the external environment but fails to provide any guarantees against leakage once one TEE hands data to another. As a result, the overall system depends on trusting the full content of all TEEs to not disclose data to unauthorized parties. Since TEEs may contain proprietary code, these mutual trust assumptions create unrealistic interdependencies between parties and restrict their ability to independently update and manage their software stacks.

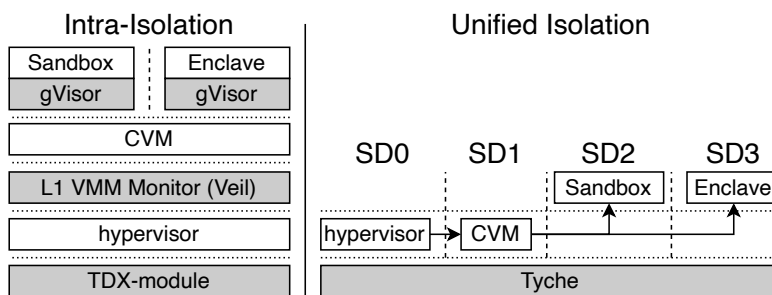


Figure 6.1 – Composing enclaves and sandboxes within a CVM. Left: Intel TDX intra-isolation, with gVisor [316] sandboxing both an application (sandbox) and a VMM-provided enclave. Right: our approach. Arrows indicate management dependencies.

Intra-isolation composes isolation boundaries within a TEE to refine trust among the components it contains. It leverages the TEE's internal privilege hierarchies—rings, VMPLs in AMD SEV-SNP [5], L1/L2 partitions in TDX [138]—to nest enclaves or sandboxes within CVMs, where more privileged software enforces isolation for less privileged components. Depending on the threat model, this provides either confidentiality [12, 327], *i.e.*, protecting an asset from the rest of the VM, or encapsulation [316, 319], *i.e.*, preventing an untrusted component from leaking sensitive data.

Intra-isolation is a popular research direction, and Table 6.1 surveys representative examples. However, each system targets a specific isolation scenario—a particular platform, threat model, and nesting configuration—and none were designed to interoperate. Some are platform-specific (*e.g.*, Veil [12] on AMD, Erebor [319] on INTEL), others compete for the same privilege layer (*e.g.*, Veil and Verismo [327] both require VMPL0), and hardware constraints can prohibit certain combinations outright (*e.g.*, no SGX enclaves inside TDX CVMs).

Yet real workloads require composing multiple forms of intra-isolation simultaneously. This is common in cloud scenarios, where distrustful entities must jointly process sensitive data: each party needs to protect its own assets while preventing leakage by the other. Figure 6.1 (left) illustrates a concrete instance: a workload must process sensitive user data using proprietary code or resources that the user does not trust. One such example is private LLM inference in the cloud: the user must reveal prompts to an untrusted LLM runtime without allowing exfiltration, while the model owner must keep its proprietary model hidden from the rest of the CVM. Supporting this controlled sharing demands composing the CVM with nested sandboxes and enclaves. Since SGX is unavailable inside TDX CVMs, an L1 VMM monitor (*e.g.*, similar to Veil [12]) emulates nested enclaves: the model owner's runtime executes inside the enclave, while the user controls the surrounding CVM to encapsulate the enclave and prevent prompt leakage. The hierarchy deepens further—ptrace-gVisor [316] mediates the enclave's access to CVM resources (network, filesystem) or sandboxes other applications, while NaCl [315] or Wasm [121] compartmentalize code within the enclave, as in Ryoan [133].

This example exposes the fundamental limitation: intra-isolation inevitably stacks mechanisms and enforcement layers. Each layer duplicates enforcement across hardware and software, expanding the trusted computing base (TCB) and forcing developers to integrate, configure, and maintain sub-systems that were never designed to work together. Correct isolation becomes fragile and error-prone, and attestation grows complicated by uncertainty about which layers and policies must be measured to guarantee end-to-end security. In the LLM example above, to ensure prompts are not leaked, the user must not only attest the TDX module and that the CVM's OS and gVisor sandbox are correctly configured, but also that the privileged software implementing nested enclaves does not provide a covert channel through

Table 6.1 – Nested composition of isolation abstractions in prior work. Rows denote the outer isolation boundary and columns the inner boundary. Prior systems support only specific combinations, each requiring dedicated hardware or software extensions, motivating the need for a unified composable isolation solution.

Outer \ Inner	No Nesting	Sandbox	Enclave	VM	CVM
Sandbox	gVisor [316], Wasmer [121]	Capsicum [303]	-	-	-
Enclave	SGX [136], Nitro Enclave [38]	Ryoan [133]	Nested-Enclave [224]	-	-
VM	VT-x [292]	Hyperlight [201]	SGX [136]	KVM Nested-Virt	Hyper-V confidential L2
CVM	TDX [138], SEV-SNP [5]	Erebor [319]	Veil [12]	OpenHCL [197]	-

which prompts could be exfiltrated, bypassing the sandbox entirely. No single attestation today provides such an end-to-end guarantee.

Our insight is that existing isolation solutions differ in implementation and trust model, but they all aim to control and attest which components can access which resources—that is, to enforce access and boundary control. Intra-isolation duplicates these mechanisms to compose isolation: it creates nested boundaries to refine control over sub-components at different layers of the stack. This duplication is necessary only because the cloud lacks a way to compose access and boundary control directly.

We thus propose a third approach that makes composable isolation a first-class cloud abstraction. A single trusted entity externally provides and attests an abstraction centered on composable access and boundary control. By making composition a core feature, it allows components to define and attest custom boundaries and support diverse trust models without burdening tenants with enforcement. The result is lower deployment complexity, a smaller software and hardware TCB, and precise management of nuanced trust relationships in the cloud.

Our approach raises two key research questions: (1) *What small set of primitives enables a single abstraction to compose isolation and support fine-grained heterogeneous trust models?* (2) *How can we support backward compatibility with existing software and hardware stacks while introducing a fundamentally different cloud isolation model?* These questions guide our design and ensure our solution is both general and practical.

We realize this approach in TYCHE, a security monitor that introduces a unified isolation abstraction for composable isolation: security domains (SDs). An SD is an execution environment with access to a controlled subset of the machine’s resources. Software defines

custom isolation boundaries using a small set of primitives provided as software capabilities with clear semantics, enabling creation of SDs, resource transfer or sharing, and reclamation. Attestation reports an SD’s capabilities and covers its entire lifecycle, making access and boundary control explicit: how the SD was constructed, how and with whom it shares resources, and how its resources can be revoked. Composition is intrinsic: SDs can recursively create children by subdividing or sharing their own resources. TYCHE tracks and attests each SD’s resources, interactions, and content, explicitly distinguishing between shared and exclusive ones. This small set of primitives subsumes prior mechanisms: exclusive access supports confidentiality and integrity, while control over shared resources and interaction enable encapsulation.

TYCHE runs on bare metal at the highest privilege level, attested by a hardware root of trust [285], and enforces isolation for all other software running as SDs. Implemented in RUST [250], it uses standard hardware access-control mechanisms and supports deployment across platforms. We present a full implementation on x86-64 using virtualization technologies [147, 292] and a firmware prototype on RISC-V [302]. For compatibility, our TYCHE SDK allows LINUX SDs to compose sandboxes, enclaves, or CVMs as separate SDs; this replaces intra-isolation mechanisms (Figure 6.1, right) and covers the entire design space of Table 6.1 with TYCHE’s unified semantics, attestation, and enforcement.

Our evaluation on x86-64 shows that TYCHE incurs only a few percent overhead compared to native execution for diverse workloads—including web servers, databases, key-value stores, and LLM inference—running SDs as enclaves, CVMs, or nested enclaves within CVMs. In a confidential LLM inference scenario with mutually distrustful users, model owners, and CSPs, the slowdown is just 2% compared to bare-metal LINUX.

In the remainder of this chapter, we first elaborate on the background and motivation (Section 6.2), provide a high-level overview (Section 6.3) before describing TYCHE’s API and capabilities (Section 6.4), their implementation and enforcement on hardware by platform-specific backends (Section 6.5), and the TYCHE SDK (Section 6.6). We then evaluate TYCHE (Section 6.7) and discuss related work (Section 6.8) in security monitors and micro-kernels from which we drew inspiration for TYCHE’s design.

6.2 Background

Observations: Computer system security requires resource (*i.e.*, memory, CPU, interrupts, devices) and fault isolation between software components that implement different tasks or services. Isolation is however rarely absolute, as components need to cooperate to carry out computations. For instance, two processes might share memory; isolation between a VM and a hypervisor is unidirectional to enable, *e.g.*, emulated devices or migration. Isolation

in systems is thus more accurately described as an attempt to enforce the least privilege principle [255]: each component should be granted only the minimal access rights it needs to perform its intended operation. Historically, systems were designed under the assumption that a single organization controlled the entire software stack and hardware, but that assumption no longer holds as cloud platforms and modern deployments combine components from multiple independent sources and place critical privileged components under the control of external providers in the cloud.

As modern systems integrate components from diverse sources, enforcing least privilege increasingly relies on *compartmentalization*: structuring the system so that each component's access to the machine is tightly bounded. *Confidential computing* provides one form of compartmentalization by preventing access from more privileged code. It departs from traditional system designs that conflate privilege with trust and treat resource management as justification for access. Examples include enclaves [98, 136, 191] and CVMs [5, 138, 175], which differ mainly in how they interface with the surrounding system, not in their underlying trust model. Another form of compartmentalization is *encapsulation*, which restricts how a component may interact with the rest of the system, often to prevent information leaks or interference with other components. Sandboxing mechanisms [121, 314, 315], as well as containers, are examples of encapsulation.

While often treated separately, confidential computing and encapsulation are two sides of the same coin [107]. Both aim at controlling access to resources. Encapsulation focuses on *sharing*: explicitly granting a component-controlled access to resources such as memory or interfaces. Confidential computing, in contrast, focuses on *exclusivity*: ensuring that a component's resources cannot be accessed by others, either via access control mechanisms [128, 136, 191] or encryption and integrity guarantees [5, 138]. Existing solutions often emphasize only one side, reflecting the trust model of a particular component and producing rigid abstractions—for example, the binary notion of trust in TEEs—without recognizing the common underlying goal. From a holistic cloud security perspective, all solutions are mere instances of access and boundary control. Such control enables higher-level properties such as confidentiality, integrity, encapsulation, and controlled sharing: the ability to selectively expose component resources while guaranteeing they cannot be leaked to unintended parties. To address the varying components' trust models in a coherent whole, isolation needs to be *composed*, *i.e.*, components must be able to define and control their own boundaries. We define composable isolation as the ability to assemble isolation boundaries in a way that preserves and combines their security guarantees.

In a cloud setting, where control over software and hardware may lie with an external party, isolation must be *attested*. Attestation demonstrates to a local or remote entity that a known software version is running on a platform and is isolated from the rest of the system according to the intended security policies. It typically relies on a hardware root of trust, such as the

CPU [5, 136, 138] or a Trusted Platform Module (TPM) [285] and may also involve trusted software.

Lessons learned and goals: From the discussion above, two key properties emerge as fundamental for composing isolation in heterogeneous cloud systems. First, management must be decoupled from access, and trust decoupled from privilege, reflecting the insight from TEEs. Second, access and boundary control must be attestable, so it is possible to verify that a component’s interactions and resource accesses respect intended policies. These lessons address our first research question: *What small set of primitives enables a single abstraction to compose isolation and support fine-grained heterogeneous trust models?* We argue that the two sufficient primitives are *attestation*, and *access and boundary control*. With those, we can not only re-construct existing higher-level abstractions but also provide finer-grained and composable isolation.

Approach: Providing these primitives while addressing our second research question, *i.e.*, preserving the existing software stack, is challenging and requires careful design and implementation. In particular, it requires designing an interface that is small and easy to integrate with existing software, while implementing an enforcement that does not disrupt existing software hierarchies. For this, we identify capabilities and security monitors as attractive candidates.

Capabilities provide a structured and explicit mechanism for access control. They are unforgeable tokens—created either in software by a trusted entity [154] or implemented in hardware [15, 310]—that encapsulate access to an object. Holding a capability grants specific access rights to the object, while transferring it to another component revokes access from the sender and grants it to the recipient. Capabilities are used by security-focused kernels [154] and provide clear semantics on resource access and sharing.

Monitors are small software components that conveniently *retrofit* [69, 320] new isolation boundaries into the existing software stack with minimal disruption by running at higher privilege levels, *e.g.*, through virtualization [69, 150, 191, 320]. They strive to remain passive unless their isolation services are used, leaving existing software execution mostly unchanged, *e.g.*, they do not replace the kernel for process isolation. They are popular for both confidential computing [12, 69, 98, 128, 150, 320] and encapsulation [34, 264, 319] to enforce fine grained access control.

Put together, a monitor that implements capabilities for access and boundary control can provide a single abstraction to compose isolation, support fine-grained heterogeneous trust models, and remain compatible with existing software and hardware stacks. This is what we design and implement with TYCHE.

Unlike systems in Table 6.1, TYCHE covers all scenarios with a single enforcement layer, provides coherent attestation across multiple (potentially nested) SDs, and enables cases

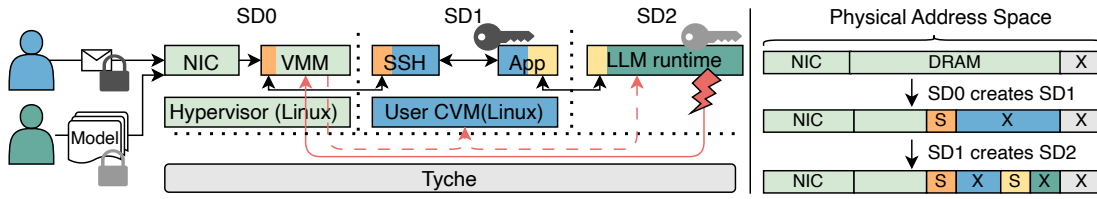


Figure 6.2 – Mutual-distrust LLM inference deployment on top of Tyche. The left side shows the participating SDs—SD0 hypervisor, SD1 CVM, and SD2 nested enclave. The right side shows the machine’s physical memory address space as SDs are created; the colors indicate the regions accessible to each SD on the left, distinguishing exclusive regions (X) from shared ones (S). The encrypted model, prompts, and replies traverse memory along the black path, while interrupts are routed as shown by the red arrows.

not addressed by prior work, such as private shared memory between two TEEs. Because its semantics are decoupled from the underlying platform, TYCHE achieves this uniformly across hardware; the single enforcement layer further reduces deployment complexity, making TYCHE both general and portable.

6.3 Overview

In this section we present an overview of TYCHE’s design, its threat model, and an example deployment in Figure 6.2 that we will use throughout the paper.

6.3.1 Architecture

The TYCHE security monitor exposes a capability-based API for creating and isolating security domains (SDs), its core abstraction. All software other than the monitor runs inside an SD, an execution environment whose configuration and owned capabilities define its access to machine resources—cores, devices, memory regions, and interrupts—attested by TYCHE as either shared or exclusive. An SD can create child SDs, partitioning or sharing subsets of its resources and transferring control to children on selected cores through capability operations. Rather than catering to a fixed trust model, SDs provide a unified, attestable abstraction for controlling shared and exclusive resources, accommodating a wide range of security policies, such as sandboxes, enclaves, and CVMs, and enabling their composition.

The API decouples resource management from the attestable enforcement of access restrictions. Capabilities represent resources and govern SD interactions, letting domains implement their own isolation boundaries and scheduling policies. TYCHE tracks resource allocation and attests whether resources are shared or exclusive, capturing and enforcing dependencies between SDs. It guarantees parents can reclaim resources or cores, while chil-

dren are assured exclusive resources remain so unless explicitly shared, and that revocation or control transfer do not leak information.

Local and remote attestation make SD relationships explicit and verify end-to-end guarantees. Attestation has two parts: (1) a TPM-rooted measurement of the boot process and a public key, binding a TYCHE binary to a physical machine and ensuring it runs alone at the highest privilege; and (2) a report generated by TYCHE and signed with the private key describing the SD's resources, how they are shared, delegated to children, or can be reclaimed.

TYCHE's isolation model unifies compartmentalization and supports the composition of confidential computing and encapsulation. Confidentiality and integrity derive from exclusive access; encapsulation is enforced by restricting a child SD's interactions and overlap of resources to trusted peers. Recursive SD construction enables composition, letting each domain define its own boundaries.

TYCHE's platform-independent capabilities make it portable across architectures, with hardware-specific backends enforcing their semantics efficiently. On INTEL, TYCHE uses VT-x [292]'s extended page tables and the IOMMU [147]; on RISC-V, it runs as M-mode firmware using Physical Memory Protection [302]. The TYCHE SDK adds a kernel driver to LINUX environments to build nested sandboxes, enclaves, and CVMs using SDs. This compatibility with existing hardware and software stacks ensures TYCHE's practicality.

6.3.2 Threat Model

TYCHE assumes the underlying hardware, including physical devices and access control mechanisms, is trusted and part of its TCB. The CSP and tenants are adversarial. We consider an attacker, running arbitrary code within an SD, restricted to an authorized subset of the machine's resources, such as cores, memory, and device configuration space. In particular, the attacker might try to exploit: (1) the monitor's API, (2) device configuration space, and (3) privileged instructions to, e.g., emit or disable interrupts. The attacker aims to access resources or SD state outside of its or the device's authorized sets, compromise the monitor's metadata, steal its private key, or hog resources to prevent revocation.

Out-of-scope:

Physical attacks, such as accessing DRAM or the PCI bus to read the monitor or an SD's memory, are out of scope of the current implementation, although they could be mitigated with hardware support, e.g., total memory encryption [137] and PCI bus encryption [225].

Side-channel-based attacks are not explicitly addressed by TYCHE, beyond appropriate flushes upon transitions, as it does not track shared micro-architectural state. They however can be mitigated within the current implementation through core partitioning [57, 328], as discussed in Chapter 5, and physical memory allocation based on cache-coloring [84, 300].

Denial-of-service attacks to exhaust the monitor's memory are possible, but they only

prevent the creation of new SDs and do not prevent the revocation, attestation, or isolation enforcement of existing ones. On x86-64, they can be mitigated by requiring SDs to supply memory for the monitor’s metadata. Inherent to the cloud, the CSP can deny service by turning off the platform [40].

6.3.3 Running Example

Figure 6.2 shows a public cloud deployment where a user performs LLM inference with a proprietary model. The threat model involves full mutual distrust: the user does not trust the CSP or model owner with prompts, the model owner does not trust the CSP or user with model weights, and the CSP does not trust either party. Private inference in this setting requires confidential computing to protect prompts and weights, composed with encapsulation to prevent the LLM runtime from leaking prompts.

At boot, TYCHE controls all resources and assigns them via capabilities to SD0, the CSP’s LINUX+KVM [178] hypervisor running TYCHE SDK. To instantiate the user CVM, SD0 uses the SDK to partition its memory, creating SD1 with exclusive memory (blue in Figure 6.2), shared memory (orange), and CPU cores. SD0 then transfers control to SD1 on those cores. Once booted, SD1 (LINUX + TYCHE SDK) creates SD2 for the LLM runtime in userspace, partitions its exclusive memory, and transfers a region (dark green) to SD2, forming an enclave isolated from both the CSP and the CVM but encapsulated by SD1. The model owner attests SD2’s exclusive memory before provisioning the encrypted model. Communication uses shared memory: SD0–SD1 share a region for VIRTIO [179] networking (orange); SD1–SD2 share a private region (yellow), not accessible by SD0, to exchange plaintext prompts and replies. Encrypted prompts arrive at SD1 via orange, are decrypted into private memory (blue), and forwarded to SD2 via yellow; replies follow the reverse path, encrypted by SD1 before being sent to the VIRTIO network. As there is no direct communication between SD2 and SD0, SD1 fully encapsulates the untrusted LLM runtime in SD2 and thus prevents prompt leakage. The model is delivered encrypted via orange, passed to SD2 through yellow, and decrypted in its private memory (dark green).

All parties use TYCHE’s remote attestation: (1) The CSP confirms it retains resource control and manages interrupts (red arrows). (2) The user confirms SD1 is isolated from SD0 and encapsulates SD2, which only communicates with the CVM. (3) The model owner confirms SD2 runs in exclusive memory without leaking the model. (4) The user and model owner confirm interrupts do not leak data and that TYCHE zeroes memory before returning it to the CSP.

Table 6.2 – Tyche’s API

Call	Description
CREATE	Create a security domain
SET/GET	Set/Get a security domain’s register or policy
SEND	Transfer a capability ownership to an SD
SEAL	Seal a security domain
ATTEST	Attest a security domain
ENUMERATE	Discover info. about owned capabilities
SWITCH	Control transfer into a security domain
ALIAS	Create new memory region by aliasing one
CARVE	Create new memory region by carving one
REVOKE	Revoke a capability’s child
GETCHAN	Create a channel from an SD or existing channel

6.4 TYCHE Design: API & Capabilities

This section addresses our first research question by designing a small set of primitives to compose isolation and support heterogeneous trust models. We focus on primitives for access and boundary control with clear, predictable semantics. To achieve this, we devise software capabilities centered on sharing, transferring, and reclaiming resources. Our capabilities make shared access and interactions explicit while decoupling policies from mechanisms [163] for portability across hardware platforms. TYCHE capabilities encode enough information to attest how an SD was created, what interactions it may have with others, and under which conditions it can be revoked. This explicitness and fixed semantics gives the system predictable behavior, enabling long-term reasoning about isolation guarantees and trust relationships from a single attestation.

TYCHE capabilities are unforgeable tokens issued by the monitor. They are owned by security domains (SDs) and mediate access to two object types: memory regions and SDs. Memory capabilities grant access to physical memory ranges, while SD capabilities govern SD management, *i.e.*, creation, configuration, attestation, and execution. CPU cores and interrupts fall under SD capabilities, and devices are modeled as SDs. In this section, we show that these two capability types and the narrow set of operations in Table 6.2 to derive new capabilities are enough to isolate SDs and manage their nuanced trust relationships.

6.4.1 Capability Derivation Trees

TYCHE maintains two capability derivation trees (CDTs): one for memory regions and one for security domains. CDTs, widely used in capability systems [154, 266], derive each capability from an existing one, inheriting equal or reduced access. Derived capabilities appear as children of their parent node in their trees. CDTs ensure (1) monotonicity of operations, (2) a record of operations, and (3) efficient cascading revocation of an entire subtree by revoking its root.

In TYCHE, we further leverage the CDT structure to track overlapping resource access and dependencies between SDs, ensuring these relationships are reflected in attestation and that operations always leave the system in a consistent state. The region CDT tracks exclusive and shared access to physical memory, while the SD CDT encodes SD management hierarchies, scheduling, and interrupt routing. Both trees support efficient revocation and explicitly encode access policies and inter-object dependencies.

6.4.2 Memory Region Capabilities

TYCHE region capabilities offer a compact, attestable representation of shared or exclusive access to physical memory ranges, with security *attributes* that define guarantees upon revocation. TYCHE initializes the CDT with a root memory region marked as *exclusive*, defined by a start and end address, access rights (read, write, execute), and an empty set of attributes. TYCHE transfers this root region's ownership to SD0, the first domain to run on the machine.

Memory operations: An SD creates new region capabilities from owned ones using TYCHE's *alias* and *carve* API calls (Table 6.2). *Alias* creates a child region capability, marked as *aliased*, for a subrange of the parent region's physical addresses with equal or reduced access rights while preserving the parent's access. A *carve* similarly creates a subregion but removes access to it from the parent region (see Figure 6.3, hatched area). If the parent region was *exclusive*, the carved region is *exclusive*; otherwise, it is marked as *aliased*. Carving enables

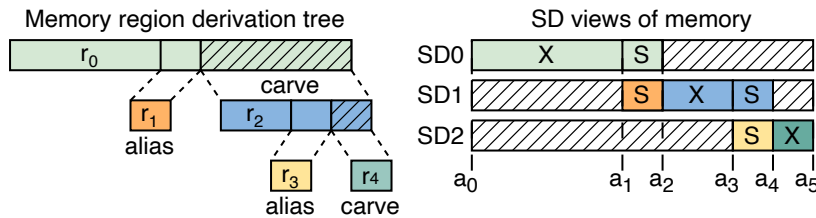


Figure 6.3 – Memory region derivation tree and SD memory views based on Figure 6.2. New regions are created by carving and aliasing an existing region. Sends between SDs are omitted. (S)hared and e(X)clusive memory is reported on views.

confidential memory: an exclusive region stems from an unbroken chain of carves and no operation outside its subtree can alter its exclusivity. TYCHE ensures a capability's access rights and exclusivity are determined locally, based on its initial range, exclusivity status, and direct children. In [Figure 6.3](#), r_0 initially grants exclusive access from a_0 to a_5 . After an alias (r_1) and a carve (r_2), it retains exclusive access only from a_0 to a_1 and shared (aliased) access from a_1 to a_2 , regardless of further subdivisions of r_1 or r_2 .

Memory management: TYCHE's *send* and *revoke* calls let SDs transfer and reclaim memory. *Send* transfers ownership of a region, allocating it to the receiver. *Revoke* acts on a parent region to undo a child alias or carve, letting the parent's owner reclaim memory sent via that child. Revocations cascade, deleting the entire subtree rooted in the revoked capability: revoking r_2 from r_0 also revokes r_3 and r_4 , restoring r_0 's access from a_2 to a_5 . To provide security guarantees to the receiver, TYCHE allows the sender to optionally attach *attributes* to the transferred region: (1) *hash* captures a cryptographic hash of an exclusive region's content, enabling the receiver to ensure it contains the correct initial data; (2) *clean* ensures the region is zeroed upon revocation to prevent data leakage; and (3) *vital* revokes the receiver if the capability is revoked, enforcing a minimal memory set necessary for functionality. Attributes are set only when sending to an unsealed receiver ([Section 6.4.3](#)), are bound to ownership rather than the CDT, and are non-monotonic.

The receiver then inspects received regions using *enumerate* or *attest*. These calls report a region's status (exclusive or aliased), initial range, access rights, attributes, and direct children, whether owned or not, as shown for r_2 in [Listing 6.1](#).

6.4.3 Security Domain Capabilities

TYCHE initializes SD0, the first SD which owns the root memory region, runs on all cores, and handles all interrupts; all other SDs are created from subsets of these initial resources.

SD creation and configuration: *Create* enables an SD to spawn a child in the CDT and obtain an SD capability referencing the newly created domain. The capability enables the parent to configure, send regions, seal, attest, schedule, and revoke the child. The child is initially *unsealed* and cannot execute. The parent configures the child via *set*, specifying per-core registers state and SD *policies*. Policies define the child's allowed cores, permitted monitor calls and whether they are allowed from user space, whether it can receive new capabilities after *sealing*, and interrupt policies. Policies are monotonic: TYCHE rejects *sets* exceeding the parent's rights. Memory is provisioned as in [Figure 6.4](#), with the parent aliasing and carving its regions before sending them. *Seal* makes the SD executable and prevents further sets to its registers and policies. Revoking an SD triggers the cascading revocation of all its owned capabilities and its CDT subtree.

```

sd1 = domain {r1, r2, sd2}
    | registers.HASH: 8988ef57...
    | cores: 0b11
    | mon.api: 0b1111111111 | RECEIVE
    | interrupts: {
    |   0 -> {Report, registers: 0b0},
    |   ...
    | }
sd2 = domain {r3, r4}
    | registers.HASH: 978de00f...
    | cores: 0b01
    | mon.api: 0b00001110000 | !RECEIVE
    | interrupts: {
    |   0 -> {Not report, registers: 0b0},
    |   ...
    | }
r1 = aliased a1 a2 with RW_
r2 = exclusive a2 a5 with RWX, HASH | CLEAN | VITAL
    | HASH: 755ee2b2...
    | alias at a3 a4 for r3 with RW_
    | carve at a4 a5 for r4 with RWX
signature: a0e0d23f26564bd5...

```

Listing 6.1 – Simplified attestation for SD1 with Figure 6.3’s nomenclature. Allowed monitor API calls are encoded as bitmaps based on the order from Table 6.2 (e.g., bit 0 is create).

SD execution & interrupts: Control transfers between SDs, *i.e.*, transitions on a core, occur via explicit monitor calls or upon interrupts. The *switch* API call implements a call-return model: TYCHE ensures the callee is authorized to run on the core, saves the caller’s state, loads the callee, and transfers control. Switching to a child requires its SD capability; a switch with no SD argument returns to the parent.

Interrupts trigger SD control transfers. They propagate through the CDT, which encodes routing and handling policies. For each interrupt vector, an SD’s interrupt policies specify whether to *Deliver*, *Report*, or *Not report* the interrupt, and which registers the parent can access during handling. An interrupt marked as *Deliver* is received by the SD directly. When an interrupt not marked as *Deliver* occurs, TYCHE preempts the SD and walks the CDT upwards to transfer control to the first SD (the handler) it finds with a *Deliver* policy. To the handler, the interrupt appears as a return from a switch to its direct child, with the interrupt acting as the return value. After taking care of the interrupt, the handler resumes the child’s execution by performing a switch. The return path walks the CDT downwards, with TYCHE reporting the interrupt to all SDs with a *Report* policy in a manner similar to the original handler SD. These SDs observe the interrupt as if it originated from their direct child and decide whether to resume execution. Those with a *Not report* policy are skipped.

Consider the deployment in Figure 6.2. SD2 triggers a divide-by-zero exception (interrupt vector 0). Per Listing 6.1, SD2’s policy for vector 0 is *Not report*, so the exception traps to TYCHE, which walks up the CDT searching for an ancestor SD with a *Deliver* policy. Skipping SD1, whose policy is *Report*, TYCHE switches to SD0—the first ancestor with a *Deliver* policy—presenting the event to SD0 as a return from a switch to SD1. SD0 handles the exception as if originating from SD1’s subtree, with access to SD1’s registers restricted by SD1’s policy, then switches back to SD1. Because SD1 is marked *Report*, TYCHE delivers the return to SD1, which sees it as a return from a switch to SD2 caused by a divide-by-zero. Before resuming SD2, SD1 may inspect or modify SD2’s register state—within the bounds defined by its policy—and decide how to proceed, for example resuming SD2 after correcting the faulting state. This routing and return protocol enables flexible interrupt delivery for isolation abstractions ranging from sandboxes to nested VMs, while providing attestable scheduling guarantees on how SDs can be preempted.

SDs’ interactions: SDs communicate via shared memory, control transfers to direct children, and interrupts. To ensure that parents remain responsible for scheduling and revoking their children, SD capabilities cannot be transferred between SDs. To enable non parent-child SDs to attest each other and exchange regions without relying on a common ancestor, TYCHE supports *channel* capabilities. Channels are derived from SD capabilities using *getchan* and appear as children in the CDT. They act as weak references to SDs and can be transferred between SDs via *send*. Channels allow non parent-child SDs to directly attest each other, exchange memory regions (e.g., to establish private shared memory), or share other channels—but not to schedule, configure, or revoke an SD.

Devices: TYCHE models devices as SDs, with configuration space, DMA, and port I/O access mediated through region capabilities. Interrupt routing follows the same protocol as above, but TYCHE allows backends to optimize core-routing using platform specific hardware mechanisms. SDs on the CPU interact with devices through shared regions for the device’s configuration space and MMIO. TYCHE provides SD0 with a channel to every device. SD0 delegates device access to an SD by carving the configuration space and duplicating a channel, sending both to the SD to enable direct SD to device interactions.

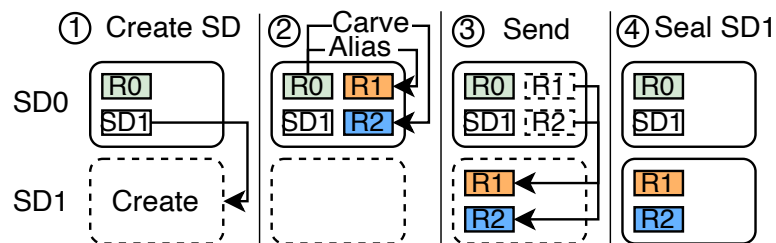


Figure 6.4 – Capability operations to create and configure SD1.

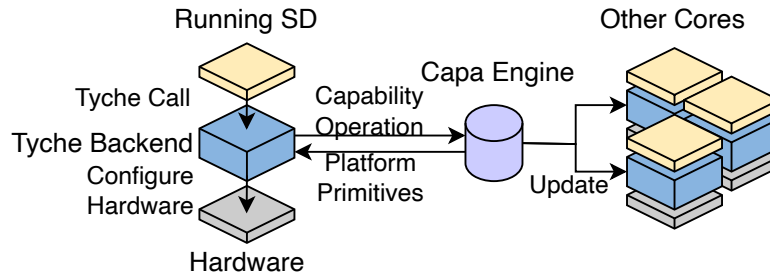


Figure 6.5 – The capability engine maintains the system state across all cores.

6.4.4 Attestation and Security Guarantees

SDs can request an attestation for themselves and any SD for which they own an SD capability (or channel). TYCHE’s attestation reports the SD’s configuration, including a hash of initial register content, policies, and a description of owned capabilities. For remote attestation, an SD’s *attest* call includes the remote verifier’s public key and nonce, as well as a public key generated by the domain itself, ensuring they are all measured and signed by TYCHE.

Conceptually, the attestation provides, for each owned capability, visibility into its direct children in their respective CDT, but not further. Listing 6.1 shows the attestation of SD1. It reports the configuration of SD1 and its child SD2, along with their owned capabilities. Allowed cores, monitor calls, and registers that can be read and written upon interrupts are encoded as bitmaps. Lineage between regions is made explicit through naming: *e.g.*, r_3 , owned by SD2, is derived from r_2 . If SD2 held a capability not derived from SD1, it would appear only in SD2’s set of owned capabilities with a fresh name and no further information. Indices are reused here for clarity; in practice, TYCHE assigns fresh names, starting from 0 based on the requester’s capabilities, to avoid revealing information about the broader system.

TYCHE’s attestation makes sharing and interactions explicit, while providing guarantees on initial SD state, run time events, and resource reclamation. TYCHE thus supports confidentiality and integrity, encapsulation, or even the privacy of communications between distrustful SDs. Confidentiality and integrity are ensured by exclusive ownership of regions and the absence of leakage through interrupts or revocation (*i.e.*, attributes). Encapsulation, as for SD2 in Listing 6.1, is achieved by ensuring a child’s regions are a subset of the parent’s exclusive ones, and that its policies prevent receiving or sending capabilities after sealing. The child can still be confidential, as is the case for SD2, and SD1 can safely share information with it via their private shared memory in r_3 without risk of leakage or trusting SD2’s implementation.

6.5 Implementation

Figure 6.5 shows the TYCHE security monitor has two components: (1) a platform-independent *capability engine* and (2) a platform-specific *backend*. The backend translates hardware events, such as interrupts or calls to TYCHE, into capability operations and forwards them to the capability engine, shared across cores. The capability engine implements the system’s global capability state machine, validates and executes operations and notifies the backend of configuration changes on affected cores via *updates*. The capability engine and backend are part of the TCB.

To construct a backend and provide the attestable enforcement of isolation by the monitor and its capability engine, the hardware must provide: (1) the ability to establish trust in the monitor; (2) the monitor’s exclusive oversight of an access control mechanism to enforce resource isolation (memory, CPU, interrupts, devices); (3) a direct, secured communication channel between SDs and the monitor.

This section describes the custom loader we use on bare metal to establish trust in the monitor, the capability engine, details the access control enforcement and communication in the x86-64 backend, and provides an overview for RISC-V.

6.5.1 TYCHE Attested Boot

TYCHE boots via a custom bootloader that enumerates CPU cores, devices, and DRAM to generate a *boot-info* manifest. It reserves memory for TYCHE per a compile-time configuration, loads the monitor (capability engine and backend), and maps the boot-info into its address space. The bootloader generates an attestation key pair, measures the monitor, boot-info, and public key into a TPM [285] PCR, and transfers control to the monitor with the private key. Using the boot-info, the backend initializes the capability engine, creates SD0 (*e.g.*, a stock LINUX kernel) with access to all unused memory and device regions, and sets up an I/O SD for each DMA-capable device with an alias to SD0’s memory. Control is then transferred on all cores to SD0.

Like prior work [191], an SD attestation includes a TPM quote and a domain attestation signed by TYCHE (Section 6.4). The quote binds the monitor binary, platform, and attestation key via PCR values, enabling trust in the monitor and transitively in SD attestations. The attestation covers the full boot chain, including our bootloader. We also prototyped dynamic root of trust (DRTM) support [8, 135] via INTEL TxT [135], reducing the TCB to just the capability engine and backend.

6.5.2 Capability Engine

The capability engine implements the region and SD capability derivation trees (CDTs from [Section 6.4](#)), validates and executes capability operations, and computes *updates* supplied to the backend to reflect configuration changes onto the hardware. It is implemented as a standalone, bare-metal (no-std) RUST library. The engine consists of 4K lines of code, uses no `unsafe` [249] to ensure memory safety, and is fuzzed as part of our continuous integration (CI) setup using LLVM's `libFuzzer` [188] to proactively detect and fix bugs.

The capability engine exposes the API defined in [Section 6.4](#) for capability operations and integrates within a backend via a *platform* interface (RUST trait). This interface is supplied by the backend to provide platform-specific primitives to manage per-SD platform state (e.g., per-core registers and access control mechanism configuration such as extended page tables (EPTs)), map and unmap physical memory ranges in an SD's platform state, manage interrupts, and implement cross-core synchronization primitives in the form of IPIs, barriers, and locks.

At all times, the engine tracks which SD executes on each core and ensures a consistent state view across cores. The engine uses the CDTs to determine which SDs and regions an operation affects and serializes operations with overlapping targets to ensure consistency. For each operation, the engine derives the set of affected cores and enqueues an update reflecting the result of the operation in per-affected-core queues, as shown on [Figure 6.5](#). The engine uses the platform interface to preempt affected cores (IPIs), requesting them to process the update and block on a barrier while the initiating core makes the new state globally visible. They are then unblocked and apply the changes to their local hardware state. This ensures the atomicity of capability and hardware state changes across cores. In practice, only a small set of updates are needed: access right changes, SD revocation, and interrupt delivery.

For example, sending a carve is handled by a core in the engine, which validates the request, computes the new capability state, and enqueues an access-right update on all cores running the sender or receiver. Using the platform IPI, it preempts these cores so they can process the update. The update involves two synchronization barriers: the first ensures all affected cores are preempted; the second waits for the new platform state. Between them, the initiating core updates the platform state – unmapping the region from the sender and mapping it into the receiver (e.g., via EPTs on x86-64). Finally, it finalizes the capability state and releases the second barrier, letting other cores apply the new state locally (e.g., TLB shutdown).

The capability engine requires memory to allocate capability metadata and SD state from. TYCHE provides this memory in two ways: statically or dynamically. In the static configuration, TYCHE reserves a contiguous block at boot and passes it to the engine. On RISC-V, this is required because the architecture provides only a small number of PMP entries, and TYCHE must protect itself and its metadata using at most one region (§ 6.5.4). To prevent memory

exhaustion from becoming a covert channel or denial-of-service vector, TYCHE can enforce per-SD capability quotas.

On platforms without this restriction, *e.g.*, using EPTs on x86-64, TYCHE supports dynamic memory management in which SDs supply memory for their children’s metadata [154, 170]. This is integrated into region capabilities semantics with a *META* attribute (§ 6.4.2). Regions transferred as *META* to a child must be exclusive, *vital*, and are *cleaned* (zeroed) on revocation. The monitor uses these regions to host, *e.g.*, a child’s EPTs. They are inaccessible to both parent and child, but are included as other regions in attestations.

Both approaches introduce trade-offs. Static reservation exposes TYCHE to denial-of-service via resource exhaustion, though isolation, attestation, switches, interrupts, and revocation always succeed (§ 6.3.2). Dynamic management avoids this limitation but enables potential cache-based side-channel attacks on metadata, which should be mitigated with allocation based on cache coloring – hence their inclusion in attestation.

6.5.3 TYCHE x86-64 Backend in Root Mode

INTEL VT-x [292] accelerates virtualization by running a host in root mode and guest VMs in non-root mode. A virtual machine control structure (VMCS) governs guest execution, virtualizing privileged operations like `cr3` writes without host intervention. Extended page tables (EPTs) [2, 292] restrict non-root memory access. Guests trap to the host via the `vmcall` instruction.

The TYCHE x86-64 backend reuses VT-x to isolate SDs, running in root mode and executing SDs in non-root mode. Each SD has its own EPT (enforcing region capability access) and per-core VMCS. Devices are also modeled as SDs, with memory isolation enforced via INTEL IOMMU [147]. TYCHE uses INIT IPs to trap cores into the monitor and implements synchronization with semaphores, atomics, and spinlocks. The backend is 6K lines of RUST, including all error codes and VMCS field enums. Note that unsafe code is required for hardware setup.

SDs interact with the monitor via the non-interposable *vmcall* instruction. Arguments and return values are passed through general-purpose registers, with capabilities referenced by SD-local indices (like UNIX file descriptors). Large values (*e.g.*, attestations) use a multi-*vmcall* protocol. We measured register-based communication to be efficient, supporting tens of thousands of large (4KB) attestations per second. Registers avoid monitor access to SD memory, reducing risks like confused deputy [123] and cache attacks. The monitor accesses memory only when no SD can, to enforce `Clean` (zeroing) or `Hash` (reading). Future work aims to offload even these to more restricted environments [77], ensuring complete isolation.

SD switches save the caller’s VMCS and load the callee’s. The backend manages general-purpose registers in software and handles other state, including MSRs, using a mix of hardware

and software to prevent leakage. EPT TLB entries are tagged with virtual processor IDs to reduce flushes on transitions. An SD runs until it returns or receives an unhandled interrupt. All control transfers — whether through `switch` or interrupts — go through the monitor, which tracks per-core state and enforces SD policies.

SD interrupt policies are offloaded to hardware via VMCS where possible. INTEL VT-x allows fine-grained traps for exceptions (vectors 0-31) but only a single bit for external interrupts. Thus, external interrupts trap to the backend which selectively decides to either re-inject them or switch to an ancestor SD per [Section 6.4](#). When supported, the x86-64 backend uses hardware APIC virtualization and IOMMU interrupt remapping to reduce traps and gain finer control. It can also expose VMCS fields as SD registers, letting parents manage interrupt delivery at the cost of limiting child policies.

6.5.4 TYCHE RISC-V Backend in M-Mode

On RISC-V, TYCHE runs in machine mode (M-mode), and the backend uses Physical Memory Protection (PMP) [\[302\]](#) to enforce region capabilities. PMP entries enforce permission on contiguous segments of physical addresses and can only be configured from M-mode. Each core has a limited number of PMP registers (up to 64), with one reserved to protect TYCHE itself. If an SD configuration cannot be satisfied with the available PMP registers, a synthetic exception is injected and delegated to a parent SD following the interrupt routing protocol. As capability revocations always succeed, the parent can recover regions sent to the child. Like x86-64's IOMMU, TYCHE requires IO-PMPs [\[96, 245\]](#) to defend against rogue DMA requests. Like x86-64, interrupts and exceptions can be either trapped or delivered directly to an SD by configuring the `mideleg` and `medeleg` registers. Communication with the monitor occurs via *ecalls* (for S-mode) or illegal instruction traps (for U-mode), as S-mode manages *ecalls* from U-mode within an SD to provide fast system calls.

Running in M-mode imposes more constraints than the x86-64 virtualization-based backend, as PMPs are only available in limited supply (8 on our board [\[272\]](#)). The SDs must carefully manage memory to maximize the use of contiguous memory, like existing RISC-V security monitors [\[35, 77, 166, 328\]](#). Alternatively, future support for H-mode could enable a virtualization-based backend.

6.6 Integrating Existing Software with TYCHE

The TYCHE SDK is a set of drivers and ports of popular software frameworks that let stock LINUX SDs run unmodified workloads as sandboxes, enclaves, or CVMs in separate SDs. These domains can be composed to secure complex cloud deployments. The TYCHE SDK is *not* part of TYCHE's TCB.

6.6.1 Interfacing with TYCHE from LINUX Environments

We provide *Tyche-Capa*, a kernel driver on x86-64 and RISC-V that lets LINUX environments in an SD interact with the monitor and create new SDs. TYCHE-Capa abstracts capabilities and exposes a simple interface — *ioctl* commands, file descriptors, and *mmap* — to create, manage, and run SDs. It allocates memory from kernel pools, reserves pages (as in ballooning [301]), and ensures revocation to avoid leaks after crashes. To reduce fragmentation and work around RISC-V's limited PMP entries, it can reserve contiguous physical memory at boot via kernel parameters. TYCHE-Capa schedules SDs as part of their host process, like VMs under KVM [178], and allows them to coexist with OS isolation tools—cgroups, namespaces, and syscall filtering—at the process boundary.

6.6.2 Backward Compatibility

The TYCHE SDK provides backward-compatibility with popular virtual machine monitors (VMMs) and enclave frameworks. It relies on the TYCHE-Capa driver as a compatibility layer to run VMs, CVMs, sandboxes, and enclaves as SDs.

VMs and CVMs with KVM on INTEL x86-64: *KVM-Tyche* is a fork of the KVM-INTEL [178] driver, modified to run VMs and CVMs as child SDs (*not* as nested VMs). The porting effort involved modifying just 400 LOC out of 14.6k LOC, replacing INTEL VT-x instructions and EPT management with TYCHE-Capa calls. With this patch, TYCHE can support popular KVM-based VMMs and container frameworks [10, 45, 187, 316].

KVM-TYCHE maintains the runtime behavior of VMs compared to KVM-INTEL and thus uses TYCHE's ability to delegate APIC virtualization to a parent SD. For CVMs, additional logic was added to account for the VM's memory and state being unavailable to the host. KVM-TYCHE is backward-compatible with existing VMMs for non-confidential VMs. A small 20-line patch to the LKVM [187] VMM enables confidential VM support by requesting exclusive memory from TYCHE-Capa.

Sandboxes and enclaves on x86 with GRAMINE:

GRAMINE [114], formerly GRAPHENE [290], is a library OS (libOS) for running applications inside INTEL SGX [136] enclaves; it shields applications by mediating system calls to the untrusted OS. In most cases, GRAMINE supports unmodified binaries. *Gramine-Tyche* is a fork of GRAMINE SGX platform abstraction layer that runs enclaves and sandboxes as SDs in userspace, isolating programs in exclusive or shared memory without INTEL SGX hardware. We modified 300 lines of code (LOC) out of 14.9k LOC, replacing INTEL SGX logic with TYCHE-Capa calls. The SDK also populates the SD's page tables: it allocates the corresponding physical pages, carves and sends them to the child SD, and initializes its page table root

register (`cr3`). Once the enclave SD is sealed, its policy prevents the parent from reading or writing `cr3`, ensuring the parent cannot tamper with the child's address translation.

GRAMINE-TYCHE does not yet support `fork`. This is a limitation of our port, as doing so requires non-trivial cross-enclave synchronization and state transfer, not an intrinsic constraint of TYCHE. For sandboxing, SDs complement OS-based mechanisms by providing efficient, hardware-enforced, and OS-independent isolation with a reduced TCB. They enable attestation not only of the sandbox boundaries but also of the trusted code implementing higher-level policies (*e.g.*, syscall filtering). Future SDK support could enable sandboxing within enclaves, which is beyond the capabilities of standard OS isolation.

Enclaves on RISC-V with KEYSTONE: KEYSTONE [166] is a RISC-V TEE framework, consisting of a LINUX kernel driver and the Eyrie enclave runtime. Porting KEYSTONE to TYCHE required 20 LOC changes to the runtime for compatibility with TYCHE's API, and 150 LOC to the KEYSTONE driver to interact with TYCHE-Capa for creating and managing SDs.

6.6.3 Preserving Runtime Behavior

TYCHE SDK preserves existing abstraction semantics, allowing fair performance comparisons (Section 6.7). Future work could better exploit TYCHE's features—for example, running GRAMINE's libOS as privileged software to reduce enclave exits [138, 161], or using SDs in a TYCHE-aware VMM for efficient I/O and delegated interrupt handling. Timer interrupts, currently routed to the hypervisor, could instead follow SD policies to avoid VM exits.

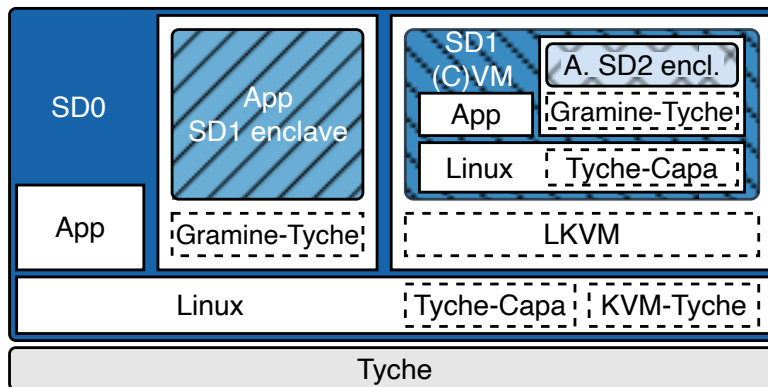


Figure 6.6 – Superposed system and SD views of the deployments benchmarked in Figure 6.10. SDs are blue rounded boxes. System abstractions (processes, kernels, and VMs) are full rectangles, and libraries and drivers dotted ones.

6.7 Evaluation

This section evaluates TYCHE’s design and performance on x86-64. It reports microbenchmarks for the full-fledged x86-64 implementation and compares it with the prototype on RISC-V (Section 6.7.1). It then focuses on x86-64 to measure the performance of real-world applications isolated with various threat models (Section 6.7.2), using native execution as a baseline and including INTEL SGX enclaves and native VMs as reference points. Case studies (Section 6.7.3) then detail the confidential Llama CPU-based inference for mutually distrustful model and prompt owners and other isolation deployments enabled by TYCHE.

Experimental setup: On x86-64, TYCHE runs on a 16-core INTEL i7-10700 CPU with INTEL SGX [136] v1 and an EPC of 94MB, using a stock LINUX kernel v6.2 as SD0. On RISC-V, it runs on a STARFIVE VisionFive2 board [272] (JH7110 SoC) with 8 PMP entries per hart and a stock v5.15 kernel. VMs and CVMs use v6.2 with 4 GiB, 8 cores, and VIRTIO devices [179]. CVMs also enable SWIOTLB [180] to copy I/O into SD0-shared memory. Network experiments use a separate 12-core client, *wrk* [109] or *memtier* [162], with 400 connections over Ethernet. Results average the best 9 of 10 runs.

Naming conventions: We refer to bare-metal LINUX as “native” and KVM-based LINUX VMs as “native VMs”. SD0 runs LINUX and creates SD1 VMs, CVMs, and GRAMINE-based enclaves. SD2 enclaves are nested inside SD1 CVMs and isolated from both SD0 and their parent. SGX enclaves run with GRAMINE-SGX. Sandbox SDs perform identically to enclave SDs and are omitted. See Figure 6.6.

6.7.1 Microbenchmarks

We measure the average latency (over 1000 ops) of TYCHE’s primitives on both platforms to compare hardware mechanisms. We then evaluate CPU and I/O overheads on x86-64 across SD configurations, compared to equivalent native deployment.

Creation/Revocation: Figure 6.7 shows carves are more expensive than aliases as they remove memory from SD0 and notify other cores via IPIs (Section 6.5.2). This difference is especially noticeable on x86-64, where carves trigger a walk on SD0’s EPT. On both platforms, *hash* and *clean* increase latencies with the SD’s size, as they require reading and writing memory, respectively.

Switches: In Table 6.3, SD switches on RISC-V are 3x slower than on x86-64, despite faster privilege layer transitions to/from TYCHE. On x86-64, TYCHE uses a hardware-software combination to efficiently save and restore SD states, while RISC-V relies entirely on software. Overall, TYCHE’s switch latencies on x86-64 (1.2 μ s) and RISC-V (3.9 μ s) are competitive with related work and hardware extensions [12, 136, 138, 166, 290].

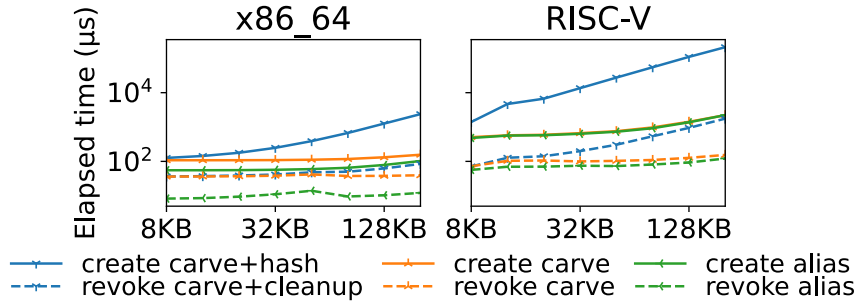


Figure 6.7 – Create and revoke latencies as a function of size for carve, with and without hash and clean, and alias SD memory.

Table 6.3 – Average switch latency and standard deviation, including monitor entry+exit (hardware privilege transitions).

	Enter+Exit TYCHE	Total switch cost
x86-64	$0.493 \pm 0.017 \mu\text{s}$	$1.171 \pm 0.002 \mu\text{s}$
RISC-V	$0.246 \pm 0.000 \mu\text{s}$	$3.897 \pm 0.007 \mu\text{s}$

RISC-V prototype: In Figure 6.8, TYCHE on RISC-V competes with native and unmodified KEYSTONE on most RV8 microbenchmarks, with a $< 10\%$ slowdown for SD1 enclaves on short-lived programs (qsort, norx) due to the extra indirection to the TYCHE-Capa driver.

CPU overheads on x86-64: Figure 6.9 shows CoreMark-PRO results from 1 to 16 cores. SD0 matches bare-metal, with minor slowdowns (up to $\sim 2\%$) at 2-4 cores due to virtualization amplifying cache and TLB effects. SD1 VMs and CVMs show slight overhead: $\sim 1\%$ over native VMs and $\sim 4\%$ over native, mainly from timer interrupt transitions. CVMs add no extra cost over VMs.

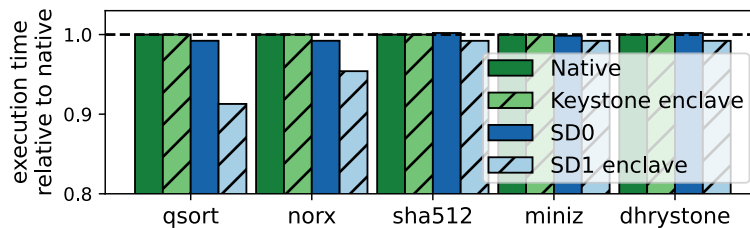


Figure 6.8 – RV8 RISC-V CPU microbenchmarks comparison between native, unmodified Keystone, and Tyche.

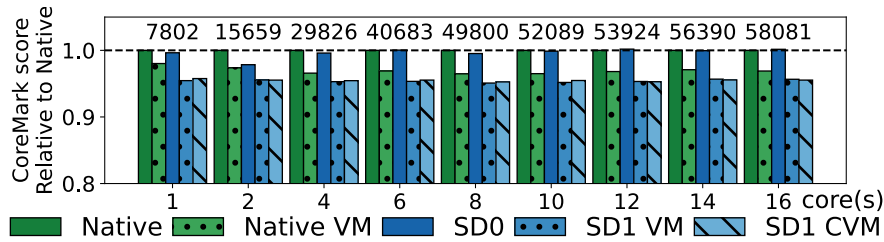


Figure 6.9 – CoreMark-Pro performance relative to bare-metal Linux (Native) for different SD deployments and varying number of cores on x86-64 (raw native score at the top).

Multi-(C)VMs: We group the 16 cores into 2-, 4-, or 8-core sets, launching one LINUX (C)VM per group (e.g., 8 VMs for 2-core groups). Each VM boots a full LINUX kernel, runs two-minutes *sysbench* [158] CPU and memory benchmarks, saves the results, and shuts down:

```
sysbench cpu --threads=$(nproc) --time=120 run
sysbench memory --time=120 --threads=$(nproc) \
  --memory-block-size=128M --memory-access-mode=rnd \
  --memory-oper=write run
```

CPU throughput (events/sec) and memory throughput (random writes over 128 MiB) show only 2% CPU and < 1% memory overhead for SDs versus native VMs. *kvm_stat* [181] shows similar event distributions; the extra cost is due to monitor indirections on timer interrupts and synchronized access to SD0 state. Running two SD (C)VMs per group yields similar overheads compared to native VMs, showing they remain stable under oversubscription.

I/O overheads on x86-64: We measure REDIS I/O using memtier [162] with pipeline=1 GETs to avoid masking latency and to expose TYCHE’s overhead. Table 6.4 details the full configuration of the networking benchmarks. Figure 6.10 and Figure 6.11 show throughput and latency across SD configurations. Most native and SD (C)VM overhead comes from the LKVM user-space VIRTIO driver; running native and SD VMs with QEMU (no VHOST) improves throughput by 15%-points, confirming this bottleneck. With QEMU, SD VMs still

Table 6.4 – Network benchmarks configuration.

Server	Benchmark client	Threads	Connections	Workload configuration	Pipeline	Termination condition
redis	memtier	12	400	10% write, 90% read	1	20 million requests
hyper	wrk	12	400	12B payload 76B headers	1	2 minutes
lighttpd	wrk	12	400	10 kiB payload	1	2 minutes

achieve similar performance to native ones, even with VHOST enabled. This is expected as better drivers (QEMU) and VHOST improve the performance on the common path for both native and SD (C)VMs and are orthogonal to TYCHE. Enclave overheads stem from libOS copies: SD2 in particular suffers from compounded I/O paths—VIRTIO, SWIOTLB, and enclave copying. The lack of pipelining degrades the throughput compared to the baseline due to the higher latency introduced by the different levels of isolation, as shown in Figure 6.11. In particular, the relative performance of native and SD VMs compared to native or SD0 in Figure 6.10 is due to the increased latency: from 0.86ms when running natively to 1.77ms in a VM at p50 on the REDIS benchmark. Enabling pipelining hides the latency; we measure a baseline 4.16 millions req/s with pipeline = 30, and 3.54 millions in the native and SD VMs, *i.e.*, a 15% overhead rather than 40%. With lighttpd [155], we observe that increasing payload size amortizes copy and context switching overhead, reaching native and SD0 throughput for 10 KiB payloads.

6.7.2 Enclaves, VMs, CVMs, and Composable Isolation

Figure 6.10 shows TYCHE’s performance isolating real-world applications on x86-64. We use native execution as the baseline and include native VMs and SGX enclaves as reference points.

Applications: We evaluate Lighttpd [155], Hyper [134], SQLITE [127], REDIS [234], and llama-cpp [106]. Lighttpd and SQLITE are common enclave workloads [12, 290] and appear in nested enclave studies such as Veil [12], enabling direct comparison. Hyper is a high-performance, multi-threaded RUST HTTP server that scales with worker threads rather than

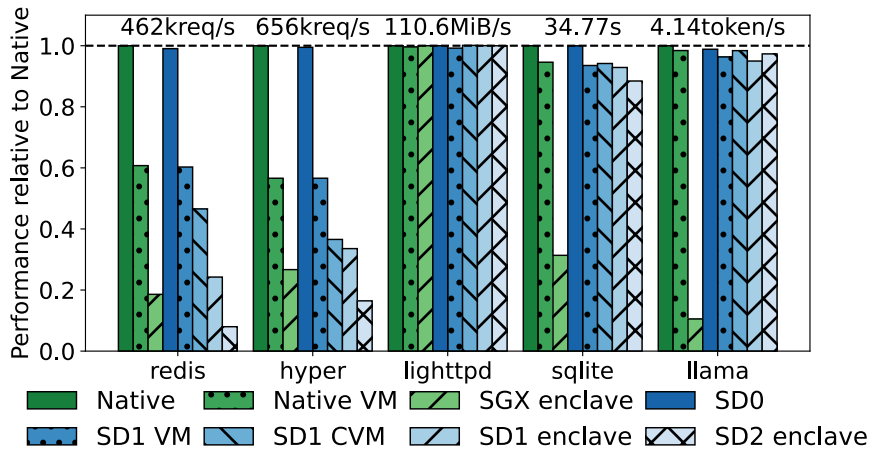


Figure 6.10 – Performance relative to native execution (green) for real-world applications isolated as SDs on x86-64 (blue).

processes [276], avoiding GRAMINE-TYCHE’s unsupported fork. HTTP servers are benchmarked with wrk [109], REDIS with memtier, SQLITE with speedtest1 [127] at size 1000 (which includes 500k inserts), and Llama reports throughput [106] after generating 1000 tokens.

Configuration: Binaries are from GRAMINE’s default examples except for Llama (Section 6.7.3). GRAMINE applications use a *manifest* specifying the number of threads and enclave memory. We use default manifests [115] with *exitless* [116] disabled due to CVE-2022-21233 and CVE-2022-21166. All enclaves include at least one worker and three GRAMINE runtime threads [117] and 256MiB to 4GiB total memory. For TYCHE, we supply one physical core per manifest thread and configure native execution with the same worker-thread count. GRAMINE-SGX and GRAMINE-TYCHE use the same application binaries, which we compiled from source without modification.

SD0: SD0 shows no overhead in any benchmark compared to native, indicating that TYCHE does not impact the performance of LINUX applications. This result is expected, as there are no calls or exits to the monitor and SD0 has direct access to devices and the APIC.

VMs and CVMs: SD1 VMs match the performance of native VMs, demonstrating that KVM-TYCHE can support existing VM deployments. For I/O benchmarks (e.g., Hyper and REDIS), SD1 CVMs experience additional overheads due to extra copies through bounce buffers for I/O (see Section 6.7.1). They otherwise perform similarly to native VMs.

Enclaves: SD1 enclaves outperform SGX enclaves across all applications and are close to native for non-latency sensitive applications. SGX performs poorly on Llama, even compared to an SD2 enclave. This is likely due to the limited EPC (94MB) size in SGX v1 compared to the high memory usage (4GiB) for the model [192], llama-model and llama-cpp contexts [106] and possibly memory encryption and integrity protection, thus we would expect SGX v2 to perform better. Regardless, SD2 only incurs a reasonable overhead compared to our baseline native execution as long as extra memory copies can be amortized.

Memory usage: TYCHE requires private memory to store capabilities. Running these workloads, we measured an average of two 4KiB pages per SD used for storing capabilities. This extrapolates to, e.g., a 4MiB memory overhead for 512 SDs.

Discussion: The latency of network operations is the primary issue but not a direct consequence of TYCHE’s design, as discussed in Section 6.7.1. This is a known problem with confidential environments [96, 167, 171, 218] that could be alleviated in TYCHE, e.g., with safe device passthrough. TYCHE otherwise demonstrates that existing frameworks can be ported to run SDs, achieving performance comparable to equivalent native deployments and outperforming INTEL SGX, even in nested cases, reinforcing our claim of backward compatibility.

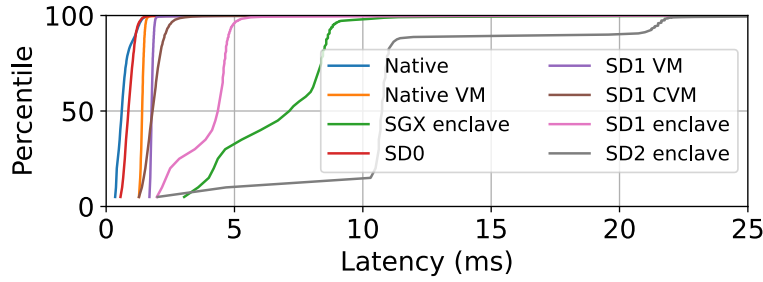


Figure 6.11 – Redis GET latency distribution as measured by memtier (max throughput) during Figure 6.10’s experiment.

6.7.3 Case Studies

Private inference with controlled sharing: This case study uses Figure 6.2’s scenario to protect user prompts from the CSP and model owner, while keeping model weights secure from the user and CSP. The CSP runs a LINUX SD0 hypervisor with KVM-TYCHE; the user runs an SD1 CVM that creates an SD2 enclave for CPU-based LLM inference using llama-cpp [106] with GRAMINE-TYCHE and four threads. The “proprietary model” is an encrypted Llama 3.2 Instruct model [192], decrypted inside the enclave as it is read from disk. The enclave has no file or network access and shares memory only with the user’s SD1 CVM, which receives prompts over SSH and forwards them to the enclave. User keys can also be stored in a separate enclave. As shown in Figure 6.10, the SD2 enclave runs near native speed ($\sim 2\%$ overhead), is $10 \times$ faster than SGX enclaves, and goes beyond prior industrial solutions [18, 86] by also protecting the model from the user. Future work on KVM-TYCHE may explore secure GPU passthrough.

Managing trust within user-applications: TYCHE’s SDs are orthogonal to privilege levels and can compose isolation even within user-level code. We prototyped compartmentalization of code within a user SD enclave. While we did not provide a full port of an intra-process isolation framework, such a port could be done in the future; this prototype demonstrates the concept and fine-grained control over application TCB.

In this experiment, a user enclave running bearSSL [229] hosts REDIS inside a nested child SD, implemented either as a sandbox or a nested enclave. Both SDs minimize their TCB by including only a modified musl libc [94], removing unnecessary system calls, and communicating through shared-memory queues. Before decrypting and forwarding requests to REDIS, the SSL enclave attests that it fully encapsulates the REDIS SD. Responses are returned via the shared queue and encrypted before being sent back.

A remote client (mentier) issues requests over a TLS connection with the SSL enclave. This experiment illustrates TYCHE's generality, showing that arbitrary composition and nesting of isolation boundaries is possible, even entirely within userspace.

Sharing without the cost: In existing cloud abstractions, communication between trusted TEEs—enclaves or CVMs—must bounce through untrusted memory, incurring software encryption, replay protection, integrity checks, and data copies to defend against tampering by untrusted software. TYCHE instead enables SDs to establish attestably private shared memory, accessible only to the two endpoints, eliminating this entire overhead. This simplifies communication, improves performance, and enables confidential service VMs—trusted VMs that expose devices or services directly to other confidential endpoints.

We compare direct SD-to-SD communication with the traditional integrity-protect, encrypt, bounce, decrypt path. Using AES-GCM and SHA-256 from the standard LINUX crypto library, we measure added latencies of 20 μ s for 1KB messages, 2ms for 1MB, and 0.24s for 100MB. With TYCHE, these costs are avoided entirely. Physical attacks are orthogonal and can be handled in hardware, *e.g.*, with single- or multi-key total memory encryption (MK-TME) [137], as discussed in Section 6.3.2.

6.8 Related Work

TYCHE combines a security monitor design with an API inspired by micro-kernels to deliver a unified isolation solution.

Security monitors: They are trusted intermediaries [254] that enhance systems with new isolation and security guarantees [69, 98, 320] and are easier to inspect, update, and extend than hardware solutions [41]. ARM CCA uses a monitor to support confidential VMs [175] and KOMODO [98] enables enclaves in ARM TRUSTZONE [22]. Security monitors are popular in confidential computing to restrict privileged software, often through the use of virtualization, *e.g.*, Inktag [130], Overshadow [69], and HyperEnclave [150] protect enclaves from untrusted OSes, Cloudvisor [320] isolates VMs from hypervisors, and Blackbox [128] secures containers. They can offer intra-VM isolation, *e.g.*, Veil [12] implements enclaves in AMD CVMs [5] while Erebor [319] focuses on sandboxes in INTEL CVMs [138], or can harden kernel integrity [34, 80, 264]. TYCHE adopts a similar implementation but differs in the abstraction it exposes. It provides a unified isolation abstraction, SDs, that subsumes and composes the isolation boundaries traditionally enforced by separate security monitors. This design is inspired by micro-kernels.

Micro-kernels: Micro-kernels [154, 176, 278] follow a minimalist design, including only essential functions that cannot be implemented in user space, such as virtual memory, IPC, and thread management. This reduces kernel complexity, improving security and maintain-

ability. By separating mechanisms from policies [163], micro-kernels provide flexibility in resource management and isolation. For instance, while the kernel handles page tables, user-space components configure virtual address spaces, enabling the *recursive construction of address spaces* [176] and custom isolation abstractions.

TYCHE leverages key micro-kernel concepts to deliver modern isolation guarantees. Its memory operations resemble L4 [176]’s *grant*, *map*, and *flush*. They however differ in that TYCHE’s operations focus on *attestable* isolation through guarantees on resource management: distinguishing explicitly between shared and exclusive resources and attesting memory is measured when received or scrubbed upon revocation (Section 6.4). TYCHE also draws inspiration from Fluke [103], which enables recursive VM isolation through “nested processes” without the overhead of naive nested virtualization. TYCHE generalizes this approach by providing a foundational mechanism to unify compartmentalization and compose confidential computing and encapsulation.

Despite its similarities to micro-kernel APIs, TYCHE is *not* a kernel replacement, does not create directly usable system abstractions such as processes, and requires an untrusted kernel in SD0 to drive the machine. The principles behind TYCHE’s attestable isolation could however be backported into existing security-oriented micro-kernels [154].

Virtualization: TYCHE is not a hypervisor even though it runs in root-mode and uses virtualization extensions on x86-64. It does *not* virtualize resources, provide full machine abstraction, or take allocation or scheduling decisions. Instead, TYCHE is closer to an exokernel [90] or a trusted state machine: it validates operations against policies encoded by capabilities to correctly enforce and attest the partitioning and sharing of resources across SDs. TYCHE shares some similarities with hypervisors that adopt micro-kernel principles to improve security [173, 274], but these efforts focus on minimizing the hypervisor’s TCB rather than providing a unified isolation abstraction to compose security boundaries. Unlike type-1 hypervisors [37, 296], which grant extra privileges to their initial domain (e.g., `DOM0` in Xen [37] or the root partition in Hyper-V [296]), SD0 in TYCHE has no special privileges.

Other mechanisms and platforms: Our RISC-V prototype in M-mode demonstrates that TYCHE does not require virtualization and can adapt to simpler access control mechanisms [15, 152, 247, 310]. TYCHE could use alternative architectures [247], like NoHype [152], that eliminate virtualization and provide simpler primitives to partition I/O, memory, and cores. CHERI [310] capabilities offer a promising alternative to page-based mechanisms, already supporting enclaves [275] and secure embedded devices [15]. TYCHE’s software-defined capabilities provide a global view of system resources with extensible policies, complementing CHERI’s efficient hardware enforcement for fine-grained memory isolation.

While we did not port TYCHE to ARM, a virtualization-based backend could be implemented, running the monitor in EL2, similar to Blackbox [128], in normal, secure [175], or realm

world [175]. Alternatively, it could run as firmware in EL3, akin to the RISC-V backend, and leverage Granule Protection Table [175] to isolate memory. Similarly, TYCHE could run as an INTEL TDX [138] module, providing attestable policies for resource management and private shared memory.

6.9 Conclusion

TYCHE shows that trust management can become a first-class cloud abstraction. Its security domain abstraction unifies composable isolation across privilege layers and system abstractions, shifting the burden of implementing and enforcing fine-grained isolation away from tenants. We showed that TYCHE is both general, by supporting and composing enclaves, sandboxes, and confidential virtual machines, and practical by running unmodified applications with near-native performance. A confidential-inference case study with mutually distrustful users, model owners, and CSPs demonstrated its applicability to modern cloud workloads with complex trust models.

Part III Future Work and Closing Remarks

In [Part II](#) we discussed how to design and integrate security policies with a system's attestation. In particular, we discussed how to block all same-core attacks across security domains—including the majority of transient-execution attacks—as well as how to design and implement a capability-based security monitor. As a result, the techniques presented enable system designers to design elaborate security policies for TEEs in such a way that a challenger can fully audit from the attestation. In combination with [Part I](#), where we discussed how to remove untrusted code from firmware mode to keep only trusted system software within the TCB, this ensures that it is possible to reason about a system's correctness without relying on proprietary or untrusted code.

[Figure III.1](#) revisits the ARM CCA deployment we used as an example when discussing attestation dilution in [§ 2.2.3](#), with the system software changes proposed in the thesis (assuming they were all ported to ARM). The most obvious change from the figure is that all system software can now be audited by the confidential VM tenant. The tenant no longer needs to trust the cloud provider firmware for the system's correctness: it can verify that the AP_BL31 measurement corresponds to the hash of a source-available virtual firmware monitor. The other changes concern isolation policies. If core gapping is enabled, the tenant can verify—from the attestation—that physical CPU cores are never shared with untrusted code, including the host hypervisor and other tenants' VMs, and thus that no same-core attacks are possible. Finally, if the RMM is extended with a capability system similar to TYCHE, the tenant can further partition its confidential VM to enforce tailored policies, such as leveraging nested enclaves.

In the last part of this thesis we briefly discuss some of the limitations of our work, and propose future research directions to address them and push the idea of undiluted attestation further. Finally, we conclude the thesis.

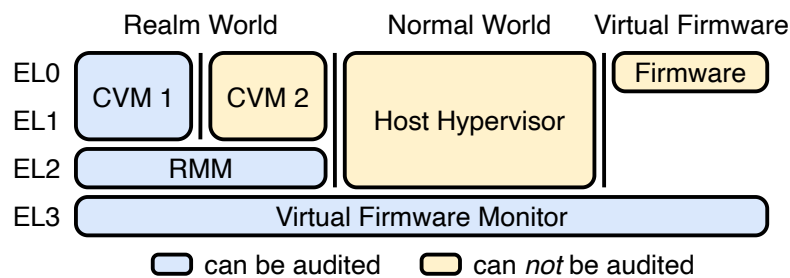


Figure III.1 – Example of Arm CCA deployment incorporating the ideas presented in this thesis, from the perspective of the tenant of CVM 1.

7 Future Research Directions

In this thesis we investigated how to build systems with undiluted attestation based on two assumptions: correctness of auditable software, and correctness of the hardware. To achieve truly undiluted attestation, we would need to study how to further weaken those assumptions. In particular, there seem to be two broad future research directions that would bear significant security benefits. The first would be to better partition software in order to reduce the size of the TCB and simplify security audits. This is especially true at the level of the operating system, where thousands of executables and libraries execute with very coarse resource control (e.g. user profiles), leading to hundreds of millions of trusted lines of code. The second would be to also verify hardware correctness, meaning the correct configuration and operation of all computing elements on the system.

7.1 Enforcing Isolation Policies at the OS Boundary

In most systems, the OS itself accounts for the biggest chunk of the TCB. As modern systems often run multiple full-blown OSes — as a host hypervisor, as a VM, or confidential VMs — the risk of software vulnerabilities or supply chain attacks at the OS level is a real concern. The XZ backdoor [306], for instance, is the result of three years of effort by an advanced threat actor, which resulted in the compromise of the ubiquitous and highly regarded XZ compression library. The backdoor *almost* reached stable LINUX distributions, and would have infected the world's entire computing infrastructure if it hadn't been for a MICROSOFT engineer who raised the alarm at the last minute.

There have been significant research efforts to develop techniques to contain or limit the impacts of such vulnerabilities. Software-based isolation [108, 314, 315] is now a mature technique and has graduated to a standard through WEBASSEMBLY [121]; system call filtering is widely available on most OSes [182, 303]; and enclaves can be used to protect sensitive processes. Yet those mechanisms are seldom used beyond a very small set of software, namely web browsers and user-space VMMs [10].

It seems the reason why we do not have fine-grained sandboxing and isolation at the OS level is no longer technical, it is a problem of adoption. With the current status quo, each program or library needs to bring its own sandboxing mechanism, and needs to do so in a way that does not break backward-compatibility with software already installed on the machine. There

seems to be no possible progressive migration toward mainstream fine-grained sandboxing at scale.

7.1.1 Dynamic Linkers Are the Narrow Waist of Operating Systems

As part of implementing the different systems described in this thesis, we ended up writing a wide variety of program loaders, in order to load a program in an enclave, a sandbox, a kernel in a VM, and so on. Program loaders are a convenient place to enforce security policies: they have access to the program to load and its meta-data. For instance, we built a tool to load enclaves as user programs on a TYCHE-enabled system. The tool takes a binary and a *manifest* describing the characteristics of the enclave — private and shared memory areas — and loads the program accordingly. This is, in fact, not a new idea [131]. It is common for some OSes to require a manifest describing security policies for an application, as is done on smartphone OSes for instance. Surprisingly, however, such a mechanism has never come to mainstream server-class OSes.

From our experience, we believe that program loaders — also called dynamic linkers — are the key enabler for deploying fine-grained isolation policies at scale. On UBUNTU, for instance, more than 99% of binaries are dynamically linked [291]. This means that the dynamic linker, *e.g.*, `ld.so`, executes before almost any program on the system, and is therefore a perfect entry point to enforce security policies.

Consider the problem of applying `seccomp` system call filters to all programs on an OS. There already exist techniques to compute the set of system calls required by an executable or library and build an appropriate `seccomp` filter [56]; the problem is to make deployment practical. We suggest that the dynamic linker is the right component for that. On LINUX, executables and libraries are encoded as ELF files, which can easily be extended with arbitrary metadata through custom sections without breaking backward-compatibility. Dynamic linkers can be extended with a `seccomp` module to recognize those additional sections, and, when all libraries in a program are compatible, to enforce the system call filtering. Figure 7.1 provides an overview of the process. As a result, dynamic linkers and `seccomp`-compatible libraries can be deployed progressively without breaking change, while eventually enabling system call filtering across all programs on the OS.

We believe that dynamic linkers are a key component for strengthening OS-level security. As a first step to enable exploring and deploying new dynamic linker security mechanisms, we started investigating modular dynamic linker designs, as described in [59]. Our design simplifies the process of extending dynamic linkers and is intended to serve as a demonstrator for various isolation policies. We believe that dynamic linkers will offer significant leverage to systems designers, and would be a fruitful avenue for further research efforts.

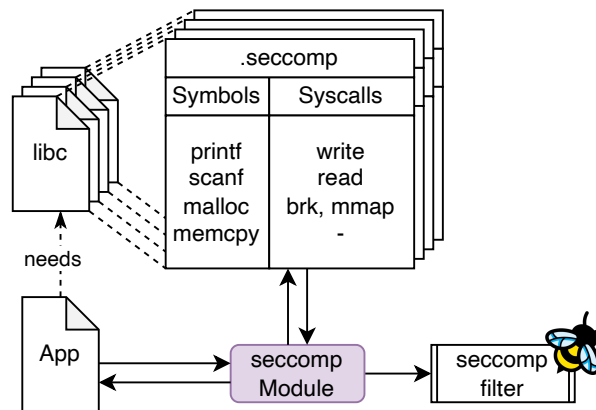


Figure 7.1 – Scalable seccomp sandboxing. A seccomp module in the dynamic linker aggregates the authorized system calls based on symbol resolution and a per-file .seccomp section.

7.2 Attestation Beyond the Application CPU

In the thesis we have been focusing on application CPU attestation, yet the CPU is not the only computing component in a computer system. Therefore, the correctness of a system does not only depend on the software running on the application CPU, but also on the software running on all other components. As a result, we believe that an important future direction for the work presented in this thesis is to extend the attestation mechanism to cover all software on the system, and to ensure attestation dilution remains as low as possible.

7.2.1 Full SoC Attestation

When discussing the ARM CCA attestation in the background chapter (§2.2.3), we mentioned two other micro-controllers whose software is part of the attestation: the Application Processor and Runtime Security Engine. Although those are ARM-specific names, most CPU vendors bundle micro-controllers with similar roles within the SoC⁸.

To give a concrete example, let us consider the EIC7700X SoC used in the Premier P550 board, one of the boards supported by MIRALIS. Figure 7.2 corresponds to the block diagram of the EIC7700X. In addition to the four RISC-V RV64GC cores (on which MIRALIS executes at the highest privilege level), we can see a RISC-V RV32 core in the “Power & Security” island, as well as an NPU and a Crypto engine. All those computing elements share a common set of resources: memory through LPDDR5, peripherals such as SPI, eMMC, and SATA. There are also security mechanisms: a system MMU, M-PMP, and I/O PMP. Understanding

⁸System on Chip

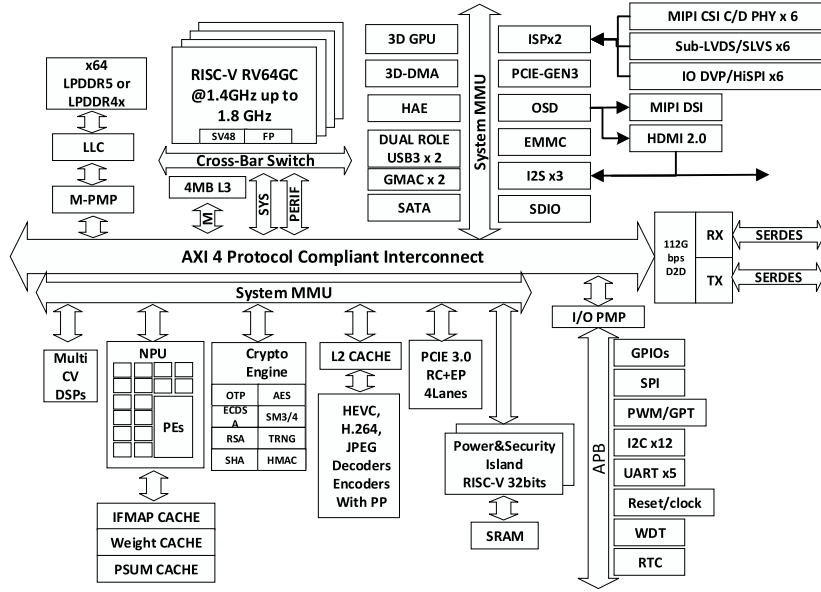


Figure 7.2 – EIC7700X SoC block diagram. The application CPU is composed of the four RISC-V RV64GC cores.

all possible interactions between those components—configuring security mechanisms appropriately—is a challenge that needs to be solved to ensure system correctness extends beyond the application CPU to the whole SoC.

There are two interesting system challenges at the SoC-level, both being addressed by different research directions. The first is to characterize the possible interactions between each component, and formally verify higher-level security guarantees. Initial results in characterizing and formalizing a production-grade SoC [100, 102] are encouraging, and it seems like the approach could be scaled to automatically analyze the security properties of a system based on its configuration. The second direction is to safely de-privilege firmware executing on co-processors on the SoC, similar to how MIRALIS de-privileges firmware on the application CPU. The emergence of memory isolation features on low-power micro-controllers, such as ARM Cortex-M and small RISC-V cores with support for PMP registers, has led to the design of safety-oriented microcontroller OSe [170, 222]. Similar to MIRALIS, such systems enable open-source software to safely sandbox proprietary logic, as can be done with OPEN-TITAN [215] for instance, and will be key to support undiluted SoC-level attestation.

We believe that combining those research directions with the results of this thesis would enable the verification of SoC-level systems, and push the idea of undiluted attestation even further.

7.2.2 Rack-Scale Attestation

Beyond the SoC, the same pattern repeats itself: multiple devices need to cooperate and coordinate accesses over shared resources. The BMC, disks, NICs, and GPUs each run their own firmware and govern access control over some resources. And the pattern repeats again when assembling machines as racks or warehouse-scale computers.

The challenge for the next frontier of computer systems will be to assemble and reason about a rack-scale attestation. There are early signs of progress toward that direction, such as the Caliptra project [71], which aims at providing each device with its own hardware root of trust, but the problem of verifying the correctness and security properties of a rack-scale system remains. For instance, consider an AI lab deploying its model on rented infrastructure shared with other tenants. How can the AI lab get the best possible guarantee that there exists no possible way for another tenant to observe the model weights?

Solving this challenge will require scaling attestation to all devices and computing components, and developing tools to automatically reason about a rack-scale attestation to verify security properties. Such work will need to encompass all levels of the system stack, from bus fabrics to software and network analysis. There remains much to do beyond the CPU to achieve truly undiluted attestation, but we believe such a future to be possible.

8 Conclusion

In this thesis we investigated how to design systems software in such a way that a system’s security properties can be logically deduced from the inspection of its attestation. We identified the main causes of attestation dilution — *i.e.*, which prevents verifying some aspects of the system’s correctness — and proposed software-only changes that address those issues.

In [Part I](#) we introduced the idea of virtual firmware monitors, which enable secure and efficient sandboxing of unmodified vendor firmware. By removing vendor firmware from the highest-privilege mode, virtual firmware monitors ensure that the software executing with the most privileges can be audited and trusted. We discussed the design and implementation of MIRALIS, a RISC-V virtual firmware monitor, and demonstrated that virtual firmware monitors can be deployed on existing hardware and virtualize unmodified firmware. We further proposed a new technique to enable push-button verification of firmware virtualization, leveraging official ISA specifications, and verified the implementation of MIRALIS.

In [Part II](#) we focused on isolation policies for trusted execution environments, and their interaction with a system’s attestation. In particular, we discussed two problems: how to block side-channel and transient-execution attacks, and how to design and integrate a flexible TEE isolation mechanism with the system’s attestation.

We remarked that most side-channel and transient-execution attacks rely on an attacker and a victim being co-scheduled on the same physical core. We proposed a technique called *core gapping* to provably enforce that two distinct security domains are never co-scheduled on the same core, and therefore block the majority of such attacks. We implemented core gapping for ARM CCA, and discussed how to integrate it with existing software with minimal software changes, while demonstrating that the technique incurs only a negligible overhead — and improves performance for some workloads.

Finally, we discussed the problem of designing primitives to support most of the TEE use-cases through a unified mechanism, rather than relying on a collection of ad-hoc abstractions. We introduced the idea of capability-based security monitors, and discussed how to integrate capabilities within the system’s attestation. We presented the implementation of TYCHE, a capability-based security monitor for x86-64 and RISC-V. We discussed how to integrate capability-based isolation with existing software, and demonstrated the flexibility of our approach with use-cases such as nested TEEs and shared confidential memory.

Brought together, the techniques presented enable removing firmware from the trusted computing base, protecting confidential VMs from transient-execution attacks, and fine-grained and composable TEE isolation. As a result, this thesis enables designing systems whose correctness can be deduced from their attestation, with only minimal assumptions.

A Appendix

AA ARM CCA Attestation

```
1 {
2   "cca-platform-token": {
3     "cca-platform-profile": "tag:arm.com,2023:cca_platform#1.0.0",
4     "cca-platform-challenge": "DSLgiphGkFhIYxgoNIm9s28J2+/rGGTfQz+m5U6i1xE=",
5     "cca-platform-implementation-id": "f0VMRgIBAQAAAAAAAAAAAAAMAPgABAAAAUFGAAAAAAAA=",
6     "cca-platform-instance-id": "AQcGBQQDAGeADw4NDAsKCQgXFhUUExIREB8eHRwbGhkY",
7     "cca-platform-config": "z8/Pzw==",
8     "cca-platform-lifecycle": 12291,
9     "cca-platform-sw-components": [
10      {
11        "measurement-type": "AP_BL1",
12        "measurement-value": "FXG17Hi9aFEr94MLtqKkSyBHx99Xv0eeuKHA5b6gpQE=",
13        "signer-id": "U3h5YwdTXfPsjYsVouLcVkBnD0wYM/jIjjA+pc/eqM=",
14        "measurement-description": "sha-256"
15      },
16      {
17        "measurement-type": "AP_BL2",
18        "measurement-value": "EBWbryYrQ6ktldtZ2uH3LGRRJzAWYeCjzk44spWpfFg=",
19        "signer-id": "U3h5YwdTXfPsjYsVouLcVkBnD0wYM/jIjjA+pc/eqM=",
20        "measurement-description": "sha-256"
21      },
22      {
23        "measurement-type": "AP_BL31",
24        "measurement-value": "Lm0xpZg6kSub+uWu+hwKGdi6PPYB00inBrTPqWYaa4o=",
25        "signer-id": "U3h5YwdTXfPsjYsVouLcVkBnD0wYM/jIjjA+pc/eqM=",
26        "measurement-description": "sha-256"
27      },
28      {
29        "measurement-type": "RMM",
30        "measurement-value": "oftQ5shvrhZ57zNRKW/WcTQRoIz43ReQpP0F+uhogWQ=",
31        "signer-id": "U3h5YwdTXfPsjYsVouLcVkBnD0wYM/jIjjA+pc/eqM=",
32        "measurement-description": "sha-256"
33      },
34      ...
35    ],
36    "cca-platform-service-indicator": "https://veraison.example",
37    "cca-platform-hash-algo-id": "sha-256"
38  },
39  "cca-realm-delegated-token": {
40    "cca-realm-profile": "tag:arm.com,2023:realm#1.0.0",
41    "cca-realm-challenge": "bobW2XzHE7xt1D285JGmtAMRwCeov4WjnaY+nORMEyqKEZ0pb65qaZnpvz5Ec...",
42    "cca-realm-personalization-value": "VGhlIHf1aWNRIGJyb3duIGZveCBqdW1wcyBvdnVvIDEzIGxhe...",
43    "cca-realm-initial-measurement": "MRMUq3NiA1DPdYg0rlxl2ejC3H/r5ufZZUu+hk4wDUK=",
44    "cca-realm-extensible-measurements": [
45      "JNWwopbMBcvYBoxQZ8W9Rzt3Ddpq4IL+06MKvj+aarE=", ...
46    ],
47    "cca-realm-hash-algo-id": "sha-256",
48    "cca-realm-public-key": "pAECIAIhWDB2+YgJG+WF7UGAGuz6uFhUjGMFfhaw5nYSC70NL5wp4FbF1BoB...",
49    "cca-realm-public-key-hash-algo-id": "sha-256"
50  }
```

Figure A.1 – Content of an Arm CCA attestation [21, 252].

Bibliography

- [1] Advanced Micro Devices (AMD). 2005. Secure Virtual Machine Architecture Reference Manual.
- [2] Advanced Micro Devices (AMD). 2008. AMD-V™ Nested Paging. <http://developer.amd.com/wordpress/media/2012/10/NPT-WP-1%201-final-TM.pdf>
- [3] Advanced Micro Devices (AMD). 2017. Protecting VM Register State with SEV-ES. <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/Protecting-VM-Register-State-with-SEV-ES.pdf>
- [4] Advanced Micro Devices (AMD). 2018. Secure Encrypted Virtualization (SEV). <https://developer.amd.com/sev/>
- [5] Advanced Micro Devices (AMD). 2020. Strengthening VM isolation with integrity protection and more. <https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more>
- [6] Advanced Micro Devices (AMD). 2021. Versioned Chip Endorsement Key (VCEK) Certificate and KDS Interface Specification. <https://docs.amd.com/v/u/en-US/57230>
- [7] Advanced Micro Devices (AMD). 2021. AMD Memory Encryption. <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/Protecting-VM-Register-State-with-SEV-ES.pdf>
- [8] Advanced Micro Devices (AMD). 2023. AMD64 Architecture Programmer's Manual Volume 2: System Programming. (2023). <https://www.amd.com/system/files/TechDocs/24593.pdf>
- [9] Advanced Micro Devices (AMD). AMD SEV-TIO: Trusted I/O for Secure Encrypted Virtualization. <https://www.amd.com/content/dam/amd/en/documents/developer/sev-tio-whitepaper.pdf>
- [10] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *Proceedings of the 17th Symposium on Networked Systems Design and Implementation (NSDI)*, 2020. 419–434.
- [11] Sten Agerholm and Peter Gorm Larsen. 1998. A Lightweight Approach to Formal Methods. In *Proceedings of the International Workshop on Current Trends in Applied Formal Method*, 1998. 168–183. https://doi.org/10.1007/3-540-48257-1_10
- [12] Adil Ahmad, Botong Ou, Congyu Liu, Xiaokuan Zhang, and Pedro Fonseca. 2023. Veil: A Protected Services Framework for Confidential Virtual Machines. In *Proceedings of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXVIII)*, 2023. 378–393. <https://doi.org/10.1145/3623278.3624763>

- [13] Eyad Alkassar, Mark A. Hillebrand, Wolfgang J. Paul, and Elena Petrova. 2010. Automated Verification of a Small Hypervisor. In *Proceedings of the 3rd International Conference on Verified Software, Theories, Tools and Experiments (VSTTE)*, 2010. 40–54. https://doi.org/10.1007/978-3-642-15057-9_3
- [14] Saar Amar, Tony Chen, David Chisnall, Nathaniel Wesley Filardo, Ben Laurie, Hugo Lefeuvre, Kunyan Liu, Simon W Moore, Robert Norton-Wright, Margo Seltzer, and others. 2025. CHERIoT RTOS: An OS for Fine-Grained Memory-Safe Compartments on Low-Cost Embedded Devices. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025. 67–84.
- [15] Saar Amar, David Chisnall, Tony Chen, Nathaniel Wesley Filardo, Ben Laurie, Kunyan Liu, Robert M. Norton, Simon W. Moore, Yucong Tao, Robert N. M. Watson, and Hongyan Xia. 2023. CHERIoT: Complete Memory Safety for Embedded Devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023. 641–653. <https://doi.org/10.1145/3613424.3614266>
- [16] Nadav Amit, Dan Tsafir, Assaf Schuster, Ahmad Ayoub, and Eran Shlomo. 2015. Virtual CPU validation. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP)*, 2015. 311–327.
- [17] James P Anderson. 1972. *Computer security technology planning study*.
- [18] APPLE. 2024. Private Cloud Compute: A new frontier for AI privacy in the cloud. <https://security.apple.com/blog/private-cloud-compute/>
- [19] APPLE. APPLE Platform Security. https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf
- [20] ARM Limited. ARM Realm Management Extension (RME) System Architecture. <https://documentation-service.arm.com/static/69427c8d6f88316d78a854b3>
- [21] ARM Limited. Download and inspect the attestation token. <https://learn.arm.com/learning-paths/servers-and-cloud-computing/cca-veraison/attestation-token/>
- [22] ARM Limited. Building a Secure System using TRUSTZONE Technology. <https://documentation-service.arm.com/static/5f212796500e883ab8e74531>
- [23] ARM Limited. Straight-line Speculation. <https://developer.arm.com/documentation/102825/latest/>
- [24] ARM Limited. Realm Management Monitor specification. <https://developer.arm.com/documentation/den0137/1-0eac5/>
- [25] ARM Limited. SMC Calling Convention. <https://developer.arm.com/documentation/den0028/latest/>
- [26] ARM Limited. ARMv-A Base RevC AEM fixed virtual platform (FVP). <https://www.arm.com/products/development-tools/simulation/fixed-virtual-platforms>
- [27] ARM Limited. CCA-capable LINUX kernel. <https://gitlab.arm.com/linux-arm/linux-cca>
- [28] ARM Limited. CCA-capable kvmtool VMM. <https://gitlab.arm.com/linux-arm/kvmtool-cca>

- [29] Alasdair ARMstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA semantics for ARMv8-a, RISC-v, and CHERI-MIPS. *Proc. ACM Program. Lang.* 3, POPL (2019), 71:1–71:31. <https://doi.org/10.1145/3290384>
- [30] Sergei Arnautov, Bohdan Trach, Franz Gregor, Thomas Knauth, André Martin, Christian Priebe, Joshua Lind, Divya Muthukumaran, Dan O’Keeffe, Mark Stillwell, David Goltzsche, David M. Eysers, Rüdiger Kapitza, Peter R. Pietzuch, and Christof Fetzer. 2016. SCONE: Secure LINUX Containers with INTEL SGX. In *Proceedings of the 12th Symposium on Operating System Design and Implementation (OSDI)*, 2016. 689–703.
- [31] Vytautas Astrauskas, Aurel Bílý, Jonás Fiala, Zachary Grannan, Christoph Matheja, Peter Müller, Federico Poli, and Alexander J. Summers. 2022. The PRUSTI Project: Formal Verification for RUST. In *Proceedings of the 14th NASA Formal Methods Symposium (NFM ’22)*, 2022. 88–108. https://doi.org/10.1007/978-3-031-06773-0_5
- [32] Jeremy Avigad. 2026. Mathematicians in the age of AI. *arXiv preprint arXiv:2603.03684* (2026).
- [33] Sacha-Élie Ayoun, Opale Sjöstedt, and Azalea Raad. 2025. SOTERIA: Efficient Symbolic Execution as a Functional Library. *CoRR* (2025). <https://doi.org/10.48550/arXiv.2511.08729>
- [34] Ahmed M. Azab, Peng Ning, Jitesh Shah, Quan Chen, Rohan Bhutkar, Guruprasad Ganesh, Jia Ma, and Wenbo Shen. 2014. Hypervision Across Worlds: Real-time Kernel Protection from the ARM TRUSTZONE Secure World. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2014. 90–102. <https://doi.org/10.1145/2660267.2660350>
- [35] Raad Bahmani, Ferdinand Brasser, Ghada Dessouky, Patrick Jauernig, Matthias Klimmek, Ahmad-Reza Sadeghi, and Emmanuel Stapf. 2021. CURE: A Security Architecture with CUsomizable and Resilient Enclaves. In *Proceedings of the 30th USENIX Security Symposium*, 2021. 1073–1090.
- [36] Enrico Barberis, Pietro Frigo, Marius Muench, Herbert Bos, and Cristiano Giuffrida. 2022. Branch History Injection: On the Effectiveness of Hardware Mitigations Against Cross-Privilege Spectre-v2 Attacks. In *Proceedings of the 31st USENIX Security Symposium*, 2022. 971–988.
- [37] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. 2003. Xen and the art of virtualization. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, 2003. 164–177.
- [38] Jeff Barr. 2020. AWS Nitro Enclaves – Isolated EC2 Environments to Process Confidential Data. <https://aws.amazon.com/blogs/aws/aws-nitro-enclaves-isolated-ec2-environments-to-process-confidential-data/>
- [39] Luiz André Barroso, Jimmy Clidaras, and Urs Hölzle. 2013. The datacenter as a computer. *Morgan and Claypool* (2013).

- [40] Andrew Baumann, Marcus Peinado, and Galen C. Hunt. 2015. Shielding Applications from an Untrusted Cloud with Haven. *ACM Trans. Comput. Syst.* 33, 3 (2015), 8:1–8:26. <https://doi.org/10.1145/2799647>
- [41] Andrew Baumann. 2017. Hardware is the new Software. In *Proceedings of The 16th Workshop on Hot Topics in Operating Systems (HotOS-XVI)*, 2017. 132–137.
- [42] Jonathan Behrens, Adam Belay, and M. Frans Kaashoek. 2022. Performance evolution of mitigating transient execution attacks. In *Proceedings of the 2022 EuroSys Conference*, 2022. 251–265.
- [43] Jonathan Behrens, Adam Belay, and M. Frans Kaashoek. 2022. Performance evolution of mitigating transient execution attacks. In *Proceedings of the 2022 EuroSys Conference*, 2022. 251–265. <https://doi.org/10.1145/3492321.3519559>
- [44] Jonathan Behrens, Cel Skeggs, Samuel Ortiz, and Frans Kaashoek. RVirt. <https://github.com/mit-pdos/RVirt>
- [45] Fabrice Bellard. 2005. QEMU, a Fast and Portable Dynamic Translator. In *USENIX ATC, FREENIX Track*, 2005. 41–46.
- [46] James Bornholt, Rajeev Joshi, Vytas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, Jacob Van Geffen, and Andrew Warfield. 2021. Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021. 836–850. <https://doi.org/10.1145/3477132.3483540>
- [47] Pietro Borrello, Andreas Kogler, Martin Schwarzl, Moritz Lipp, Daniel Gruss, and Michael Schwarz. 2022. $\mathcal{A}$ PIC Leak: Architecturally Leaking Uninitialized Data from the Microarchitecture. In *Proceedings of the 31st USENIX Security Symposium*, 2022. 3917–3934.
- [48] Edouard Bugnion, Scott Devine, Kinshuk Govil, and Mendel Rosenblum. 1997. Disco: Running Commodity Operating Systems on Scalable Multiprocessors. *ACM Trans. Comput. Syst.* 15, 4 (1997), 412–447.
- [49] Edouard Bugnion, Scott Devine, Mendel Rosenblum, Jeremy Sugerman, and Edward Y. Wang. 2012. Bringing Virtualization to the x86 Architecture with the Original VMWARE Workstation. *ACM Trans. Comput. Syst.* 30, 4 (2012), 12:1–12:51.
- [50] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. 2018. Foreshadow: Extracting the Keys to the INTEL SGX Kingdom with Transient Out-of-Order Execution. In *Proceedings of the 27th USENIX Security Symposium*, 2018. 991–1008.
- [51] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. 2019. Breaking Virtual Memory Protection and the SGX Ecosystem with Foreshadow. *IEEE Micro* 39, 3 (2019), 66–74. <https://doi.org/10.1109/MM.2019.2910104>

- [52] Jo Van Bulck, Daniel Moghimi, Michael Schwarz, Moritz Lipp, Marina Minkin, Daniel Genkin, Yuval Yarom, Berk Sunar, Daniel Gruss, and Frank Piessens. 2020. LVI: Hijacking Transient Execution through Microarchitectural Load Value Injection. In *Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P)*, 2020. 54–72.
- [53] Jo Van Bulck, Daniel Moghimi, Michael Schwarz, Moritz Lipp, Marina Minkin, Daniel Genkin, Yuval Yarom, Berk Sunar, Daniel Gruss, and Frank Piessens. 2020. LVI: Hijacking Transient Execution through Microarchitectural Load Value Injection. In *Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P)*, 2020. 54–72. <https://doi.org/10.1109/SP40000.2020.00089>
- [54] Claudio Canella, Jo Van Bulck, Michael Schwarz, Moritz Lipp, Benjamin von Berg, Philipp Ortner, Frank Piessens, Dmitry Evtyushkin, and Daniel Gruss. 2019. A Systematic Evaluation of Transient Execution Attacks and Defenses. In *Proceedings of the 28th USENIX Security Symposium*, 2019. 249–266.
- [55] Claudio Canella, Daniel Genkin, Lukas Giner, Daniel Gruss, Moritz Lipp, Marina Minkin, Daniel Moghimi, Frank Piessens, Michael Schwarz, Berk Sunar, Jo Van Bulck, and Yuval Yarom. 2019. Fallout: Leaking Data on Meltdown-resistant CPUs. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019. 769–784. <https://doi.org/10.1145/3319535.3363219>
- [56] Claudio Canella, Mario Werner, Daniel Gruss, and Michael Schwarz. 2021. Automating Seccomp Filter Generation for LINUX Applications. In *Proceedings of the 2021 ACM Cloud Computing Security Workshop (CCSW)*, 2021. 139–151. <https://doi.org/10.1145/3474123.3486762>
- [57] Charly Castes and Andrew Baumann. 2024. Sharing is leaking: blocking transient-execution attacks with core-gapped confidential VMs. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXIX)*, 2024. 267–281. <https://doi.org/10.1145/3622781.3674190>
- [58] Charly Castes and Edouard Bugnion. 2022. Attestable Software Versioning for Stateful Confidential Computing. In *5th Workshop on System Software for Trusted Execution (SysTEX 2022)*, 2022.
- [59] Charly Castes and Adrien Ghosn. 2023. Dynamic Linkers Are the Narrow Waist of Operating Systems. In *Proceedings of the 12th Workshop on Programming Languages and Operating Systems (PLOS)*, 2023. 26–33. <https://doi.org/10.1145/3623759.3624548>
- [60] Charly Castes, François Costa, Nate Foster, Thomas Bourgeat, and Edouard Bugnion. 2025. Lightweight Hypervisor Verification: Putting the Hardware Burger on a Diet. In *Proceedings of The 20th Workshop on Hot Topics in Operating Systems (HotOS-XX)*, 2025. 27–33. <https://doi.org/10.1145/3713082.3730373>
- [61] Charly Castes, François Costa, Nate Foster, Thomas Bourgeat, and Edouard Bugnion. 2025. Lightweight Hypervisor Verification: Putting the Hardware Burger on a Diet. In *Proceedings of The 20th Workshop on Hot Topics in Operating Systems (HotOS-XX)*, 2025. 27–33. <https://doi.org/10.1145/3713082.3730373>

- [62] Charly Castes, Adrien Ghosn, Neelu Shivprakash Kalani, Yuchen Qian, Marios Kogias, Mathias Payer, and Edouard Bugnion. 2023. Creating Trust by Abolishing Hierarchies. In *Proceedings of The 19th Workshop on Hot Topics in Operating Systems (HotOS-XIX)*, 2023. 231–238. <https://doi.org/10.1145/3593856.3595900>
- [63] Charly Castes, Neelu Shivprakash Kalani, Sofia Saltovskaia, Noé Terrier, Abel Vexina Wilkinson, and Edouard Bugnion. 2024. Kicking the Firmware Out of the TCB with the MIRALIS Virtual Firmware Monitor. In *Proceedings of the 2nd Workshop on Kernel Isolation, Safety and Verification (KISV)*, 2024. 8–15. <https://doi.org/10.1145/3698576.3698764>
- [64] Can Cebeci, Yong-Hao Zou, Diyu Zhou, George Candea, and Clément Pit-Claudel. 2024. Practical Verification of System-Software Components Written in Standard C. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*, 2024. 455–472.
- [65] David Cerdeira, Nuno Santos, Pedro Fonseca, and Sandro Pinto. 2020. SoK: Understanding the Prevailing Security Vulnerabilities in TRUSTZONE-assisted TEE Systems. In *Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P)*, 2020. 1416–1432. <https://doi.org/10.1109/SP40000.2020.00061>
- [66] David Cerdeira, Nuno Santos, Pedro Fonseca, and Sandro Pinto. 2020. SoK: Understanding the Prevailing Security Vulnerabilities in TRUSTZONE-assisted TEE Systems. In *Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P)*, 2020. 1416–1432.
- [67] Boru Chen, Yingchen Wang, Pradyumna Shome, Christopher W. Fletcher, David Kohlbrenner, Riccardo Paccagnella, and Daniel Genkin. 2024. GoFetch: Breaking Constant-Time Cryptographic Implementations Using Data Memory-Dependent Prefetchers. In *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [68] Haogang Chen, Yandong Mao, Xi Wang, Dong Zhou, Nickolai Zeldovich, and M. Frans Kaashoek. 2011. LINUX kernel vulnerabilities: state-of-the-art defenses and open problems. In *Proceedings of the 2011 Asia-Pacific Workshop on Systems (APSys)*, 2011. 5. <https://doi.org/10.1145/2103799.2103805>
- [69] Xiaoxin Chen, Tal Garfinkel, E. Christopher Lewis, Pratap Subrahmanyam, Carl A. Waldspurger, Dan Boneh, Jeffrey S. Dworkin, and Dan R. K. Ports. 2008. Overshadow: a virtualization-based approach to retrofitting protection in commodity operating systems. In *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XIII)*, 2008. 2–13.
- [70] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. 2024. INTEL TDX Demystified: A Top-Down Approach. *ACM Comput. Surv.* 56, 9 (2024), 238:1–238:33.
- [71] CHIPS Alliance. Caliptra: A Datacenter System on a Chip (SoC) Root of Trust (RoT). <https://chipsalliance.github.io/caliptra-web/>
- [72] Joseph I. Choi and Kevin R. B. Butler. 2019. Secure Multiparty Computation and Trusted Hardware: Examining Adoption Challenges and Opportunities. *Secur. Com-*

- mun. Networks* 2019, (2019), 1368905:1–1368905:28. <https://doi.org/10.1155/2019/1368905>
- [73] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 2010 ACM Symposium on Cloud Computing (SOCC)*, 2010. 143–154.
 - [74] David Cooper, Stefan Santesson, Stephen Farrell, Sharon Boeyen, Russell Housley, and W. Timothy Polk. 2008. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. *RFC 5280*, (2008), 1–151. <https://doi.org/10.17487/RFC5280>
 - [75] Fernando J. Corbató and Victor A. Vyssotsky. 1965. Introduction and overview of the multics system. In *Proceedings of the Fall Joint Computer Conference (AFIPS '65)*, 1965. 185–196. <https://doi.org/10.1145/1463891.1463912>
 - [76] Victor Costan and Srinivas Devadas. 2016. INTEL SGX Explained. *IACR Cryptol. ePrint Arch.* 2016, (2016), 86.
 - [77] Victor Costan, Ilia A. Lebedev, and Srinivas Devadas. 2016. Sanctum: Minimal Hardware Extensions for Strong Software Isolation. In *Proceedings of the 25th USENIX Security Symposium*, 2016. 857–874.
 - [78] Ben Cottier, Robi Rahman, Loredana Fattorini, Nestor Maslej, Tamay Besiroglu, and David Owen. 2024. The rising costs of training frontier AI models. *arXiv preprint arXiv:2405.21015* (2024).
 - [79] Miles Dai, Riccardo Paccagnella, Miguel Gomez-Garcia, John D. McCalpin, and Mengjia Yan. 2022. Don't Mesh Around: Side-Channel Attacks and Mitigations on Mesh Interconnects. In *Proceedings of the 31st USENIX Security Symposium*, 2022. 2857–2874.
 - [80] Nathan Dautenhahn, Theodoros Kasampalis, Will Dietz, John Criswell, and Vikram S. Adve. 2015. Nested Kernel: An Operating System Architecture for Intra-Kernel Privilege Separation. In *Proceedings of the 20th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XX)*, 2015. 191–206.
 - [81] Drew Davidson, Benjamin Moench, Thomas Ristenpart, and Somesh Jha. 2013. FIE on Firmware: Finding Vulnerabilities in Embedded Systems Using Symbolic Execution. In *Proceedings of the 22nd USENIX Security Symposium*, 2013. 463–478.
 - [82] Patrick Davis, Sean Coffey, Lubjana Beshaj, and Nathaniel D Bastian. 2024. Emerging technologies for data security in zero trust environments. *The Cyber Defense Review* 9, 2 (2024), 49–72.
 - [83] Brian Delgado, Tejaswini Vibhute, and Karen L Karavanic. 2020. Applying the Principle of Least Privilege to System Management Interrupt Handlers with the INTEL SMI Transfer Monitor. In *Proceedings of the 9th International Workshop on Hardware and Architectural Support for Security and Privacy*, 2020. 1–9.

- [84] Xiaowan Dong, Zhuojia Shen, John Criswell, Alan L. Cox, and Sandhya Dwarkadas. 2018. Shielding Software From Privileged Side-Channel Attacks. In *Proceedings of the 27th USENIX Security Symposium*, 2018. 1441–1458.
- [85] Eclipsium. UEFicanhazbufferoverflow: Widespread Impact from Vulnerability in Popular PC and Server Firmware. <https://eclipsium.com/blog/>
- [86] Edgeless. Edgeless Continuum AI. <https://docs.edgeless.systems/continuum/0.3/overview>
- [87] EEMBC. CoreMark PRO. <https://www.eembc.org/coremark-pro>
- [88] Kevin Elphinstone and Gernot Heiser. 2013. From L3 to seL4 what have we learnt in 20 years of L4 microkernels?. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, 2013. 133–150.
- [89] Dawson R Engler, M Frans Kaashoek, and James O'Toole Jr. 1995. Exokernel: An operating system architecture for application-level resource management. *ACM SIGOPS Operating Systems Review* 29, 5 (1995), 251–266.
- [90] Dawson R. Engler, M. Frans Kaashoek, and James W. O'Toole Jr. 1995. Exokernel: An Operating System Architecture for Application-Level Resource Management. In *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*, 1995. 251–266. <https://doi.org/10.1145/224056.224076>
- [91] European Comission. Cyber Resilience Act. <https://digital-strategy.ec.europa.eu/en/library/cyber-resilience-act>
- [92] European Commission. 2024. Digital Operational Resilience Act (DORA). https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=OJ:L_202401774
- [93] Guy C. Fedorkow, Eric Voit, and Jessica Fitzgerald-McKay. 2024. Remote Integrity Verification of Network Devices Containing Trusted Platform Modules. *RFC 9683*, (2024), 1–36. <https://doi.org/10.17487/RFC9683>
- [94] Rich Felker. musl libc. <https://www.musl-libc.org/>
- [95] Bo Feng, Alejandro Mera, and Long Lu. 2020. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. In *Proceedings of the 29th USENIX Security Symposium*, 2020. 1237–1254.
- [96] Erhu Feng, Dahu Feng, Dong Du, Yubin Xia, Wenbin Zheng, Siqi Zhao, and Haibo Chen. 2024. sIOPMP: Scalable and Efficient I/O Protection for TEEs. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXIX)*, 2024. 1061–1076. <https://doi.org/10.1145/3620665.3640378>
- [97] Erhu Feng, Xu Lu, Dong Du, Bicheng Yang, Xueqiang Jiang, Yubin Xia, Binyu Zang, and Haibo Chen. 2021. Scalable Memory Protection in the PENG LAI Enclave. In *Proceedings of the 15th Symposium on Operating System Design and Implementation (OSDI)*, 2021. 275–294.
- [98] Andrew Ferraiuolo, Andrew Baumann, Chris Hawblitzel, and Bryan Parno. 2017. KOMODO: Using verification to disentangle secure-enclave hardware from software. In

Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP), 2017. 287–305.

- [99] Ben Fiedler, Roman Meier, Jasmin Schult, Daniel Schwyn, and Timothy Roscoe. 2023. Specifying the de-facto OS of a production SoC. In *Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification (KISV)*, 2023. 18–25.
- [100] Ben Fiedler, Roman Meier, Jasmin Schult, Daniel Schwyn, and Timothy Roscoe. 2023. Specifying the de-facto OS of a production SoC. In *Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification (KISV)*, 2023. 18–25. <https://doi.org/10.1145/3625275.3625400>
- [101] Ben Fiedler, Daniel Schwyn, Constantin Gierczak-Galle, David A. Cock, and Timothy Roscoe. 2023. Putting out the hardware dumpster fire. In *Proceedings of The 19th Workshop on Hot Topics in Operating Systems (HotOS-XIX)*, 2023. 46–52.
- [102] Ben Fiedler, Daniel Schwyn, Constantin Gierczak-Galle, David A. Cock, and Timothy Roscoe. 2023. Putting out the hardware dumpster fire. In *Proceedings of The 19th Workshop on Hot Topics in Operating Systems (HotOS-XIX)*, 2023. 46–52. <https://doi.org/10.1145/3593856.3595903>
- [103] Bryan Ford, Mike Hibler, Jay Lepreau, Patrick Tullmann, Godmar Back, and Stephen Clawson. 1996. Microkernels Meet Recursive Virtual Machines. In *Proceedings of the 2nd Symposium on Operating System Design and Implementation (OSDI)*, 1996. 137–151. <https://doi.org/10.1145/238721.238769>
- [104] Anthony C. J. Fox, Gareth Stockwell, Shale Xiong, Hanno Becker, Dominic P. Mulligan, Gustavo Petri, and Nathan Chong. 2023. A Verification Methodology for the ARM® Confidential Computing Architecture: From a Secure Specification to Safe Implementations. *Proc. ACM Program. Lang.* 7, OOPSLA1 (2023), 376–405. <https://doi.org/10.1145/3586040>
- [105] Tal Garfinkel, Ben Pfaff, Jim Chow, Mendel Rosenblum, and Dan Boneh. 2003. Terra: a virtual machine-based platform for trusted computing. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, 2003. 193–206. <https://doi.org/10.1145/945445.945464>
- [106] Georgi Gerganov. LLaMA-C++. <https://github.com/ggerganov/llama.cpp>
- [107] Adrien Ghosn, Charly Castes, Neelu S. Kalani, Yuchen Qian, Marios Kogias, and Edouard Bugnion. 2025. Rethinking the confidential cloud through a unified low-level abstraction for composable isolation. *CoRR* (2025). <https://doi.org/10.48550/arXiv.2507.12364>
- [108] Adrien Ghosn, James R. Larus, and Edouard Bugnion. 2019. Secured Routines: Language-based Construction of Trusted Execution Environments. In *Proceedings of the 2019 USENIX Annual Technical Conference (ATC)*, 2019. 571–586.
- [109] Will Glozer. 2021. Wrk - a HTTP benchmarking tool. <https://github.com/wg/wrk>
- [110] Oded Goldreich. 1998. Secure multi-party computation. *Manuscript. Preliminary version* 78, 110 (1998), 1–108.

- [111] Google. Android Security. <https://services.google.com/fh/files/misc/android-enterprise-security-paper-2023.pdf>
- [112] Google. AMD: Microcode Signature Verification Vulnerability. <https://github.com/google/security-research/security/advisories/GHSA-4xq7-4mgh-gp6w>
- [113] Floris Gorter, Taddeus Kroes, Herbert Bos, and Cristiano Giuffrida. 2024. Sticky Tags: Efficient and Deterministic Spatial Memory Error Mitigation using Persistent Memory Tags. In *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P)*, 2024. 4239–4257. <https://doi.org/10.1109/SP54263.2024.00263>
- [114] GRAMINE Project. The GRAMINE library OS. <https://gramineproject.io/>
- [115] GRAMINE Project. GRAMINE CI Examples. <https://github.com/gramineproject/gramine/tree/master/CI-Examples>
- [116] GRAMINE Project. GRAMINE: Exitless. <https://gramine.readthedocs.io/en/stable/manifest-syntax.html?highlight=exitless#number-of-rpc-threads-exitless-feature>
- [117] GRAMINE Project. GRAMINE: threading. <https://gramine.readthedocs.io/en/stable/manifest-syntax.html?highlight=num%20threads#number-of-threads>
- [118] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Newman) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. 2016. CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels. In *Proceedings of the 12th Symposium on Operating System Design and Implementation (OSDI)*, 2016. 653–669.
- [119] Aditya Gulavani, Hormuzd M. Khosravi, Balaji Masanamuthu Chinnathurai, and Shiny Sebastian. Performance Considerations of Hardware-Isolated Partitioned VMs with INTEL TDX. <https://www.intel.com/content/www/us/en/developer/articles/technical/tdx-performance-isolated-partitioned-vm.html>
- [120] Suyash Gupta, Sajjad Rahnema, Shubham Pandey, Natacha Crooks, and Mohammad Sadoghi. 2023. Dissecting BFT Consensus: In Trusted Components we Trust!. In *Proceedings of the 2023 EuroSys Conference*, 2023. 521–539. <https://doi.org/10.1145/3552326.3587455>
- [121] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. 2017. Bringing the web up to speed with WEBASSEMBLY. In *Proceedings of the ACM SIGPLAN 2017 Conference on Programming Language Design and Implementation (PLDI)*, 2017. 185–200. <https://doi.org/10.1145/3062341.3062363>
- [122] Ori Hadary, Luke Marshall, Ishai Menache, Abhisek Pan, Esaias E. Greeff, David Dion, Star Dorminey, Shailesh Joshi, Yang Chen, Mark Russinovich, and Thomas Moscibroda. 2020. Protean: VM Allocation Service at Scale. In *Proceedings of the 14th Symposium on Operating System Design and Implementation (OSDI)*, 2020. 845–861.
- [123] Norman Hardy. 1988. The Confused Deputy (or why capabilities might have been invented). *ACM SIGOPS Oper. Syst. Rev.* 22, 4 (1988), 36–38. <https://doi.org/10.1145/54289.871709>

- [124] Gernot Heiser. 2025. *The seL4 Microkernel – An Introduction*. <https://sel4.systems/About/SEL4-whitepaper.pdf>
- [125] Mathé Hertogh, Sander Wiebing, and Cristiano Giuffrida. 2024. Leaky Address Masking: Exploiting Unmasked Spectre Gadgets with Noncanonical Address Translation. In *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P)*, 2024. 3773–3788. <https://doi.org/10.1109/SP54263.2024.00158>
- [126] Mathé Hertogh, Manuel Wiesinger, Sebastian Österlund, Marius Muench, Nadav Amit, Herbert Bos, and Cristiano Giuffrida. 2023. Quarantine: Mitigating Transient Execution Attacks with Physical Domain Isolation. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*, 2023. 207–221. <https://doi.org/10.1145/3607199.3607248>
- [127] D. Richard Hipp. SQLite. <https://www.sqlite.org/>
- [128] Alexander Van't Hof and Jason Nieh. 2022. BlackBox: A Container Security Monitor for Protecting Containers on Untrusted Operating Systems. In *Proceedings of the 16th Symposium on Operating System Design and Implementation (OSDI)*, 2022. 683–700.
- [129] Jana Hofmann, Emanuele Vannacci, Cédric Fournet, Boris Köpf, and Oleksii Oleksenko. 2023. Speculation at Fault: Modeling and Testing Microarchitectural Leakage of CPU Exceptions. In *Proceedings of the 32nd USENIX Security Symposium*, 2023. 7143–7160.
- [130] Owen S. Hofmann, Sangman Kim, Alan M. Dunn, Michael Z. Lee, and Emmett Witchel. 2013. InkTag: secure applications on an untrusted operating system. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XVIII)*, 2013. 265–278.
- [131] Galen C. Hunt and James R. Larus. 2007. Singularity: rethinking the software stack. *ACM SIGOPS Oper. Syst. Rev.* 41, 2 (2007), 37–49. <https://doi.org/10.1145/1243418.1243424>
- [132] Ray Hunt. 2001. PKI and digital certification infrastructure. In *Proceedings. Ninth IEEE International Conference on Networks, ICON 2001.*, 2001. 234–239.
- [133] Tyler Hunt, Zhiting Zhu, Yuanzhong Xu, Simon Peter, and Emmett Witchel. 2016. Ryoan: A Distributed Sandbox for Untrusted Computation on Secret Data. In *Proceedings of the 12th Symposium on Operating System Design and Implementation (OSDI)*, 2016. 533–549.
- [134] hyper Contributors. RUST Hyper. <https://hyper.rs/>
- [135] INTEL Corp. 2014. Trusted Execution Technology. <https://www.intel.com/content/www/us/en/developer/articles/tool/intel-trusted-execution-technology.html>
- [136] INTEL Corp. 2015. INTEL Software Guard Extensions (INTEL SGX). <https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html>
- [137] INTEL Corp. 2017. Multi-Key Total Memory Encryption. <https://edc.intel.com/content/www/us/en/design/ipla/software-development-platforms/client/>

[platforms/alder-lake-desktop/12th-generation-intel-core-processors-datasheet-volume-1-of-2/002/intel-multi-key-total-memory-encryption/](#)

- [138] INTEL Corp. 2021. INTEL Trust Domain Extensions. <https://cdrdv2.intel.com/v1/dl/getContent/690419>
- [139] INTEL Corp. INTEL Trust Authority. <https://www.intel.com/content/www/us/en/security/trust-authority.html>
- [140] INTEL Corp. Speculative Behavior of SWAPGS and Segment Registers. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/speculative-behavior-swaps-and-segment-registers.html>
- [141] INTEL Corp. Snoop-Assisted L1 Data Sampling. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/snoop-assisted-l1-data-sampling.html>
- [142] INTEL Corp. Processor MMIO Stale Data Vulnerabilities. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/processor-mmio-stale-data-vulnerabilities.html>
- [143] INTEL Corp. Trust Domain Security Guidance for Developers. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/best-practices/trusted-domain-security-guidance-for-developers.html>
- [144] INTEL Corp. INTEL Transactional Synchronization Extension (TSX) Disable Update for Selected Processors. <https://cdrdv2.intel.com/v1/dl/getContent/643557>
- [145] INTEL Corp. INTEL SGX Attestation Technical Details. <https://www.intel.com/content/www/us/en/security-center/technical-details/sgx-attestation-technical-details.html>
- [146] INTEL Corp. INTEL TDX Connect Architecture Specification. <https://www.intel.com/content/www/us/en/content-details/773614/intel-tdx-connect-architecture-specification.html>
- [147] INTEL Corp. INTEL virtualization technology for directed I/O, architecture specification. <https://www.intel.com/content/www/us/en/content-details/774206/intel-virtualization-technology-for-directed-i-o-architecture-specification.html>
- [148] Daniel Jackson. 2001. Lightweight Formal Methods. In *Proceedings of the 2001 International Symposium on Formal Methods Europe*, 2001. 1. https://doi.org/10.1007/3-540-45251-6_1
- [149] Jai Vijayan. Millions of Devices Vulnerable to 'PKFail' Secure Boot Bypass Issue. <https://www.darkreading.com/endpoint-security/millions-of-devices-vulnerable-to-pkfail-secure-boot-bypass-issue>
- [150] Yuekai Jia, Shuang Liu, Wenhao Wang, Yu Chen, Zhengde Zhai, Shoumeng Yan, and Zhengyu He. 2022. HyperEnclave: An Open and Cross-platform Trusted Execution Environment. In *Proceedings of the 2022 USENIX Annual Technical Conference (ATC)*, 2022. 437–454.

- [151] David Kaplan. 2024. Hardware VM Isolation in the Cloud. *Commun. ACM* 67, 1 (2024), 54–59. <https://doi.org/10.1145/3624576>
- [152] Eric Keller, Jakub Szefer, Jennifer Rexford, and Ruby B. Lee. 2010. NoHype: virtualized cloud infrastructure without the virtualization. In *Proceedings of the 37th International Symposium on Computer Architecture (ISCA)*, 2010. 350–361. <https://doi.org/10.1145/1815961.1816010>
- [153] Juhee Kim, Jinbum Park, Sihyeon Roh, Jaeyoung Chung, Youngjoo Lee, Taesoo Kim, and Byoungyoung Lee. 2025. Tiktag: Breaking ARM's Memory Tagging Extension with Speculative Execution. In *Proceedings of the 46th IEEE Symposium on Security and Privacy (S&P)*, 2025. 4063–4081. <https://doi.org/10.1109/SP61157.2025.00039>
- [154] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David A. Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: formal verification of an OS kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, 2009. 207–220.
- [155] Jan Kneschke. Lighttpd. <https://www.lighttpd.net/>
- [156] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2019. Spectre Attacks: Exploiting Speculative Execution. In *IEEE Symposium on Security and Privacy*, 2019. 1–19.
- [157] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2019. Spectre Attacks: Exploiting Speculative Execution. In *IEEE Symposium on Security and Privacy*, 2019. 1–19. <https://doi.org/10.1109/SP.2019.00002>
- [158] Alexey Kopytov. Sysbench: Scriptable Database and System Performance Benchmark. <https://github.com/akopytov/sysbench>
- [159] Mark Kuhne, Stavros Volos, and Shweta Shinde. 2024. DORAMI: Privilege Separating Security Monitor on RISC-V TEEs. *CoRR* (2024).
- [160] Máté Kukri. TPM GPIO fail: How bad OEM firmware ruins TPM security. <https://mkukri.xyz/2024/06/01/tpm-gpio-fail.html>
- [161] Dmitrii Kuvaiskii, Dimitrios Stavrakakis, Kailun Qin, Cedric Xing, Pramod Bhatotia, and Mona Vij. 2024. GRAMINE-TDX: A Lightweight OS Kernel for Confidential VMs. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024. 4598–4612. <https://doi.org/10.1145/3658644.3690323>
- [162] REDIS Labs. MEMTIER Benchmark. https://github.com/REDISLabs/memtier_benchmark
- [163] Butler W. Lampson and Howard E. Sturgis. 1976. Reflections on an Operating System Design. *Commun. ACM* 19, 5 (1976), 251–265. <https://doi.org/10.1145/360051.360074>

- [164] Alex Landau, Muli Ben-Yehuda, and Abel Gordon. 2011. SplitX: Split Guest/Hypervisor Execution on Multi-Core. In *Proceedings of the 3rd USENIX Workshop on I/O Virtualization (WIOV)*, 2011.
- [165] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Oded Padon, and Bryan Parno. 2024. VERUS: A Practical Foundation for Systems Verification. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*, 2024. 438–454. <https://doi.org/10.1145/3694715.3695952>
- [166] Dayeol Lee, David Kohlbrenner, Shweta Shinde, Krste Asanovic, and Dawn Song. 2020. KEYSTONE: an open framework for architecting trusted execution environments. In *Proceedings of the 2020 EuroSys Conference*, 2020. 38:1–38:16.
- [167] Hugo Lefeuvre, David Chisnall, Marios Kogias, and Pierre Olivier. 2023. Towards (Really) Safe and Fast Confidential I/O. In *Proceedings of The 19th Workshop on Hot Topics in Operating Systems (HotOS-XIX)*, 2023. 214–222. <https://doi.org/10.1145/3593856.3595913>
- [168] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. FLUX: Liquid Types for RUST. *Proc. ACM Program. Lang.* 7, PLDI (2023), 1533–1557. <https://doi.org/10.1145/3591283>
- [169] Dirk Leinenbach and Thomas Santen. 2009. Verifying the MICROSOFT Hyper-V Hypervisor with VCC. In *Proceedings of the 16th International Symposium on Formal Methods (FM)*, 2009. 806–809. https://doi.org/10.1007/978-3-642-05089-3_51
- [170] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B. Giffin, Pat Pannuto, Prabal Dutta, and Philip Alexander Levis. 2017. Multiprogramming a 64kB Computer Safely and Efficiently. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*, 2017. 234–251.
- [171] Dingji Li, Zeyu Mi, Chenhui Ji, Yifan Tan, Binyu Zang, Haibing Guan, and Haibo Chen. 2023. Bifrost: Analysis and Optimization of Network I/O Tax in Confidential Virtual Machines. In *Proceedings of the 2023 USENIX Annual Technical Conference (ATC)*, 2023. 1–15.
- [172] Dingji Li, Zeyu Mi, Yubin Xia, Binyu Zang, Haibo Chen, and Haibing Guan. 2021. TwinVisor: Hardware-isolated Confidential Virtual Machines for ARM. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021. 638–654.
- [173] Shih-Wei Li, John S. Koh, and Jason Nieh. 2019. Protecting Cloud Virtual Machines from Hypervisor and Host Operating System Exploits. In *Proceedings of the 28th USENIX Security Symposium*, 2019. 1357–1374.
- [174] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. 2021. Formally Verified Memory Protection for a Commodity Multiprocessor Hypervisor. In *Proceedings of the 30th USENIX Security Symposium*, 2021. 3953–3970.
- [175] Xupeng Li, Xuheng Li, Christoffer Dall, Ronghui Gu, Jason Nieh, Yousuf Sait, and Gareth Stockwell. 2022. Design and Verification of the ARM Confidential Compute Architecture. In *Proceedings of the 16th Symposium on Operating System Design and Implementation (OSDI)*, 2022. 465–484.

- [176] Jochen Liedtke. 1995. On micro-Kernel Construction. In *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*, 1995. 237–250.
- [177] Christian Lindenmeier, Mathias Payer, and Marcel Busch. 2024. EL3XIR: Fuzzing COTS Secure Monitors. In *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [178] The LINUX Kernel Community. LINUX Kernel Virtual Machine. https://linux-kvm.org/page/Main_Page
- [179] The LINUX Kernel Community. VIRTIO. <https://www.linux-kvm.org/page/Virtio>
- [180] The LINUX Kernel Community. LINUX - DMA and swiotlb. <https://docs.kernel.org/core-api/swiotlb.html>
- [181] LINUX Kernel Community. kvm_stat: KVM statistics reporting tool. https://www.kernel.org/doc/html/latest/virt/kvm/tools/kvm_stat.html
- [182] LINUX Kernel Community. Seccomp BPF (SECure COMPUting with filters). https://www.kernel.org/doc/Documentation/prctl/seccomp_filter.txt
- [183] Steven B. Lipner. 2015. The Birth and Death of the Orange Book. *IEEE Ann. Hist. Comput.* 37, 2 (2015), 19–31. <https://doi.org/10.1109/MAHC.2015.27>
- [184] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. 2018. Melt-down. *CoRR* (2018).
- [185] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. 2015. Last-Level Cache Side-Channel Attacks are Practical. In *IEEE Symposium on Security and Privacy*, 2015. 605–622. <https://doi.org/10.1109/SP.2015.43>
- [186] Tong Liu, Zhen Huang, Jiaao Li, Jianyu Niu, Guoxing Chen, and Yinqian Zhang. 2024. SoK: Opportunities for Accelerating Multi - Party Computation via Trusted Hardware. In *Proceedings of the 3rd IEEE International Symposium on Secure and Private Execution Environment Design (SEED)*, 2024. 143–154. <https://doi.org/10.1109/SEED61283.2024.00024>
- [187] The lkvm Contributors. Native KVM LINUX Tool. <https://github.com/lkvm/lkvm>
- [188] LLVM contributors. libFuzzer: A library for in-process, coverage-guided fuzzing. <https://llvm.org/docs/LibFuzzer.html>
- [189] Laurence Lundblade, Giridhar Mandyam, Jeremy O'Donoghue, and Carl Wallace. 2025. The Entity Attestation Token (EAT). *RFC 9711*, (2025), 1–85. <https://doi.org/10.17487/RFC9711>
- [190] Lorenzo Martignoni, Stephen McCamant, Pongsin Poosankam, Dawn Song, and Petros Maniatis. 2012. Path-exploration lifting: hi-fi tests for lo-fi emulators. In *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XVII)*, 2012. 337–348. <https://doi.org/10.1145/2150976.2151012>

- [191] Jonathan M. McCune, Yanlin Li, Ning Qu, Zongwei Zhou, Anupam Datta, Virgil D. Gligor, and Adrian Perrig. 2010. TRUSTVISOR: Efficient TCB Reduction and Attestation. In *"IEEE Symposium on Security and Privacy"*, 2010. 143–158.
- [192] Meta. Llama-3.2-1B-Instruct. <https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct>
- [193] Jesse De Meulemeester, David Oswald, Ingrid Verbauwhede, and Jo Van Bulck. 2026. Battering RAM: Lowcost interposer attacks on confidential computing via dynamic memory aliasing. In *47th IEEE Symposium on Security and Privacy (S&P)*, 2026.
- [194] Jesse De Meulemeester, Luca Wilke, David F. Oswald, Thomas Eisenbarth, Ingrid Verbauwhede, and Jo Van Bulck. 2025. BadRAM: Practical Memory Aliasing Attacks on Trusted Execution Environments. In *Proceedings of the 46th IEEE Symposium on Security and Privacy (S&P)*, 2025. 4117–4135. <https://doi.org/10.1109/SP61157.2025.00104>
- [195] Thomas Meyer-Lehnert. 2024. Towards a Unified Firmware for Enzian. B.S. thesis.
- [196] MICROSOFT AZURE. MICROSOFT AZURE Attestation. <https://learn.microsoft.com/en-us/azure/attestation/overview>
- [197] MICROSOFT. OpenHCL. https://openvmm.dev/guide/user_guide/openhcl.html
- [198] MICROSOFT. Virtualization-based security (VBS) enclaves. <https://learn.microsoft.com/en-us/windows/win32/trusted-execution/vbs-enclaves>
- [199] MICROSOFT. Analysis and mitigation of speculative store bypass (CVE-2018-3639). <https://msrc.microsoft.com/blog/2018/05/analysis-and-mitigation-of-speculative-store-bypass-cve-2018-3639/>
- [200] MICROSOFT. Manage Hyper-V hypervisor scheduler types. <https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/manage/manage-hyper-v-scheduler-types>
- [201] MICROSOFT. Introducing Hyperlight: Virtual machine-based security for functions at scale. <https://opensource.microsoft.com/blog/2024/11/07/introducing-hyperlight-virtual-machine-based-security-for-functions-at-scale/>
- [202] Mitar Milutinovic, Warren He, Howard Wu, and Maxinder Kanwal. 2016. Proof of Luck: an Efficient Blockchain Consensus Protocol. In *Proceedings of the 1st Workshop on System Software for Trusted Execution (SysTEX)*, 2016. 2:1–2:6. <https://doi.org/10.1145/3007788.3007790>
- [203] Daniel Moghimi. 2023. Downfall: Exploiting Speculative Data Gathering. In *Proceedings of the 32nd USENIX Security Symposium*, 2023. 7179–7193.
- [204] Mathias Morbitzer, Manuel Huber, Julian Horsch, and Sascha Wessel. 2018. SEVered: Subverting AMD's Virtual Machine Encryption. In *Proceedings of the 11th European Workshop on Systems Security (EUROSEC)*, 2018. 1:1–1:6. <https://doi.org/10.1145/3193111.3193112>
- [205] Leonardo Mendonça de Moura and Nikolaj S. Bjørner. 2008. Z3: An Efficient SMT Solver. In *Proceedings of 14th International Conference on Tools and Algorithms*

- for Construction and Analysis of Systems (TACAS)*, 2008. 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
- [206] Kit Murdock, David F. Oswald, Flavio D. Garcia, Jo Van Bulck, Daniel Gruss, and Frank Piessens. 2020. Plundervolt: Software-based Fault Injection Attacks against INTEL SGX. In *Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P)*, 2020. 1466–1482. <https://doi.org/10.1109/SP40000.2020.00057>
 - [207] Near AI. Private Inference. <https://docs.near.ai/cloud/private-inference/>
 - [208] Luke Nelson, James Bornholt, Ronghui Gu, Andrew Baumann, Emina Torlak, and Xi Wang. 2019. Scaling symbolic evaluation for automated verification of systems code with Serval. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019. 225–242.
 - [209] Luke Nelson, Helgi Sigurbjarnarson, Kaiyuan Zhang, Dylan Johnson, James Bornholt, Emina Torlak, and Xi Wang. 2017. Hyperkernel: Push-Button Verification of an OS Kernel. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*, 2017. 252–269.
 - [210] Michael Nieves, Kelley Dempsey, Victoria Yan Pillitteri, and others. 2017. An introduction to information security. *NIST special publication* 800, 12 (2017), 101.
 - [211] Enrique Nissim and Krzysztof Okupski. "Sinkclose" Vulnerability Affects Every AMD CPU Dating Back to 2006. <https://www.techpowerup.com/325488/sinkclose-vulnerability-affects-every-amd-cpu-dating-back-to-2006>
 - [212] William D. Norcott, Don Capps, and others. IOZONE Filesystem Benchmark. <https://www.iozone.org/>
 - [213] Oleksii Oleksenko, Marco Guarnieri, Boris Köpf, and Mark Silberstein. 2023. Hide and Seek with Spectres: Efficient discovery of speculative information leaks with random testing. In *Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P)*, 2023. 1737–1752. <https://doi.org/10.1109/SP46215.2023.10179391>
 - [214] Oleksii Oleksenko, Bohdan Trach, Robert Krahn, Mark Silberstein, and Christof Fetzer. 2018. Varys: Protecting SGX Enclaves from Practical Side-Channel Attacks. In *Proceedings of the 2018 USENIX Annual Technical Conference (ATC)*, 2018. 227–240.
 - [215] OPENTITAN Project. OPENTITAN: The First Open Source Silicon Root of Trust. <https://opentitan.org/>
 - [216] Oracle. Hard Partitioning with Oracle LINUX KVM. <https://www.oracle.com/a/ocom/docs/linux/ol-kvm-hard-partitioning.pdf>
 - [217] Meni Orenbach, Pavel Lifshits, Marina Minkin, and Mark Silberstein. 2017. Eleos: Exit-Less OS Services for SGX Enclaves. In *Proceedings of the 2017 EuroSys Conference*, 2017. 238–253. <https://doi.org/10.1145/3064176.3064219>
 - [218] Meni Orenbach, Pavel Lifshits, Marina Minkin, and Mark Silberstein. 2017. Eleos: Exit-Less OS Services for SGX Enclaves. In *Proceedings of the 2017 EuroSys Conference*, 2017. 238–253. <https://doi.org/10.1145/3064176.3064219>
 - [219] Tavis Ormandy. Zenbleed. <https://cmpxchg8b.com/zenbleed.html>

- [220] Tavis Ormandy. Reptar. <https://lock.cmpxchg8b.com/reptar.html>
- [221] Barclay Osborn, Justin McWilliams, Betsy Beyer, and Max Saltonstall. 2016. Beyond-Corp: Design to Deployment at Google. *login Usenix Mag.* 41, 1 (2016).
- [222] Oxide Computer Company. A small open-source operating system for deeply-embedded computer systems. <https://hubris.oxide.computer/>
- [223] Wojciech Ozga, Guernsey D. H. Hunt, Michael V. Le, Elaine R. Palmer, and Avraham Shinnar. 2023. Towards a Formally Verified Security Monitor for VM-based Confidential Computing. In *Proceedings of the 12th International Workshop on Hardware and Architectural Support for Security and Privacy (HASP2023)*, 2023.
- [224] Joongun Park, Naegyeong Kang, Taehoon Kim, Youngjin Kwon, and Jaehyuk Huh. 2020. Nested Enclave: Supporting Fine-grained Hierarchical Isolation with SGX. In *Proceedings of the 47th International Symposium on Computer Architecture (ISCA)*, 2020. 776–789. <https://doi.org/10.1109/ISCA45697.2020.00069>
- [225] PCI-SIG. TEE Device Interface Security Protocol (TDISP).
- [226] Marcus Peinado, Yuqun Chen, Paul England, and John Manferdelli. 2004. NGSCB: A trusted open system. In *Australasian Conference on Information Security and Privacy*, 2004. 86–97.
- [227] Solal Pirelli, Akvile Valentukonyte, Katerina J. Argyraki, and George Candea. 2022. Automated Verification of Network Function Binaries. In *Proceedings of the 19th Symposium on Networked Systems Design and Implementation (NSDI)*, 2022. 585–600.
- [228] Gerald J. Popek and Robert P. Goldberg. 1974. Formal Requirements for Virtualizable Third Generation Architectures. *Commun. ACM* 17, 7 (1974), 412–421.
- [229] Thomas Pornin. BearSSL. <https://bearssl.org/>
- [230] Hany Ragab, Andrea Mambretti, Anil Kurmus, and Cristiano Giuffrida. 2024. GhostRace: Exploiting and Mitigating Speculative Race Conditions. In *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [231] Hany Ragab, Alyssa Milburn, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2021. CrossTalk: Speculative Data Leaks Across Cores Are Real. In *Proceedings of the 42nd IEEE Symposium on Security and Privacy (S&P)*, 2021. 1852–1867. <https://doi.org/10.1109/SP40001.2021.00020>
- [232] Hany Ragab, Alyssa Milburn, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2021. CrossTalk: Speculative Data Leaks Across Cores Are Real. In *Proceedings of the 42nd IEEE Symposium on Security and Privacy (S&P)*, 2021. 1852–1867.
- [233] Joseph Ravichandran, Weon Taek Na, Jay Lang, and Mengjia Yan. 2022. PACMAN: attacking ARM pointer authentication with speculative execution. In *Proceedings of the 49th International Symposium on Computer Architecture (ISCA)*, 2022. 685–698. <https://doi.org/10.1145/3470496.3527429>
- [234] REDIS Ltd. REDIS. <https://github.com/redis/redis>

- [235] REDIS. REDIS benchmark. <https://redis.io/docs/management/optimization/benchmarks/>
- [236] Alastair Reid. 2016. Trustworthy specifications of ARM® v8-A and v8-M system level architecture. In *Proceedings of the 2016 Formal Methods in Computer-Aided Design Conferenc (FMCAD)*, 2016. 161–168. <https://doi.org/10.1109/FMCAD.2016.7886675>
- [237] Xida Ren, Logan Moody, Mohammadkazem Taram, Matthew Jordan, Dean M. Tullsen, and Ashish Venkat. 2021. I See Dead μ ops: Leaking Secrets via INTEL/AMD Micro-Op Caches. In *Proceedings of the 48th International Symposium on Computer Architecture (ISCA)*, 2021. 361–374. <https://doi.org/10.1109/ISCA52012.2021.00036>
- [238] Eric Rescorla. 2018. The Transport Layer Security (TLS) Protocol Version 1.3. *RFC 8446*, (2018), 1–160. <https://doi.org/10.17487/RFC8446>
- [239] RISC-V AP-TEE Task Group. Confidential VM Extension (CoVE) for Confidential Computing on RISC-V platforms. https://lists.riscv.org/g/tech-ap-tee/attachment/128/0/riscv-cove_v0.3_comments_QCOM.pdf
- [240] RISC-V International. 2023. RISC-V SBI specification. <https://github.com/riscv-non-isa/riscv-sbi-doc>
- [241] RISC-V International. RISC-V Supervisor Domains Access Protection. <https://github.com/riscv/riscv-smmmtt>
- [242] RISC-V International. RISC-V Open Source Supervisor Binary Interface (OPENSBI). <https://github.com/riscv-software-src/opensbi>
- [243] RISC-V International. RISC-V Supervisor Binary Interface Specification. <https://github.com/riscv-non-isa/riscv-sbi-doc/releases/tag/vv3.0-rc1>
- [244] RISC-V International. RISC-V Profiles. <https://github.com/riscv/riscv-profiles>
- [245] RISC-V International. RISC-V IOPMP Specification. <https://github.com/riscv-non-isa/iopmp-spec>
- [246] Dennis Ritchie and Ken Thompson. 1974. The UNIX Time-Sharing System. *Commun. ACM* 17, 7 (1974), 365–375. <https://doi.org/10.1145/361011.361061>
- [247] Michael Roitzsch, Till Miemietz, Christian von Elm, and Nils Asmussen. 2023. Software-Defined CPU Modes. In *Proceedings of The 19th Workshop on Hot Topics in Operating Systems (HotOS-XIX)*, 2023. 23–29. <https://doi.org/10.1145/3593856.3595894>
- [248] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. Zero trust architecture. *NIST special publication* 800, 207 (2020), 1–52.
- [249] RUST Foundation. 2023. The RUSTonomicon - Meet Safe and Unsafe. <https://doc.rust-lang.org/nomicon/meet-safe-and-unsafe.html>
- [250] RUST Foundation. The RUST programming language. <https://www.rust-lang.org/>
- [251] RUSTSBI Contributors. RUSTSBI: RISC-V Supervisor Binary Interface Library in RUST. <https://github.com/rustsbi/rustsbi>

- [252] G. Mandyam S. Frost T. Fossati. ARM's Confidential Compute Architecture Reference Attestation Token. <https://www.ietf.org/archive/id/draft-ffm-rats-cca-token-00.html>
- [253] Ravi Sahita, Vedvyas Shanbhogue, Andrew Bresticker, Atul Khare, Atish Patra, Samuel Ortiz, Dylan Reid, and Rajnesh Kanwal. 2023. CoVE: Towards Confidential Computing on RISC-V Platforms. In *Proceedings of the 20th ACM International Conference on Computing Frontiers (CF)*, 2023. 315–321.
- [254] Jerome Saltzer and M Frans Kaashoek. 2009. *Principles of computer system design: an introduction*. Morgan Kaufmann.
- [255] Jerome H. Saltzer and Michael D. Schroeder. 1975. The protection of information in computer systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- [256] Michael Sammler, Angus Hammond, Rodolphe Lepigre, Brian Campbell, Jean Pichon-Pharabod, Derek Dreyer, Deepak Garg, and Peter Sewell. 2022. Islaris: verification of machine code against authoritative ISA semantics. In *Proceedings of the ACM SIGPLAN 2022 Conference on Programming Language Design and Implementation (PLDI)*, 2022. 825–840. <https://doi.org/10.1145/3519939.3523434>
- [257] Stephan van Schaik, Andrew Kwong, Daniel Genkin, and Yuval Yarom. 2020. SGAXe: How SGX fails in practice. <https://sgaxe.com/files/SGAXe.pdf>
- [258] Stephan van Schaik, Alyssa Milburn, Sebastian Österlund, Pietro Frigo, Giorgi Maisuradze, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2019. RIDL: Rogue In-Flight Data Load. In *IEEE Symposium on Security and Privacy*, 2019. 88–105. <https://doi.org/10.1109/SP.2019.00087>
- [259] Stephan van Schaik, Marina Minkin, Andrew Kwong, Daniel Genkin, and Yuval Yarom. 2021. CacheOut: Leaking Data on INTEL CPUs via Cache Evictions. In *Proceedings of the 42nd IEEE Symposium on Security and Privacy (S&P)*, 2021. 339–354. <https://doi.org/10.1109/SP40001.2021.00064>
- [260] Tobias Scharnowski, Nils Bars, Moritz Schloegel, Eric Gustafson, Marius Muench, Giovanni Vigna, Christopher Kruegel, Thorsten Holz, and Ali Abbasi. 2022. Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. In *Proceedings of the 31st USENIX Security Symposium*, 2022. 1239–1256.
- [261] Benedict Schlüter and Shweta Shinde. 2025. RMPocalypse: How a Catch-22 Breaks AMD SEV-SNP. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025. 3840–3854. <https://doi.org/10.1145/3719027.3765233>
- [262] Michael Schwarz, Moritz Lipp, Daniel Moghimi, Jo Van Bulck, Julian Stecklina, Thomas Prescher, and Daniel Gruss. 2019. ZombieLoad: Cross-Privilege-Boundary Data Sampling. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019. 753–768. <https://doi.org/10.1145/3319535.3354252>
- [263] Michael Schwarz, Martin Schwarzl, Moritz Lipp, Jon Masters, and Daniel Gruss. 2019. NetSpectre: Read Arbitrary Memory over Network. In *Proceedings of the 24th*

- European Symposium on Research in Computer Security (ESORICS)*, 2019. 279–299. https://doi.org/10.1007/978-3-030-29959-0_14
- [264] Arvind Seshadri, Mark Luk, Ning Qu, and Adrian Perrig. 2007. SecVisor: a tiny hypervisor to provide lifetime kernel code integrity for commodity OSes. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007. 335–350. <https://doi.org/10.1145/1294261.1294294>
 - [265] Syed Muhammad Ali Shah, Maurizio Morisio, and Marco Torchiano. 2012. An Overview of Software Defect Density: A Scoping Study. In *Proceedings of the 19th Asia-Pacific Software Engineering Conference (APSEC)*, 2012. 406–415. <https://doi.org/10.1109/APSEC.2012.93>
 - [266] Jonathan S. Shapiro, Jonathan M. Smith, and David J. Farber. 1999. EROS: a fast capability system. In *Proceedings of the 17th ACM Symposium on Operating Systems Principles (SOSP)*, 1999. 170–185. <https://doi.org/10.1145/319151.319163>
 - [267] Signal. Technology preview: Private contact discovery for Signal. <https://signal.org/blog/private-contact-discovery/>
 - [268] Helgi Sigurbjarnarson, James Bornholt, Emina Torlak, and Xi Wang. 2016. Push-Button Verification of File Systems via Crash Refinement. In *Proceedings of the 12th Symposium on Operating System Design and Implementation (OSDI)*, 2016. 1–16.
 - [269] Quinn Snell, Armin Mikler, and John Gustafson. NetPIPE: A Network Protocol Independent Performance Evaluator. In *Proceedings of the IASTED International Conference on Intelligent Information Management and Systems*,.
 - [270] Livio Soares and Michael Stumm. 2010. FlexSC: Flexible System Call Scheduling with Exception-Less System Calls. In *Proceedings of the 9th Symposium on Operating System Design and Implementation (OSDI)*, 2010. 33–46.
 - [271] Stanford Institute for Human-Centered AI. Artificial INTELLIGENCE Index Report 2025. https://hai.stanford.edu/assets/files/hai_ai-index-report-2025_chapter1_final.pdf
 - [272] STARFIVE. VisionFive 2 RISCv board. <https://www.starfivetech.com/en/site/boards>
 - [273] Julian Stecklina and Thomas Prescher. 2018. LazyFP: Leaking FPU Register State using Microarchitectural Side-Channels. *CoRR* (2018).
 - [274] Udo Steinberg and Bernhard Kauer. 2010. NOVA: a microhypervisor-based secure virtualization architecture. In *Proceedings of the 2010 EuroSys Conference*, 2010. 209–222. <https://doi.org/10.1145/1755913.1755935>
 - [275] Thomas Van Strydonck, Job Noorman, Jennifer Jackson, Leonardo Dias, Robin Vanderstraeten, David Oswald, Frank Piessens, and Dominique Devriese. 2023. CHERI-TrEE: Flexible enclaves on capability machines. In *EuroS&P-8th IEEE European Symposium on Security and Privacy*, 2023.
 - [276] Igor Sysoev and Inc. NGINX. nginx: High Performance Load Balancer, Web Server, & Reverse Proxy. <https://nginx.org/>
 - [277] Tacito Security. iTLB multihit. <https://www.tacitosecurity.com/multihit.html>

- [278] Andrew S. Tanenbaum. 1987. *Operating systems: design and implementation*. Prentice-Hall.
- [279] Runzhou Tao, Jianan Yao, Xupeng Li, Shih-Wei Li, Jason Nieh, and Ronghui Gu. 2021. Formal Verification of a Multiprocessor Hypervisor on ARM Relaxed Memory Hardware. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021. 866–881. <https://doi.org/10.1145/3477132.3483560>
- [280] Peter S. Tasker. 1981. Trusted Computer Systems. In *Proceedings of the 2nd IEEE Symposium on Security and Privacy (S&P)*, 1981. 99–100. <https://doi.org/10.1109/SP.1981.10020>
- [281] Lan Tianyu. Hyper-V patches for supporting AMD SEV-SNP enlightened guest. <https://lore.kernel.org/lkml/20230627032248.2170007-1-ltykernel@gmail.com/T/>
- [282] Tinfoil. Running Private DeepSeek R1 with Verifiable Security. <https://tinfoil.sh/blog/2025-02-03-running-private-deepseek>
- [283] Emina Torlak and Rastislav Bodík. 2013. Growing solver-aided languages with rosette. In *Proceedings of the 2013 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2013. 135–152. <https://doi.org/10.1145/2509578.2509586>
- [284] Daniël Trujillo, Johannes Wikner, and Kaveh Razavi. 2023. Inception: Exposing New Attack Surfaces with Training in Transient Execution. In *Proceedings of the 32nd USENIX Security Symposium*, 2023. 7303–7320.
- [285] Trusted Computing Group. 2015. Trusted Platform Module (TPM) – ISO/IEC 11889. <https://www.iso.org/standard/66510.html>
- [286] Trusted Firmware Project. Trusted Firmware-A. <https://trustedfirmware-a.readthedocs.io/en/latest/>
- [287] Trusted Firmware Project. Trusted Firmware-A, Security Advisories. https://trustedfirmware-a.readthedocs.io/en/latest/security/_advisories/index.html
- [288] Trusted Firmware. TFV-4: Malformed Firmware Update SMC can allow an Unprivileged Caller to Modify Secure Memory Content. https://trustedfirmware-a.readthedocs.io/en/latest/security_advisories/security-advisory-tfv-4.html
- [289] Trusted Firmware. Realm management monitor reference implementation (TF-RMM). <https://github.com/TF-RMM/tf-rmm>
- [290] Chia-Che Tsai, Kumar Saurabh Arora, Nehal Bandi, Bhushan Jain, William Jannen, Jitin John, Harry A. Kalodner, Vrushali Kulkarni, Daniela Oliveira, and Donald E. Porter. 2014. Cooperation and security isolation of library OSe for multi-process applications. In *Proceedings of the 2014 EuroSys Conference*, 2014. 9:1–9:14. <https://doi.org/10.1145/2592798.2592812>
- [291] Chia-Che Tsai, Bhushan Jain, Nafees Ahmed Abdul, and Donald E. Porter. 2016. A study of modern LINUX API usage and compatibility: what to support when you're supporting. In *Proceedings of the 2016 EuroSys Conference*, 2016. 16:1–16:16. <https://doi.org/10.1145/2901318.2901341>

- [292] Richard Uhlig, Gil Neiger, Dion Rodgers, Amy L. Santoni, Fernando C. M. Martins, Andrew V. Anderson, Steven M. Bennett, Alain Kägi, Felix H. Leung, and Larry Smith. 2005. INTEL Virtualization Technology. *Computer* 38, 5 (2005), 48–56.
- [293] US Department of Defense. Department Of Defense Trusted Computer System Evaluation Criteria. <https://irp.fas.org/nsa/rainbow/std001.htm>
- [294] Alexa VanHattum, Daniel Schwartz-Narbonne, Nathan Chong, and Adrian Sampson. 2022. Verifying Dynamic Trait Objects in RUST. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022. 321–330. <https://doi.org/10.1109/ICSE-SEIP55303.2022.9794041>
- [295] Amit Vasudevan, Sagar Chaki, Limin Jia, Jonathan M. McCune, James Newsome, and Anupam Datta. 2013. Design, Implementation and Verification of an eXtensible and Modular Hypervisor Framework. In *IEEE Symposium on Security and Privacy*, 2013. 430–444. <https://doi.org/10.1109/SP.2013.36>
- [296] Anthony Velte and Toby Velte. 2009. *Microsoft virtualization with Hyper-V*. McGraw-Hill, Inc.
- [297] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale cluster management at Google with Borg. In *Proceedings of the 2015 EuroSys Conference*, 2015. 18:1–18:17. <https://doi.org/10.1145/2741948.2741964>
- [298] Jose Rodrigo Sanchez Vicarte, Michael Flanders, Riccardo Paccagnella, Grant Garrett-Grossman, Adam Morrison, Christopher W. Fletcher, and David Kohlbrenner. 2022. Augury: Using Data Memory-Dependent Prefetchers to Leak Data at Rest. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P)*, 2022. 1491–1505. <https://doi.org/10.1109/SP46214.2022.9833570>
- [299] Jose Rodrigo Sanchez Vicarte, Pradyumna Shome, Nandeeka Nayak, Caroline Trippel, Adam Morrison, David Kohlbrenner, and Christopher W. Fletcher. 2021. Opening Pandora's Box: A Systematic Study of New Ways Microarchitecture Can Leak Private Data. In *Proceedings of the 48th International Symposium on Computer Architecture (ISCA)*, 2021. 347–360. <https://doi.org/10.1109/ISCA52012.2021.00035>
- [300] Stavros Volos, Cédric Fournet, Jana Hofmann, Boris Köpf, and Oleksii Oleksenko. 2024. Principled Microarchitectural Isolation on Cloud CPUs. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024. 183–197. <https://doi.org/10.1145/3658644.3690183>
- [301] Carl A. Waldspurger. 2002. Memory Resource Management in VMWARE ESX Server. In *Proceedings of the 5th Symposium on Operating System Design and Implementation (OSDI)*, 2002.
- [302] Andrew Waterman, Krste Asanović, and John Hauser. 2021. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20211203. <https://github.com/riscv/riscv-isa-manual>
- [303] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capicum: Practical Capabilities for UNIX. In *Proceedings of the 19th USENIX Security Symposium*, 2010. 29–46.

- [304] Ofir Weisse, Valeria Bertacco, and Todd M. Austin. 2017. Regaining Lost Cycles with HotCalls: A Fast Interface for SGX Secure Enclaves. In *Proceedings of the 44th International Symposium on Computer Architecture (ISCA)*, 2017. 81–93. <https://doi.org/10.1145/3079856.3080208>
- [305] Sander Wiebing, Alvis de Faveri Tron, Herbert Bos, and Cristiano Giuffrida. 2024. InSpectre Gadget: Inspecting the Residual Attack Surface of Cross-privilege Spectre v2. In *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [306] Wikipedia contributors. 2024. XZ Utils backdoor — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/XZ_utils_backdoor
- [307] Johannes Wikner and Kaveh Razavi. 2022. RETBLEED: Arbitrary Speculative Code Execution with Return Instructions. In *Proceedings of the 31st USENIX Security Symposium*, 2022. 3825–3842.
- [308] Maurice Vincent Wilkes and Roger Michael Needham. 1979. The Cambridge CAP computer and its operating system. (1979).
- [309] Nils Wistoff, Andreas Kuster, Michael Rogenmoser, Robert Balas, Moritz Schneider, and Luca Benini. 2023. Protego: A Low-Overhead Open-Source I/O Physical Memory Protection Unit for RISC-V. In *Proceedings of the 1st Safety and Security in Heterogeneous Open System-on-Chip Platforms Workshop (SSH-SoC 2023)*, 2023.
- [310] Jonathan Woodruff, Robert N. M. Watson, David Chisnall, Simon W. Moore, Jonathan Anderson, Brooks Davis, Ben Laurie, Peter G. Neumann, Robert M. Norton, and Michael Roe. 2014. The CHERI capability model: Revisiting RISC in an age of risk. In *Proceedings of the 41st International Symposium on Computer Architecture (ISCA)*, 2014. 457–468. <https://doi.org/10.1109/ISCA.2014.6853201>
- [311] Wenjie Xiong and Jakub Szefer. 2022. Survey of Transient Execution Attacks and Their Mitigations. *ACM Comput. Surv.* 54, 3 (2022), 54:1–54:36. <https://doi.org/10.1145/3442479>
- [312] Ying Yan, Changzheng Wei, Xuepeng Guo, Xuming Lu, Xiaofu Zheng, Qi Liu, Chenhui Zhou, Xuyang Song, Boran Zhao, Hui Zhang, and Guofei Jiang. 2020. Confidentiality Support over Financial Grade Consortium Blockchain. In *SIGMOD Conference*, 2020. 2227–2240. <https://doi.org/10.1145/3318464.3386127>
- [313] Ganxiang Yang, Wei Qiang, Yi Rong, Xuheng Li, Fanqi Yu, Jason Nieh, and Ronghui Gu. 2026. Highly Automated Verification of Security Properties for Unmodified System Software. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXXI)*, 2026. 912–928. <https://doi.org/10.1145/3779212.3790171>
- [314] Zachary Yedidia. 2024. Lightweight Fault Isolation: Practical, Efficient, and Secure Software Sandboxing. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXIX)*, 2024. 649–665. <https://doi.org/10.1145/3620665.3640408>
- [315] Bennet Yee, David Sehr, Gregory Dardyk, J. Bradley Chen, Robert Muth, Tavis Ormandy, Shiki Okasaka, Neha Narula, and Nicholas Fullagar. 2009. Native Client: A Sandbox for Portable, Untrusted x86 Native Code. In *Proceedings of the 30th IEEE*

- Symposium on Security and Privacy (S&P)*, 2009. 79–93. <https://doi.org/10.1109/SP.2009.25>
- [316] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2019. The True Cost of Containing: A gVisor Case Study. In *Proceedings of the 11th workshop on Hot topics in Cloud Computing (HotCloud)*, 2019.
 - [317] Arseniy Zaostrovnykh, Solal Pirelli, Rishabh R. Iyer, Matteo Rizzo, Luis Pedrosa, Katerina J. Argyraki, and George Candea. 2019. Verifying software network functions with no verification expertise. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019. 275–290.
 - [318] The ZEPHYR Project. ZEPHYR Project: Scalable Real-Time Operating System (RTOS). <https://www.zephyrproject.org/>
 - [319] Chuqi Zhang, Rahul Priolkar, Yuancheng Jiang, Yuan Xiao, Mona Vij, Zhenkai Liang, and Adil Ahmad. 2025. Erebor: A Drop-In Sandbox Solution for Private Data Processing in Untrusted Confidential Virtual Machines. In *Proceedings of the 2025 EuroSys Conference*, 2025. 1210–1228. <https://doi.org/10.1145/3689031.3717464>
 - [320] Fengzhe Zhang, Jin Chen, Haibo Chen, and Binyu Zang. 2011. CloudVisor: retrofitting protection of virtual machines in multi-tenant cloud with nested virtualization. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*, 2011. 203–216.
 - [321] Ruiyi Zhang, Lukas Gerlach, Daniel Weber, Lorenz Hetterich, Youheng Lü, Andreas Kogler, and Michael Schwarz. 2024. CacheWarp: Software-based Fault Injection using Selective State Reset. In *Proceedings of the 33rd USENIX Security Symposium*, 2024.
 - [322] Ruiyi Zhang, Taehyun Kim, Daniel Weber, and Michael Schwarz. 2023. (M)WAIT for It: Bridging the Gap between Microarchitectural and Architectural Side Channels. In *Proceedings of the 32nd USENIX Security Symposium*, 2023. 7267–7284.
 - [323] Yiming Zhang, Yuxin Hu, Zhenyu Ning, Fengwei Zhang, Xiapu Luo, Haoyang Huang, Shoumeng Yan, and Zhengyu He. 2023. SHELTER: Extending ARM CCA with Isolation in User Space. In *Proceedings of the 32nd USENIX Security Symposium*, 2023. 6257–6274.
 - [324] Shixuan Zhao, Mengyuan Li, Yinqian Zhang, and Zhiqiang Lin. 2022. vSGX: Virtualizing SGX Enclaves on AMD SEV. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P)*, 2022. 321–336. <https://doi.org/10.1109/SP46214.2022.9833694>
 - [325] Zirui Neil Zhao, Adam Morrison, Christopher W. Fletcher, and Josep Torrellas. 2024. Last-Level Cache Side-Channel Attacks Are Feasible in the Modern Public Cloud. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-XXIX)*, 2024. 582–600. <https://doi.org/10.1145/3620665.3640403>
 - [326] Zirui Neil Zhao, Adam Morrison, Christopher W. Fletcher, and Josep Torrellas. 2024. Everywhere All at Once: Co-Location Attacks on Public Cloud FaaS. In *Proceedings of the 29th International Conference on Architectural Support for Programming*

Languages and Operating Systems (ASPLOS-XXIX), 2024. 133–149. <https://doi.org/10.1145/3617232.3624867>

- [327] Ziqiao Zhou, Anjali, Weiteng Chen, Sishuai Gong, Chris Hawblitzel, and Weidong Cui. 2024. VeriSMo: A Verified Security Module for Confidential VMs. In *Proceedings of the 18th Symposium on Operating System Design and Implementation (OSDI)*, 2024. 599–614.
- [328] Ziqiao Zhou, Yizhou Shan, Weidong Cui, Xinyang Ge, Marcus Peinado, and Andrew Baumann. 2023. Core slicing: closing the gap between leaky confidential VMs and bare-metal cloud. In *Proceedings of the 17th Symposium on Operating System Design and Implementation (OSDI)*, 2023. 247–267.

Charly Castes

charly.castes@epfl.ch | +33 6 85 93 22 12 | charlycst.github.io

EDUCATION

École Polytechnique Fédérale de Lausanne (EPFL) Switzerland <i>Ph.D. in Computer Science, advised by Edouard Bugnion</i>	Sep 2021 - Present
École Polytechnique Fédérale de Lausanne (EPFL) Switzerland <i>Master in Computer Science</i>	Sep 2019 - Sep 2021
École Polytechnique France <i>Master in Data Science</i>	Sep 2016 - Sep 2020

PROFESSIONAL EXPERIENCE & ACADEMIC VISITING

Columbia University , <i>Visiting Researcher</i> New York, NY Host: Jason Nieh	Sep 2025 - Dec 2025
Google (SRG) , <i>Visiting Researcher</i> Zurich, Switzerland Host: Andrew Baumann	Jun 2023 - Dec 2023
Inria , <i>Visiting Researcher</i> Bordeaux, France Hosts: Emmanuel Agullo, Olivier Aumage, and Emmanuelle SAILlard	Mar 2021 - Aug 2021
Joko (fintech startup), <i>Software Engineer</i> Paris, France	Apr 2019 - Sep 2019
Lycée de la Borde Basse , <i>Teaching full time</i> Castres, France	Sep 2016 - Mar 2017

PUBLICATIONS

TYCHE: Composable Isolation as a Foundation to Manage Trust in the Cloud <i>Charly Castes*, Adrien Ghosn*, Neelu S. Kalani, Yuchen Qian, Marios Kogias, Edouard Bugnion</i>	EuroS&P'26
The Design and Implementation of a Virtual Firmware Monitor <i>Charly Castes, François Costa, Neelu S. Kalani, Timothy Roscoe, Nate Foster, Thomas Bourgeat, Edouard Bugnion</i>	SOSP'25 📄📄
Lightweight Hypervisor Verification: Putting the Hardware Burger on a Diet <i>Charly Castes, François Costa, Nate Foster, Thomas Bourgeat, Edouard Bugnion</i>	HotOS'25 📄📄
Sharing is leaking: blocking transient execution attacks with core-gapped CVMs <i>Charly Castes, Andrew Baumann</i>	ASPLOS'24 📄📄
Kicking the Firmware Out of the TCB with the MIRALIS Virtual Firmware Monitor <i>Charly Castes, Neelu S. Kalani, Sofia Saltovskaia, Noé Terrier, Abel Vexina Wilkinson, Edouard Bugnion</i>	KISV'24 📄📄
Dynamic Linkers Are the Narrow Waist of Operating Systems <i>Charly Castes, Adrien Ghosn</i>	PLOS'23 📄📄
Creating Trust by Abolishing Hierarchies <i>Charly Castes*, Adrien Ghosn*, Neelu S. Kalani, Yuchen Qian, Marios Kogias, Mathias Payer, Edouard Bugnion</i>	HotOS'23 📄📄
Attestable Software Versioning for Stateful Confidential Computing <i>Charly Castes, Edouard Bugnion</i>	SysTex'22 📄📄
Decentralized In-Order Execution of Sequential Task-Based Code for Shared-Memory Arch' IPDPSW'22 <i>Charly Castes, Emmanuel Agullo, Olivier Aumage, Emmanuelle SAILlard</i>	📄
Enclosure: Language-Based Restriction of Untrusted Libraries Adrien Ghosn, Marios Kogias, Mathias Payer, James R. Larus, Edouard Bugnion (<i>Note: Worked on the project as a master student, my work got included in the paper</i>)	ASPLOS'21 📄

SERVICE

Journal of System Research (website 🔗) <i>Assistant Editor</i>	2023 - Present
EPFL IC School Council <i>Scientific Staff Representative</i>	2023 - 2024
EPFL IC Graduate Student Association <i>Committee Member</i>	2022 - 2024
EuroSys 2023 <i>Artifact Evaluation Committee</i>	2022-2023
EuroSys 2022 <i>Artifact Evaluation Committee</i>	2021-2022

TEACHING

At EPFL, I was teaching assistant for the following classes:

Object Oriented Programming Prof: Jean-Cédric Chappelier	2025
Decentralized System Engineering Profs: Bryan Ford, Pierluca Borsò-Tan	2024
Software Enterprise: From Ideas to Products Profs: George Candea, Edouard Bugnion, Pierluca Borsò-Tan	2024
Decentralized System Engineering Prof: Pierluca Borsò-Tan	2023
Global Issues: Communication Profs: James Larus, Anna Jobin	2023
System Engineering Project Profs: Jean-Cédric Chappelier, Edouard Bugnion	2022
Foundation of Software Prof: Martin Odersky	2021

I occupied a full time teaching position from September 2016 to March 2017. I taught mathematics, physics, chemistry, and mechanical engineering to high school students.

SUPERVISED STUDENT PROJECTS

Lucie Mermoud Fold, a Dynamic Linker framework written in RUST	  2025
Noé Terrier - Fold, a Dynamic Linker framework written in RUST - Virtualizing M-mode interrupts - Exploring Kernel Drivers in RUST	  2025  2024   2023
François Costa Verification and deployment of MIRALIS	  2024
Frederic Khayat Porting the KEYSTONE security monitor to MIRALIS	  2024
Sofia Saltovskaia Porting MIRALIS to the VisionFive 2	 2024
Abel Vexina Wilkinson Firmware virtualization with Mirage	 2024
Maëlys Billon - A Communication Library for TEEs - Trusted Boot with INTEL TXT	  2023  2022
Filippo Costa Verifying RUST Code with Serval	  2022
David Desboeufs A Finite Interface Frame Allocator	  2022
Michael Paper Verified Page Tables Manipulation	  2022

TECHNICAL SKILLS

Languages: Expert: RUST (6+ years experience), Proficient: C, Go, Python, TypeScript

Expertise: Bare-Metal programming, Operating Systems, Virtualization, TEE, Compilers, LINUX, KVM

Technology: x86_64, INTEL VT-x, VT-d, TXT, MPK, SGX, ARM CCA, RISC-V

LANGUAGES

Fluent: French (native, french citizen), English